
Forest
PPL

Table of Contents

Introduction 7
ForestPPL Overview 7
Fundamentals 10
Trees 12
Comments 17
Configuration 18
Identifiers and Scope Manager 18
Public and private variables and functions 20
Compound statements (blocks) 22
Libraries 23
Keywords 24
HTML Reference 29
Service Commands 30
help 30
version 30
cls 30
shell 30
init 31
code 31
showcode 31
readcode(rc) 31
createpplcode (cpc) 32
display (d) 33
displaynodes (dn) 33
dstree 35
printscope 36
suspend and resume 37
debugppl (dbg) 37
traceppl 37
recreate 37
log 38
checksyntax 38

exit 38
sumdata 39
withset 40
start, stop 40
using 41
Special Commands 42
import 42
importlist (il) 42
eval 42
length 43
isexist 44
isdigits 44
Isinteger 45
Isalldigits 45
isallinteger 45
iseven, isodd 45
del 46
calc 48
sleep 48
getbykey (getk) 49
getbyvalue (getv) 49
set 49
setkvp 52
getvalue (get) 53
getname 53
gettoken 54
type 55
Nodes and Leaves 56
createnode 56
copynode 56
getnodes 58
getleaves 59
definestruct 61

insertstruct 62
createstruct 62
instancename 65
functionstruct 67
dstruct 70
dd 70
Arithmetic operators 72
Logical operators 72
Conditional ?: operator 72
Variables and Arrays 73
var 73
const 74
setconst 75
array 76
resize 79
array.push 81
array.pop 82
array.reverse 83
array.shift 83
array.remove 83
array.clear 84
array.unshift 84
array.insert 84
array.slice 85
array.sum 86
array.copy 87
array.filtercopy 87
array.min 88
array.max 88
array.average 89
array.first 89
array.last 89
array.concat 89

array.sort 90
Storage 91
storage 91
sinit 92
sget 93
sset 93
swrite 94
sinfo 95
sresize 96
sdimsize 97
Backup & Recovery 99
savedata | (sd) 99
readdata (rd) 100
Control Flow 101
if, else 101
switch, case, default 103
loop,do 105
setloopend 106
for 107
break 108
continue 109
Input and Output 110
write 110
write#, writeline 110
writearray 112
readline 113
Functions 114
function 118
call 127
return 129
getresult 130
funclist 130
funcname 131

argc 131
getargname 132
finally and failure blocks 133
Delegates and callbacks 134
delegate 134
dlgtinstance 135
dlgtset 136
dlgtcall 136
callback 137
VS Code support 138
Additional functionalities 140
Math 141
String 143
Directory 148
Queue 150
Stack 152
Dictionary 154
Convert 157
File 159
Random 161
Console 163
Vector 165
Matrix 166
DataFrame 168
StorageLib 186
NumPPL 198
Requests 207
Structure of User's DLL 218
Error detection 222
Examples of code 226
References 231

Introduction

ForestPPL Overview

ForestPPL is a console application developed in C++ using Microsoft Developer Studio, without relying on third-party packages. It operates on **PPL.DLL**, an implementation of the **Parenthesis Programming Language (PPL)**. Originally written in C#, the PPL code was translated into C++. PPL is an interpreted programming language in which all elements (statements, parameters, blocks) are enclosed in parentheses, resembling LISP syntax.

It includes a preprocessor to simplify code writing and reduce parentheses usage. PPL source code (**format: .scr**) is converted into an intermediate representation (**format: .ppl**) for immediate execution.
In this document, program text examples are shown as follows

```
>var x = 0
>d x
```

the results of running the examples are as follows

```
-----Variables  and arrays-----
-L0       x                              [0]
```

Sign **>** used as prompt in interactive mode.

Key features of PPL

1. **Extensibility:**

- Combines C++ functionalities with custom user libraries by creating DLLs via the CodeGen.exe utility.
- Enables adding full PPL functionality to any user application.

2. **Modes:**

- **ppl (base) mode**: Syntax similar to LISP, using prefix notation for mathematical and logical expressions.
  Examples:

```
var (x [0]);
set(x) (+(1)(2));
set (x)( -(0)(+(3)(2)));  // infix notation: x = -(3 + 2);
if(==(x) (1)) …
```

- **scr (preprocessor) mode**: Syntax similar to C, using infix notation for mathematical and logical expressions.
  Examples:

```
var x = 0;
set x = 1 + 2;  // or x = 1 + 2;
if (x == 1)…
```

  Commands in format scr may be used on the left side of the expression(example).

3. **Parsing:**
   - Code written in scr mode is translated into ppl mode before execution. Each parser level creates a syntax tree.
   - ForestPPL calls PPL API functions, which can also be used in other applications.

4. **Execution Modes:**
   - The operating mode (scr or ppl) is determined by the file extension or a command in the code itself.
   - scr mode simplifies coding by eliminating the need for enclosing statements in parentheses.
   - Default mode is set in **Configuration.Data**: **(Code [ppl])**

Built-In Statements and Compound Blocks

Preprocessor includes the **following statements** –
var, const, resize,
storage statements,
array statements,
set, setkvp,cpc, sumdata,
savedata,readdata,array,
write#, writearray,call,
createstruct, insertstruct,
delegate, dlgtinstance,dlgtset,dlgtcall,callback,
setloopend,savedata,readdata

and following **compound statements (blocks)** –
definestruct,function, for, if, else, switch, case, default, finally,failure.

All ppl mode statements may be also added to scr code in format ppl if these statements do not have scr mode.

Data Handling

Unicode Storage: All data is saved as Unicode symbols, while digital data is converted into strings.
Examples:

```
set  x = 5.2; saved as string "5.2"
```

Boolean values are saved as strings - "**True**" and "**False**":

```
set x = True;
```

Scope Management (Scope Manager)

ForestPPL utilizes a dedicated Scope Manager to handle memory allocation and variable lifecycles efficiently throughout the execution of a script.

- **Hierarchical Tracking**: Maintains strict tracking of execution scopes, including Global, Local, Function, and Loop boundaries.

- **Variable Lifecycle**: Handles the dynamic registration, lookup, modification, and safe removal of variables, constants, arrays, and complex structures.
- **Context Isolation**: Ensures local function variables remain isolated while safely passing arguments and resolving pointers from caller to callee scopes.
- **Dynamic Parameter Binding**: Supports advanced argument mapping across boundaries, including handling optional omitted parameters (using the # marker) and resolving loop counters.

Script Execution Stages

Script execution involves:

- Creating an array of statements (simple or compound).
- Generating a syntax tree for each statement.
- Traversing all nodes in the syntax tree and executing the associated procedures.
- Repeating the process recursively for each statement in compound blocks.
- Continuing the process for each statement in the array until completion.
- The dstree command allows users to visualize syntax trees for each script statement.

Debugging Features

ForestPPL includes commands for debugging:

- **dbg**: Enables debugging mode.
- **traceppl**: Traces code execution.
- **suspend and resume**: Temporarily pauses script execution and resumes it.
- **start and stop**: Measures the execution duration and outputs results in milliseconds.
- **On-the-fly code generation**: The script can analyze data, collect new PPL commands from them (in the form of strings) and immediately execute them.
-
- **Dynamic patches**: The ability to redefine variables, change constants or even operating logic right during program execution without restarting the kernel.
-

Interactive Debugging: If you want to make a REPL console (like Python or JavaScript) for your engine, a global eval like this would be the ideal heart of it, allowing the user to tamper with memory on the fly. ForestPPL doesn't treat programmers like children from whom sharp tools must be hidden. It grants absolute control over memory and code at runtime, trusting their expertise (see command "eval").

Fundamentals

The code is executed in the order it is written without main function.
As is customary in many programming language guides the first program is:
write("Hello World!");

Another example with using Console and String libraries:
See file examples\console\colors.scr:

```
import Console;
import String;

var text = "##Hello World!##";
array colors[] = {"Black",
                  "DarkBlue","DarkGreen","DarkCyan",
                  "DarkRed","DarkMagenta","DarkYellow",
                  "Gray","DarkGray","Blue","Green",
                  "Cyan","Red","Magenta","Yellow","White"};

array ArrayChars;
call String.ToCharArray(text, ArrayChars);

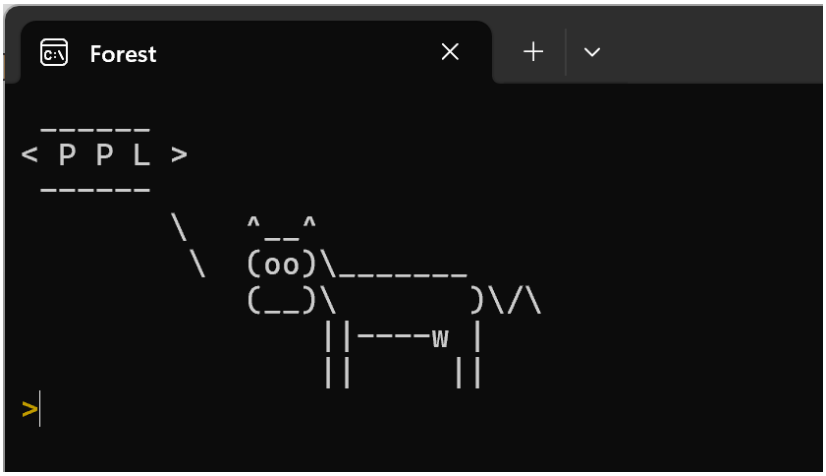
for(i,0,length(text))
{
    Console.ForegroundColor(colors[i]);
    Console.Write(ArrayChars[i]);
}
Console.DefaultColors();
var pos = Console.GetCursorPosition();
array separator[] = {comma,space};
array pos_array;
call String.Split(pos,separator,pos_array);
pos_array[0] = 0;                    // column
pos_array[1] = pos_array[1] + 1;     // row
call Console.SetCursorPosition(pos_array[0], pos_array[1]);
write();   // to new line
```



The following 4 samples illustrate the possibilities of PPL:

```
1.
>rc examples\cowsay\cowsay1.scr
array cow[] =
{
" ______",
"< P P L >",
" ------ ",
"        \   ^__^",
"         \  (oo)\_______",
"            (__)\       )\/\",
"                ||----w |",
"                ||     ||"
};

Console.Clear();
for(i,0,length(cow))
{
   call Console.SetCursorPosition(0,i);
   call Console.Write(cow[i]);
}
call Console.SetCursorPosition(0,length(cow));
```



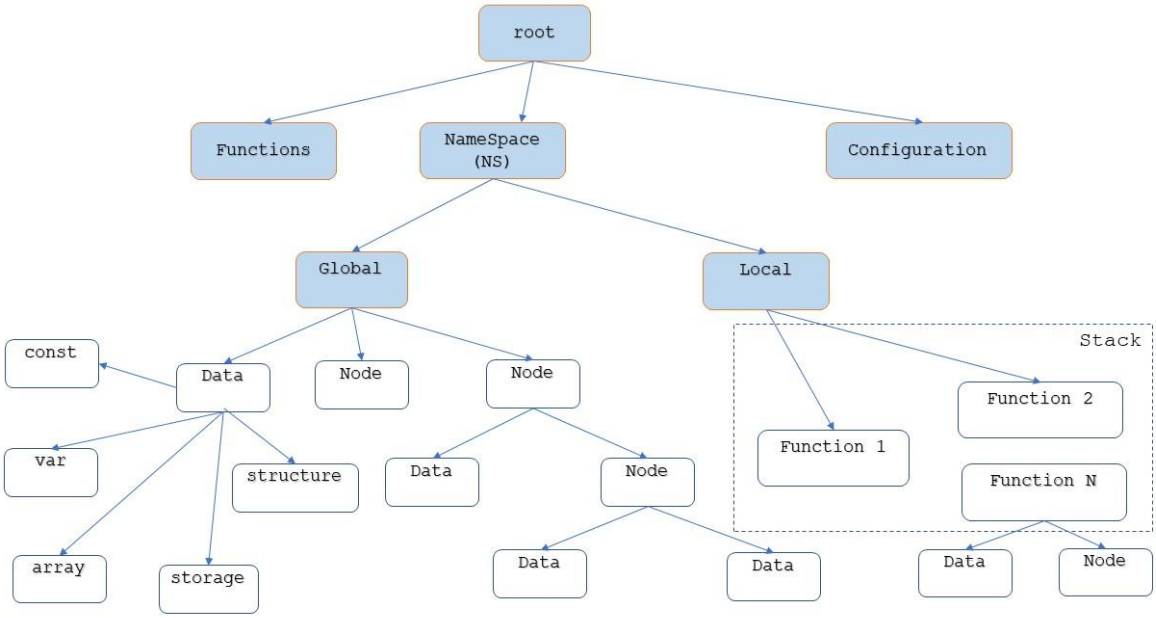

```
2.  Added loading of second file, function call, creation node and
array under this node
>rc examples\cowsay\cowsay2.scr

3. Added second operator 'for' to move cow
>rc examples\cowsay\cowsay3.scr

4. Added passing data as arguments when calling command readcode
>rc examples\cowsay\cowsay4.scr 1 1 10
```

Trees

Functions and data are stored in PPL as several kinds of Trees –
root, NS, Functions, Configuration and may be displayed on Screen, saved and restored.



Blue nodes are created automatically when ForestPPL.exe) loads or re-created when command **init** is called.

By default Tree **Functions** is filling from file **Functions\ CommonFunctions.ppl**, defined as "**default_loaded_functions**" in file **Configuration.data**. This may be changed by the user to another file. To display its contents, perform:

```
>display Functions;
```

User may perform command **readcode (rc)** to read files with user's functions and add these functions to Tree **Functions** or to node, created under Tree **Functions**.
Tree **Functions (or nodes under** Tree **Functions) saves only functions, not data (see examples 6-8 in** [function](#)**)**.
Examples:

```
>d Functions
-----Functions-----
-N2     Sum     [function]
---L0   result1
---L1   n1
---L2   n2
---N1   #       [internal_block]
```

```
-----N2 set
-------L0       result1
-------N3       +
---------L0     n1
---------L1     n2
```

Adding functions to node under Tree Functions:

```
>createnode Functions.Calc;
function
(
  Functions.Calc.Sum(result1)(n1)(n2)
  (
     (set(result1)(+(n1)  (n2)))
  )
);
>d Functions
-----Functions-----
-N2     Calc    [Node]
---N3   Sum     [function]
-----L0 result1
-----L1 n1
-----L2 n2
-----N1 #       [internal_block]
-------N2       set
---------L0     result1
---------N3     +
-----------L0   n1
-----------L1   n2
```

Tree **Configuration** is filling from file **Configuration.data**, to display its contents perform:

```
>display Configuration;
(default_loaded_functions  [Functions\CommonFunctions.ppl])
   //(UserFunctions1       [])
   //(UserFunctions2       [])
   //(UserFunctions3       [])
   //
   (Code                      [ppl])
   (debugppl                  [no])
   (delete_all_in_readcode    [yes])
   (log                       [no])
   (stay_interactive          [no])     // for ForestPPL.exe

   (ThreadPriorityHighest     [yes])

   //(UserFunctions1           [Functions\printchar.ppl])
   (UserImport1               [Directory])
   (UserImport2               [Math])
   (UserImport3               [String])
   (UserImport4               [File])
   (UserImport5               [Console])

   //(UserImport6             [Convert])
   //(UserImport8             [Stack])
```

```
   //(UserImport9                [Queue])
   //(UserImport10               [DataFrame])
   //(UserImport11               [Dictionary])
```

If **ThreadPriorityHighest = yes** **(Win32 API)** processing thread runs ahead of normal and above-normal threads in the process:

```
if(ThreadPriorityHighest == "yes")
   SetThreadPriority(hProcessingThread, THREAD_PRIORITY_HIGHEST);
 else
   SetThreadPriority(hProcessingThread, THREAD_PRIORITY_NORMAL);
```

Any public variables (var, const, array, storage and node) are saved in Tree **Global** as common data for all functions and for code without functions(in "main function").
Public functions are available to functions from any node in Tree **Global**.
Private variables and functions are available to functions in the node, that they belong to only. Full name variables and functions include name of node.
Example:

```
createnode N1;
   function
   (
      N1.f()
      (
         (write("public function N1.f"))
         (N1._f())
      )
   );

   function
   (
      N1._f()     // private
      (
         (write("private function N1._f"))
         (write(N2.x))
         //(write(N2._x))     // Error: [GetValue] [N2._x] private
                                  object, no access
         //(N2._f())          // Error: [Traversal] [N2._f] private
                                  function, no access
      )
   );

createnode N2;
   var(N2.x["public var N2.x"]);
   var(N2._x["private var N2._x"]);
   function
   (
      N2._f()    // private
      (
         (write("private function N2._f"))
      )
```

```
   );

N1.f();
write(N2.x);
//write(N2._x);    // Error: [GetValue] [N2._x] private object, no
                        access
```

Variables for functions are created in Tree **Local**, to display its contents perform in function:
```
>display Local
```
When exiting a function, its variables are deleted.

For illustration difference between modes scr and ppl consider the following examples:

```
>rc Examples\scr\for.scr
=== scr code for preprocessor ===
var begin = 0;
var end = 3;
for(i,begin + 1,end + 1,1)
{
  write(i);
}
=== generated by preprocessor ppl code ===
>var (begin[0]);
>var (end[3]);
>loop (i)( + ( begin )( 1 ) )( + ( end )( 1 ) )( 1 )
(
  do
  (
    (write(i))
  )
);
1
2
3
```

Statement terminator '**;**' always follows after each type of statements in scr mode.
In ppl mode statement terminator ';' does not follow after statements within compound statements(blocks) – loop, switch,if,function.

Examples in ppl mode:
```
loop (i)(0)(3)(1)
(
   do
   (
     (write(hello))
     (write(world))
   )
);
```

Comments

Two kinds of commentaries are possible:
/*…*/ - for several lines of code
and
// - for one line of code or part of line.

Configuration

Configuration is defined in the file **Configuration.data**, meaning of its members is explained in this tutorial.

Identifiers and Scope Manager

Names of nodes, variables, arrays, storage and functions contain any symbols, first symbol is any upper or lower case letter or any of the following symbols: **_$#** , but not a digit. Variables with first symbol "_" in name are hidden or private variables (see hidden variables).
A reserved name "**result**" is used for internal purposes in functions and cannot be used to create any variables.
Name "**all**" can not be used (see cmd del).
Length of identifiers is not limited. Do not set keywords and names of Libraries as identifiers.
When data is created, its full name and saved address are added to **Scope Manager**.
The **Scope Manager** intelligently manages variable lifecycles, memory cleanup, and visibility boundaries, ensuring the safety and predictability of your code.

• **Block-level scopes**: Variables and arrays created within a block (such as a loop or conditional statement) exist only within that block. Upon exiting the block, all local variables are immediately and safely deleted from memory.

• **Function isolation**: Each function call creates a strict, isolated environment. Local variables defined within a function cannot conflict with variables in the parent code. This ensures function independence and the absence of side effects, while maintaining access to global variables.

• **Intelligent variable overriding**: Variable names may be safely reused. If a local variable has the same name as a global variable, the local variable temporarily "overrides" (replaces) the global variable within its specific block, without changing the original global data.

• **Automatic conflict prevention**: The engine performs instant, real-time checks when declaring new variables. This prevents accidental name conflicts within a single scope and ensures automatic memory resizing if a variable is intentionally re-created.

• **No memory leaks**: The user never has to manually delete variables. The **Scope Manager** tracks the exact lifecycle of each data structure and immediately frees memory upon scope exit.

Symbolic values are enclosed in quotation marks, to include a quotation mark in a symbolic expression, precede it with backslash.
Example:

```
"123\"qwe" => "123"qwe"
```

Backslash before the last quote mark it is backslash, not quote mark.

```
"123\qwe\\" => "123\qwe\"
```

Public and private variables and functions

Variables, constants, arrays, storage and functions, whose names start with **underscore** are private, all other are public.
Examples:
1.

```
>code ppl;
>createnode N1;
>var(N1.x["public var N1.x"]);
>var(N1._x["private var N1._x"]);
function
(
   N1._f()    // private
   (
      (write("private function N1._f"))
      (write(N1.x))
      (write(N1._x))
   )
);
function
(
   N1.f()
   (
      (N1._f())
   )
);
N1.f();
```

```
code: ppl
private function N1._f
public var N1.x
private var N1._x
```

2.

```
>code scr;
>createnode N1;
>var N1.x = "public var N1.x";
>var N1._x = "private var N1._x";
function N1._f()    // private
{
    write("private function N1._f");
    write(N1.x);
    write(N1._x);
}
function N1.f()  // public
{
   N1._f();
}
N1.f();
```

Error occurs when re-creating a variable, it is possible to delete this variable and to create again:

```
>code scr;
>var x;
>var x;
Error: [FuncCreateVariables] name [x] already exists
>del x;
>var x;
```

To allow re-creating use command **recreate yes**

```
>code scr;
>var x;
>recreate yes;
>var x;   // re-creation
```

Compound statements (blocks)

Compound statements include one or several statements enclosed in curly brackets:

```
if (x == 1)
{
    write("COMPOUND");
    write("STATEMENTS");
}
else
{
    write("compound");
    write("statements");
}
```

If compound statements "**for**", "**if**" contain only one **not compound statement** in curly brackets it is possible to omit brackets:

```
if (x == 1)
    write("COMPOUND");
else
    write("statements");
```

```
if (x == 1)                          // right
{
    if (y == 2)
       write("right sample");
}
```

```
if (x == 1)                          // wrong
   if (y == 2)
   {
       write("wrong sample");
   }
```

Libraries

Default name of library is **Main**, it loads always when ForestPPL.exe starts. It is possible to set in file **Configuration.data** as **"UserImportN"** names of additional libraries initialization loaded . To display list of loaded libraries perform:

```
>importlist;
Main
Directory
Math
```

To display contents of any library perform:
<name of library>.help
or **help** for Main library

Example:

```
>Directory.help;
help
GetFiles
GetDirectories
SetCurrentDirectory
GetCurrentDirectory
```

To get short information about any library function perform:
<name of library>.help(function name)

```
>Math.help(Sinh)
     Returns the hyperbolic sine of the specified angle:
     Math.Sinh(double value)
```

```
For Main Library help  or ?:
>? d
        display | d [root|NS| Functions|Local|node name]
        display NS.namespace.name]
```

Keywords

Keyword formats are defined in this tutorial, all format are defined for **ppl mode** by default. Additionally defined format for **scr mode** for some keywords. All keywords are divided into 11 groups and presented below:

Service Commands

help, version, cls, shell, init, code, showcode, readcode, fdreadcode, createpplcode, display, displaynodes, dn, dstree, suspend, resume, debugppl, traceppl, recreate, log, exit, createcodeppl, sumdata, printscope,start, stop, checksyntax

Special Commands

import, importlist, eval, length, calc, sleep, isexist, isdigits, isinteger, isalldigits, isallinteger, iseven, isodd, del, getbykey, getbyvalue, set, setkvp, getvalue, getname , gettoken,type

Nodes and Leaves

createnode, copynode, getnodes, getleaves

Structures

definestruct,insertstruct,createstruct,dstruct,dd

Variables and Arrays

var, const, array, resize, array.push, array.pop, array.reverse, array.shift, array.remove, array.clear, array.unshift, array.insert, array.slice, array.sum, array.copy, array.min, array.max, array.average, array.first, array.last, array.concat

Storage

storage, sinit, sget, sset, swrite, sinfo, sresize,sdimsize

Backup and Recovery

savedata, readdata

Control Flow

if, else, switch, case, default, loop, do, for, break, continue, setloopend.

Input Output

write, writeline (write#), writearray, readline

Functions

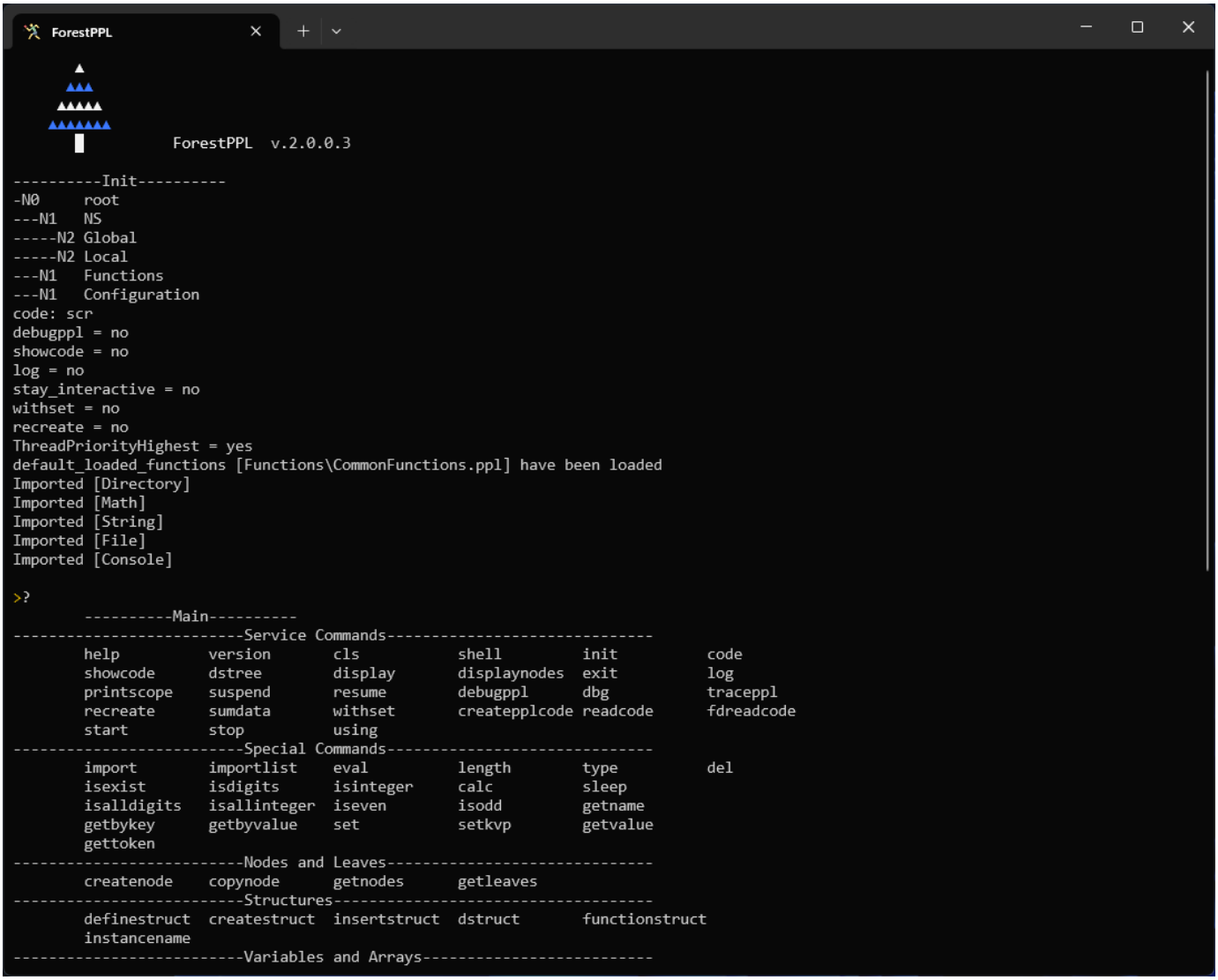
function, funclist, funcname, argc, getargname, call, return, getresult

Delegates and callbacks

delegate, dlgtinstance, dlgtset, dlgtcall, callback

ForestPPL utility

Command-line REPL utility **ForestPPL.exe** is a PPL interpreter which syntax and keywords are given in this tutorial. This utility is written in C++ without any third party packages.



These are following subdirectories and files used to work with ForestPPL.exe
Subdirectories:
\Data
\Examples
\Functions
\CodeGen
Files:
Configuration.data
ForestPPL.exe, DLL's.

Set Screen Buffer Size and Window Size in Properties\Layout.

There are 2 operating modes in accordance with ForestPPL.exe arguments:
1. **NonInteractive mode**
Execute program in file with extension scr or ppl.

ForestPPL.exe file [arg1 arg2 …]
file := file.ppl|file.scr

If arguments are present, they override the variables 1, 2 and so on in the body of the called file. Number between two symbols $ is the serial number of argument.
An error occurs if arguments quantity less than max variable number.
Value of argument is literal, not command.
When value of **stay_interactive** in file **Configuration.data = "no"** ForestPPL.exe finishes after program execution**,** when value of **stay_interactive = "yes"** ForestPPL.exe does not finish and continues in interactive mode.
Example:
File example.scr

```
var $1$ = $2$;
>ForestPPL.exe example.scr x 2;
```

2.**Interactive mode**

ForestPPL.exe

Command input from standard input stream.
To get list of commands and their short explanation perform **help** (or ?).
Examples:

```
>? display;
        display | d [root|NS|Functions|Local|node name]
>? d;
        display | d [root|NS|Functions|Local|node name]
```

Prompt **>** appears on Screen before each command.
Examples:

```
>d;
-N1     NS
---N2   Global
>var x; x=1;
>var x; set x=1;
>var x;
>x=1;
```

In addition to commands required to work with scr/ppl programs, ForestPPL.exe allows you to execute all Windows commands and save the results. Command **shell** uses for that.
Examples:

```
>code scr;
>var x;
>x = shell(cd);       // output is saved in var x
>write(x);
```

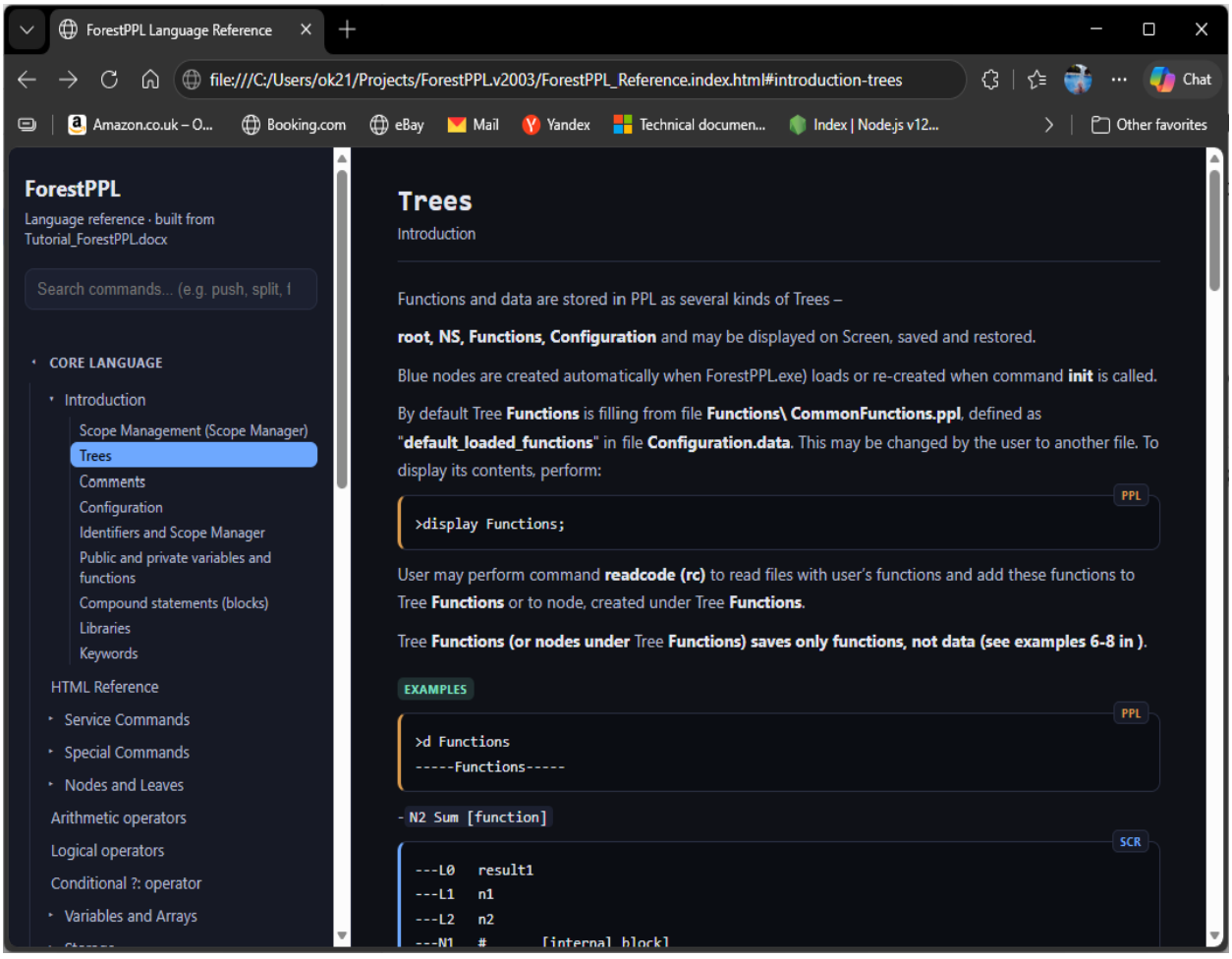
The following often used commands and operators with one parameter may be used with or without parentheses around arguments:
help (?), import, readcode (rc),showcode, createnode, isexist, display (d), displaynodes(dn),del, code,debugppl (dbg).

HTML Reference

The interactive HTML reference guide is the primary tool for quickly finding information on ForestPPL syntax and built-in functions. The file works locally and opens in any modern web browser, providing instant access to documentation without an internet connection.

Structure of the Reference Guide

- **Syntax and Arguments**: Exact call specifications for every engine command, ranging from basic variables to multidimensional Storage structures.
- **Brief Description**: A concise explanation of what a specific function does and the results it returns.
- **Practical Examples**: Ready-to-use code snippets demonstrating the correct application of a command in real scripts, allowing you to instantly copy and paste them into your project.
- **Categorization**: Commands are logically grouped into categories (e.g., mathematics, arrays, input/output, flow control) to streamline navigation.



Service Commands

help

Displays keywords list for Library by <name> or
format of command from Main library.
Library must be loaded before (see <import>)
by default name = Main, this library is loaded automatically
Format: **help | ? keyword**

Examples:

```
>? Code;
      Sets mode for Console input or displays on Screen
     code ppl | scr
>? "display"    // only display in quotes
      display | d [root|NS| Functions|Local|node name]
      display [NS.namespace.name]
```

Any other library has function help for display its contents.
<name of library>.help [(keyword)]

```
>Matrix.help (Rotate)
```

version

Display current version
Format: **version**

cls

Clears the Screen
Format: **cls**

shell

Executes Windows Console Commands, several commands are hash symbol separated.
Results of shell may be saved or displayed by commands **write, write#** or by
debugppl yes
Format: **shell (command with parameters[#command with parameters])**

Examples:

```
>write(shell (cd:\));
>dbg yes;
>shell (dir /b tests#cd);
```

init

Deletes all data and functions and creates new root, use this command for **console input only**.
Format: **init**

code

Sets mode for Console input or displays it on Screen.
Mode scr is more convenient for writing code with blocks and for using infix expressions. But in other cases there is no difference.
Format: **code [ppl|scr]**
default - ppl
It is possible to set **code** in file **Configuration.data**.

showcode

Shows or hides on Screen ppl_code when command readcode is executed or displays showcode value on Screen
Format: **showcode [yes|no]**
Default: no

Examples:

```
>showcode no;
```

readcode(rc)

Reads file with code in format scr or ppl.
At the end of the execution readcode the previous code (scr or ppl) will be set.
Format: **readcode | rc <file.scr|ppl> [arg1 arg2 …]**
If arguments are present, they override the variables 1, 2 and so on in the body of the called file.
Number between two symbols $ is the serial number of argument.
An error occurs if arguments quantity less than max variable number.
Arguments are literals, not commands.
If **delete_all_in_readcode = yes** in **Configuration.data** command
delete all is added automatically as first command when first command **readcode** is called.
The file being called may also include readcode commands. Files called by command readcode may be of different formats - .scr or .ppl.
If caller script and internal scripts are in the same path you may omit path for internals.
You may specify relatively path from ForestPPL.exe or full path, including drive:

```
>rc path\file.scr|ppl
```

```
      or
     >rc drive:\path\ file.scr|ppl
```

Examples

```
1.
     >Directory.SetCurrentDirectory(examples\ppl);
     >rc loop.ppl;  // or rc examples\ppl\loop.ppl
     loop (i)(0)(3)(1)
     (
        do
        (
           (write("PPL"))
           (write("ppl"))
        )
      );
2. File example.scr
     var $1$ = $2$;
     //pass arguments via rc
     >rc example.scr x 2;

3. reading script with command readcode inside:
File main.scr:
     write("main script");
     rc examples\rc\first.scr;
     rc examples\rc\second.scr;
     write("return from main script");
File first.scr:
     write("first script");
     rc examples\rc\third.scr;
     write("return from first script");
File second.scr:
     write("second script");
File third.scr
     write("third script");

     >rc examples\rc\main.scr;
     main script
     first script
     third script
     return from first script
     second script
     return from main script
```

createpplcode (cpc)

Creates file in format ppl from file (or files) in format scr.
Format **ppl**: **createpplcode|cpc (file.scr) [(file.ppl) [(all)]]**
Format **scr**: **createpplcode|cpc [file.scr file.ppl [all]]**

arg **all –** include all files, loaded by command **readcode.**

Examples

```
> code ppl;
>createpplcode (ttt.scr) (ttt.ppl);
>cpc (ttt.scr) (ttt.ppl);

>code scr;
>cpc ttt.scr;
```

display (d)

Displays nodes(N) and leaves(L) in Tree.
Format:
display | d [root | Global | Functions | Configuration| Local | path_node.name]
default: **Global**

Increase Screen Buffer Size.Height in ForestPPL.exe Properies\Layout in case large number of lines.

Examples:

```
 >array(y[2])(0);
  >d;
-N  NS
---N  Global
-----N  y
-------L0              [0]
-------L1              [0]
>d Global.y;
>d y;
>d Functions.Sum;
```

displaynodes (dn)

Displays nodes(N) only.
Format:
displaynodes | dn [root | Global | Functions | Configuration| Local | path_node.name]

Increase Screen Buffer Size.Height in ForestPPL.exe Properies\Layout in case large number of lines.

Example:

```
>dn Functions
-----Functions-----
-N3     Sum     [function]
---N1   #       [internal_block]
-----N2 set
-------N3       +
-N3     Sub     [function]
---N1   #       [internal_block]
-----N2 set
-------N3       -
……
```

dstree

Displays syntax tree per each statement in script and corresponding statement in format ppl.
dstree may be entered from script only, not from console in interactive mode.
dstree is for one-time running before presentation.

Format: **dstree() | dstree**

Examples:
File **examples\if.scr**

```
dstree;
for(i,0,2)
{
  if(i == 0)
  {
     write#("i == 0");
     continue;
  }
  else
    write#("i != 0 = {0}",i);
}
```

```
>rc examples\if.scr;
--N0    root
----N1  loop
------L0        i
------L1        0
------L2        2
------L3        1
------N2        do
--------N3      #               [internal_block]
----------N4    if
------------N5  ==
--------------L0        i
--------------L1        0
------------N5  #               [internal_block]
--------------N6        write
----------------L0      "i == 0"
--------------N0        continue
----------N4    else
------------N5  #               [internal_block]
--------------N6        write
----------------L0      "i != 0 = {0}"
----------------L1      i
code: scr
i == 0
i != 0 = 1
```

printscope

The printscope command is an advanced debugging tool designed to inspect how the interpreter manages memory and execution contexts at a system level.
What it does: When executed, this command queries the scopeManager directly and outputs detailed, low-level information about the active scopes. It reveals the engine's internal tracking of contexts and variable lifetimes.

How is it different from the d (display) command? It is important to understand the distinction between these two debugging tools:
d (display): This is a user-facing command. It visually renders the hierarchical syntax tree (NS -> Global / Local) and shows the declared variables alongside their current values.
printscope: This is an engine-facing command. It focuses on the mechanics of the scopeManager, revealing the underlying stack architecture, particularly how execution contexts are pushed and popped during function calls.
When to use it: The printscope command is highly valuable when debugging complex UserFunctions. It allows you to verify that local contexts are properly isolated, arguments are correctly passed, and that the scopeManager is accurately cleaning up memory upon returning from nested blocks of code.

Format: **printscope**

Examples:

```
>var x
>createnode N1
>var N1.x
>array N1.arr[5]
>printscope

=========== State of SCOPE MANAGER ===========
Depth [0]
 (Global)
 <-- TOP (Current of Context)

    * empty
    * x
    * N1 = 'node Node'
    * N1.x
    * N1.arr = 'array N1.Array 5'
```

suspend and resume

Stops script to perform manually one or several commands,
Enter **resume** - exit from command suspend and continue script execution
Enter **exit** - stop script, not exit from interpreter. If you want to exit from interpreter – run command **exit out of suspend**.

Format: **suspend**

Examples:

```
>suspend
Enter:
>d
-N1     NS
---N1   Global
>resume  // continue script execution
```

debugppl (dbg)

Displays information about creation and deletion variables, results operations and duration or displays debugppl value on Screen.
Format: **debugppl | dbg [yes|no]**
It is possible to set **debugppl** in file **Configuration.data**.

Example:

```
>var(x)
>debugppl yes
>del x
leaf [x] is deleted
>duration = 0.0054401
```

traceppl

Displays all commands and function names on screen during the execution of commands. By default – traceppl no.
Format: **traceppl [yes|no]**

Example:

```
>traceppl yes;
```

recreate

Permits recreation vars,arrays, storage and nodes. By default – recreate no.
Format: **recreate [yes|no]**

Example:

```
>recreate yes;
>code scr;
code: scr
>var x;
>var x;
>recreate no;
>var x;
Error: [FuncCreateVariables] name [x] already exists
```

log

Writes commands and results to logfile in directory **Log** or displays log value on Screen.
Format: **log [yes|no]**
It is possible to set **log** in file **Configuration.data**.
Opened logfile will be closed by command init or exit.

checksyntax

Checks every command against the syntax table (**DefaultSyntaxTable.h**) before it is executed. It applies to commands typed in the console and to scripts run from the console. A command that matches no pattern of the table is rejected with a syntax error and is not executed. Use `checksyntax no` to run commands the table does not describe yet. See **Inside checksyntax** (end of the section Error detection) for how the table works.
Format: checksyntax **[yes|no]**
It is possible to set **checksyntax** in file **Configuration.data**.

exit

Exit from ForestPPL.exe **exit** or exit from script **exit()**.

sumdata

Defines argument type for summation. By default – **sumdata digit**.
Format:
sumdata [digit | string]

Example:

```
>code scr;
>sumdata digit;
>write#(1+2);
3
>sumdata string;
>write#(1+2);    // scr-format
12
>write(1+2);
Error: [funcWrite] name [1+2] write supports only prefix
       expression
>write(+(1)(2);  // right: ppl-format
12
```

```
>sumdata string;
>var x = 1;
>call Sum(x,2);
>write#("x={0}",x);
x=12
```

It is possible to sum more than 2 items:

```
>sumdata string;
>var x;
>x = "a"+"\"" +"c" + 1;
a"c1
>sumdata digit;
>var y;
>y = 1+2+3+4;
10
```

```
sumdata digit;
function f(a,b,c,q)
{
  q = a + b + c;
}
var res;
call f(1,2,3,res);
write#("res={0}",res);

sumdata string;
call f(1,2,3,res);
write#("res={0}",res);
sumdata digit;
res=6
res=123
```

withset

Adds or no operator 'set' in code
Format:
withset [yes | no]
by default **no**.
It is possible to set in file **Configuration.data**.

start, stop

For measuring duration of a script or its part.
Format:
start | start()
stop | stop (text)

Example:

```
start;
array x[10];
for(i,0,length(x))
{
   x[i] = i;
}
writearray x row;
stop("test duration:");
-----Array x-----
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
test duration: 0.7918 msec
```

using

Creates nickname for long name. Variable (var,const, array,node,storage) must be created before. It is possible to use long name or nickname for access to this variable.

Format **ppl**:

using (short_name) (long_name)

Format **scr**:

using short_name = long_name

Example:

```
>code scr
>var xxxxx
>using x = xxxxx
>x = 0
>d
-N2     NS
---N3   Global
-----L0 empty   (const)
-----L1 xxxxx   [0]
---N3   Local
>write#("x = {0}",x)
x = 0
>write#("xxxxx = {0}", xxxxx)
xxxxx = 0
```

Special Commands

import

Loads Library from current directory or from user directory.
Format:
import Library name | Directory\Library name

Examples:

```
 >import Math;
>import DLL\Erato;    // (see examples\scr\erato.scr)
```

importlist (il)

Displays list of loaded Libraries
Format: **importlist | il**

Examples:

```
>importlist;
Main
Math
```

eval

EVAL IS NOT EVIL

Performs the operations described by the parameter, which is a quoted string in ppl format or a variable whose value is a quoted string in ppl format.
Format: **eval ("ppl expression") | eval (variable_name)**

Examples:

```
>var x = "var(r); set(r)(/(180)(Math.PI())); write(r);";
>eval(x);
>d
-N1      NS
---N2    Global
-----L0        empty                          (const)
-----L1      x    [var(r); set(r)(/(180)(Math.PI())); write(r);]
-----L2      r    [57.29577951308232]

>write(eval("+(1)(2)"))
3

>var x = "array (y)(1)(2)(3);d;"
>eval(x)
-N2      NS
---N3    Global
-----L0        empty      (const)
-----L1        x [array (y)(1)(2)(3);d;]
```

```
-----N4 y                                   [Array 3]
-------L0       #       [1]
-------L1       #       [2]
-------L2       #       [3]
```

```
>code scr
>array y[3]
>y[0] = "write(\"Hello World\")"
>eval(y[0])
Hello World

>code ppl
code: ppl
>storage(stor)(2)(2)
>sset(stor)(0)(0)("var (x)(0);d)")
>eval(sget(stor)(0)(0))
-N2     NS
---N3   Global
-----L0       empty     (const)
-----N4 stor                                [Storage 2 2x2]
-------N5       0                                  [Array element]
---------N6     Row                                [Array 2]
-----------L0       #   [var (x[0];d)]
-----------L1       #
-------N5       1                                  [Array element]
---------N6     Row                                [Array 2]
-----------L0       #
-----------L1       #
-----L1       x [0]
---N3   Local
```

```
function f(x)
{
   write#("x = {0}",x);
}
var y = "f(\"Hello\");write(\" f is ended\")";
eval (y);
x = Hello
f is ended
```

```
>var x = "write(\"Hello\")"
>eval (x)
Hello
```

length

Returns length of value for var | const or length array | one-dimensional storage
Format: **length (var | const name | array name|storage name)**

Examples:

```
>array y[3];
>write(length(y));
3
>var (x["Hello!"]);
>write(length(x));
6
>var c = "Hello";
>write(length(c));
5
```

isexist

Determines whether var, array or storage with specified name exists or not in Global or Local, returns "**True**" or "**False**".
Format: **isexist(name)**
name:= [NS.][namespace.][node.]name

Example:

```
1.
>code scr;
>dbg yes
>var x;
>isexist (x);
True
2.
>createnode Functions.New;
>isexist (Functions.New);
True
```

isdigits

Checks is value of var or member of array or storage digital, returns "**True**" or "**False**".
Format**: isdigits(var name | member of array or storage | literal)**

Example:

```
>code scr;
>dbg yes;
>var x = 1.1;
>isdigits (x);
True
```

Isinteger

Checks is value of var or member of array or storage integer, returns, returns "**True**" or "**False**".
Format**: isinteger(var name | member of array or storage | literal)**

Example:

```
>code scr;
>dbg yes;
>var x = 1;
>isinteger (x);
True
```

Isalldigits

Checks is all members of array or storage digital, returns, returns "**True**" or "**False**".
Format**: isalldigits(member of array or storage)**

Example:

```
>code scr;
>dbg yes;
>array x[] = {1,2,3};
>isalldigits (x);
True
```

isallinteger

Checks is all members of array or storage integer, returns, returns "**True**" or "**False**".
Format**: isallinteger(member of array or storage)**

Example:

```
>array(x)(1)(2)(3);
>isallinteger (x);
True
```

iseven, isodd

Checks is integer value even or odd, returns "**True**" or "**False**".
Format:
iseven(**var name | member of array or storage | literal)**
isodd(**var name | member of array or storage | literal)**

del

Deletes any kinds of data from Global or Local Tree and from ScopeManager, also deletes nodes from Functions.
Format: **del (fullname) | del fullname**
To delete all Global contents**: del all,** so name "**all**" can not be used as any kind of variable names.
If "**delete_all_in_readcode**" **= yes** in **Configuration.data** command
delete all adds automatically as first command when command **readcode** is called (**not for readdata**). Otherwise all data will be saved in memory, if necessary add this command manually.
fullname:= node path.name
node path:= node path | node

Example:

```
>createnode Node1;
>var(Node1.x);
//the following line removes Node1 and Node1.x
>del Node1;
>createnode Functions.Geo
>del Functions.Geo
```

To re-run script, that creates array or use command **recreate yes**:

```
if(isexist(y) == True)
   del y;
array y[5];
```

If "**delete_all_in_readcode**" **= yes**
command "del all" added for external readcode only, not for internal, to save for using all variables, defined in external readcode.
Example:

```
File examples\x51.scr
var x = 0;
rc examples\x51.scr;
d;

File examples\x51.scr
array y;

>rc examples\x50.scr
code: scr
code: scr
-N2     NS
---N3   Global
-----L0       empty                         (const)
-----L1       x                             [0]
-----N4 y                                   [Array 0]
---N3   Local
```

calc

Calculates **infix notation math. expression** and writes result on screen, may be used for ppl and scr modes, **but in interactive mode only**, not in .ppl or .scr files.

Format: **calc math.expression**

Examples:

```
>code ppl;
>var (x)(1);
>calc x + 2*Math.PI();
7.283185307179586
>code scr;
>array arr[] = {1,2,3};
>var x = 1;
>calc 1 + arr[0];
2
>calc 1 + arr[1+1];
4
>calc Math.Sqrt(1+3) + 1;
3
```

sleep

Suspends the interpreter for the specified number of milliseconds. If the script simply sleeps, the time and execution will complete. If a stopRequested signal is received, the sleep will be immediately interrupted and the program will exit without delay.

Format: **sleep(msec)**

Example:

```
>sleep(100);
```

getbykey (getk)

Gets value from array by name.
Format**: getbykey | getk (name array)(name element)**

Example:

```
See example in readdata
>getbykey(Colors)(Black);
0
```

if key is absent return **nan**.

getbyvalue (getv)

Gets name from array by value.
Format: **getbyvalue | getv (name array)(value element)**

Example:

```
See example in readdata
>getbyvalue(Colors)(0);
Black
```

if value is absent return **nan**.

set

Sets value for variable , array or storage element
Format **ppl**:
set (var_name | array_name [index]) (value | array_name [index]) |
set (storage_name[index1,index2,…])
(value | (storage_name[index1,index2,…] | array_name [index])
index:=value| ppl expression
Format **scr**:
set var_name | array_name [index] = value | scr expression
set storage_name [index1,index2,…] =
value | array_name[index] | storage_name [index1,index2,…] | scr expression

index:=value| scr expression

Command **set** checks whether array index is out of bounds.
In **scr-mode name of command set may be omitted**.

Examples:

```
>code ppl:
>var (x);
>set(x)(+(1)(2));
>array(y[3]);
>set(y[+(1)(2)])(0);
```

```
>code scr:
>var x;
>x = 1;
>array y[3];
>y[x + 1] = 2 + 3;
>y[0] = y[1];
 >y[3]=0;
Error: [SetElementArray] wrong index 3 array y out of of bounds
```

```
>code scr:
>var x;
>x = d;      // error, command "d" is running
>set x = "d";    // or x = "d";
>eval(x);        // this statement runs command "d"
```

```
>code scr;
>storage s[2,2];
>s[0,0] = s[0,1];
```

To calculate indexes for access to array elements command set in scr-mode creates temporary variables and deletes them at the end:
file test.scr

```
>showcode  yes;
>array y[] = {1,2,3,4,5,6,7};
>y[1+2] = y[2+2] + 1;

array (y)(1)(2)(3)(4)(5)(6)(7);
var (#0[ + (1) (2) ]);
var (#1[ + (2) (2) ]);
set (y[#0])(+ (y[#1]) (1) );
del #0;
del #1;
```

I It is possible to use logical and comparison operands:

```
>code scr;
>var x;
>x = ((1==1) && (2==2)) && (3==3);
>write(x);
True
```

```
>code scr
array x[3] = 0;
x[1+1] = ((1==1) && (2==2)) && (3==3);
writearray x row;
```

The following ppl-code will be generated:

```
>array(x[3])(0);
>var (#0[ + (1) (1) ]);
>set (x[#0])( &&  ( &&  ( ==  (1) (1) ) ( ==  (2) (2) ) ( ==
                        (3) (3) )  ) );
```

```
>del #0;
>writearray (x ) ( row );
```

```
>var x;
>x = 1==1? t:f;
>write(x);
```

The following ppl-code will be generated:

```
var (x);
if (== (1) (1) )
(
  (set (x)(t))
);
(
  else
  (
    (set (x)(f))
  )
);
write(x);
```

To set data in structure see (sample).

setkvp

Sets key and value array element
Format **ppl**:
setkvp(array_name [index])(key)(value | ppl expression)
index:=value| ppl expression
Format **scr**:
setkvp(array_name [index]) = key, value | scr expression)
index:=value| scr expression

Command **setkvp** checks whether index is out of bounds. For setting key and value command **setkvp** checks whether key already exists in array.
To calculate indexes for access to array elements command setkvp in scr-mode creates temporary variables and deletes them at the end (**for non-interactive mode only**):

To get key and value it is possible by commands **getbykey** and **getbyvalue**.

```
>code ppl:
>array(y[3]);
>setkvp(y[0])(+(1)(2));
>setkvp(y[1])( "one")(1);
>setkvp(y[2])( "two")(2);

>code scr:
>var x = 1;
>array y[3];
>setkvp y[x + 1] = "five",2 + 3;
>setkvp y[0] = "null", 0;
```

In scr-mode name of command **setkvp** may be omitted:

```
>y[0] = "five",2 + 3;
>y[1] = "Two",2;
```

getvalue (get)

Returns value of single var|const or array element.
Error: when argument is literal or not existed variable.
Format: **getvalue | get (var_name) |**
getvalue | get(array_name[index]
index:= value|ppl expression

Examples:

```
>array y[3] = 999;
>write#("getvalue(y[0]) = {0}",get(y[0]));
getvalue (y[0]) = 999
>write#("getvalue(y[0]) = {0}",get(yy[0]))
Error: [FuncGetValue] wrong name [yy]
```

To get data from structure see (sample).

getname

Returns name of single var|const | array | array element as string.
Error: when argument is literal or not existed variable.
Format: **getname (name)**

Examples:

1.

```
>code ppl;
>var (x)("ppl");
>write("{0} = {1}")(getname(x))(getvalue(x));
x = ppl
```

2.

```
function f(array: arr)
{
  write#("argname={0}  name={1}",getargname(arr), getname(arr));
}
array y;
f(y);
argname=y  name=arr

function f2(storage: stor)
{
  write#("arg_name={0} param_name={1}",
       getargname(stor),getname(stor));
}
storage s[2];
f2(s);
arg_name=s  param_name=stor
```

gettoken

Returns token in accordance with its number in string, string contains tokens, separated by "**separator**". Parts of string surrounded by quotes are passed.
As well gettoken may return number of tokens.
If number >= max number of tokens cmd returns "**Exception**".
Format: **gettoken (string)(separator)(number)**
return: **item_value**
or
gettoken (string)(separator)
return: **number of tokens**

Example:

```
1.
>dbg yes;
>gettoken("Hello,World")(",")(0);
result = Hello
>gettoken("Hello,World")(",")(1);
result = World
>gettoken("Hello,World")(",")(2);
result = Exception
```

```
2.
>code scr;
>var x = "\"Hello,World\",  PPL";
>dbg yes
>gettoken(x)(",")(0);
result = Hello,World
>gettoken(x)(",")(1);
result = PPL
3.
>code ppl;
>gettoken("\"Hello,World\",PPL")(",")(1);
result = PPL
4.
>gettoken("Hello,World")(",");
result = 2
```

If it is needed to use math.expression or result of operation call function **GetToken** from **CommonFunctions**:

```
4.
function GetToken(text,separator,index,res)
{
    res = gettoken(text)(separator)(index);
}

var name = "Nissan.Juke";
var count;
count = gettoken(name)(".");
write#("count={0}",count);
var token;

GetToken(name)(".")( -(count) (2) ) (token);  // ppl-format
  write#("token={0}",token);
call GetToken(name,".", count - 2 , token);   // scr-format
  write#("token={0}",token);
call GetToken(name,".",
     gettoken(name)(".") - 20 , token);       // scr-format
  write#("token={0}",token);
```

```
count=2;
token=Nissan
token=Nissan
token=Exception
```

type

Returns type of object (var, const, array, storage or struct).
Format: **type (name) | type name**

```
Example:
>var x;
>write(type(x));
var
```

Nodes and Leaves

createnode

Creates node in path, default path is "**Global**", it is possible to create nodes in Global, Local and Functions Trees. It will be error if name already exists (see recreate).

Format: **createnode(path.name) | createnode path.name**

Examples:

```
> createnode (Node)
> createnode Node.SubNode
>d
-N1     NS
---N1   Global
-----N2 Node
-------N3       SubNode

>createnode Functions.Geo
```

copynode

Copies one or more times node from path with new name, by default path is "Global"
Format:
copynode (src node)(dst node)[number of copies]
default number of copies: 1

Examples:

```
>code scr;
>createnode Person;
>var Person.Name;
>var Person.Family;
>var Person.Gender;
>array Person.cars[3];
>createnode Team;
>copynode(Person)(Team);
>Team.Name = Oscar;
>Team.Family = Ko;
>Team.Gender = m;
>setkvp Team.cars[0] = Juke,Nissan;
>setkvp Team.cars[1] = Qashqai,Nissan;
>d
-N1     NS
---N2   Global
-----N3 Person  [Node]
```

```
-------L0       Name
-------L1       Family
-------L3       Gender
-------N4       cars    [Array 3]
---------L0     #
---------L1     #
---------L2     #
-----N3 Team    [Person]
-------L0       Name    [Oscar]
-------L1       Family  [Ko]
-------L3       Gender  [m]
-------N1       cars    [Array 3]
---------L0     Juke    [Nissan]
---------L1     Qashqai [Nissan]
---------L2     #
---N2   Local
```

```
>createnode N
>var N.x
>var N.y
>createnode N2
>copynode (N)(N2)(2)
>d
-N2     NS
---N3   Global
-----L0       empty                     (const)
-----N4 N               [Node]
-------L0       x
-------L1       y
-----N4 N2              [N]
-------N5       0
---------L0       x
---------L1       y
-------N5       1
---------L0       x
---------L1       y
---N3   Local
```

getnodes

Creates (or recreates if exists) ppl_array with fullnames of nodes till defined nesting. Processing results of commands getnodes and getleaves allows you to find required information in hierarchical data structure.

Format:
getnodes (top node)[(nesting)](ppl_array)
Number of required nesting it is possible to get by command displaynode.
If (nesting) do not set node names under top_node will be saved in ppl_array.
For example there is file Data\Mng2.data

```
(Staff
  (Marketing
    (Managers
      (Personal Data1 [base]
        (Name [Benjamin])
        (Salary [6000])
        (Hobby
          (sport [tennis])
          (music [jazz])
        )
      )
   )
   (Clerks
     (Personal Data2 [base]
       (Name [Oliver])
       (Salary [4000])
      ……..
```

Read it:

```
>readdata (data\Mng2.data);
>d
-N2      NS
---N3    Global
-----N4 Staff
-------N5        Marketing
---------N6      Managers
-----------N7    Personal Data1   [base]
-------------L0 Name     [Benjamin]
-------------L1 Salary   [6000]
-------------N8 Hobby
---------------L0        sport    [tennis]
---------------L1        music    [jazz]
---------N6      Clerks
-----------N7    Personal Data2   [base]
-------------L0 Name     [Oliver]
-------------L1 Salary   [4000]
```

or

```
>dn Staff
-----Variables  and arrays-----
-N4     Staff
---N5   Marketing
-----N6 Managers
-------N7       Personal Data1   [base]
---------N8     Hobby
-----N6 Clerks
-------N7       Personal Data2   [base]
-------N7       Personal Data3   [base]
---------N8     Hobby
-------N7       Personal Data4   [base]
-------N7       Personal Data5   [base]
---------N8     Hobby
```

Get fullnames of nodes till nesting 7 and save in ppl_array "persons":

```
>getnodes(Staff)(7)("persons");
>d persons
-----Variables  and arrays-----
-N4     persons [Array 21]
---L0   #       [Staff.Marketing.Managers.Personal Data1]
---L1   #       [Staff.Marketing.Clerks.Personal Data2]
---L2   #       [Staff.Marketing.Clerks.Personal Data3]
---L3   #       [Staff.Marketing.Clerks.Personal Data4]
---L4   #       [Staff.Marketing.Clerks.Personal Data5]
---L5   #       [Staff.Finance.Managers.Personal Data6]
---L6   #       [Staff.Finance.Managers.Personal Data7]
---L7   #       [Staff.Finance.Managers.Personal Data8]
---L8   #       [Staff.Finance.Clerks.Personal Data9]
---L9   #       [Staff.Finance.Clerks.Personal Data10]
---L10  #       [Staff.Finance.Clerks.Personal Data11]
---L11  #       [Staff.Operations management.Managers.Personal
                                              Data12]
---L12  #       [Staff.Operations management.Managers.Personal
                                              Data13]
---L13  #       [Staff.Operations management.Clerks.Personal Data14]
---L14  #       [Staff.Operations management.Clerks.Personal Data15]
---L15  #       [Staff.Operations management.Clerks.Personal Data16]
---L16  #       [Staff.Operations management.Clerks.Personal Data17]
---L17  #       [Staff.Operations management.Clerks.Personal Data18]
---L18  #       [Staff.Operations management.Clerks.Personal Data19]
---L19  #       [Staff.Human Resource.Managers.Personal Data20]
---L20  #       [Staff.Human Resource.Clerks.Personal Data21]
```

getleaves

Creates ppl_array whose elements have names and values of node.
Format:
getleaves(node_name | value)(ppl_array)
value = var_value | array_item_value

Example:

See previous example with command getnodes

```
>array property;
>getleaves(Staff.Marketing.Managers.Personal Data1)(property)
>d property
-----Variables  and arrays-----
-N4     property        [Array 2]
---L0   "Name"  [Benjamin]
---L1   "Salary"        [6000]
```

Full code of file Data\mng2.scr to find persons with salary = 2000:

```
>var tmp;
>var salary;
>var name;
>readdata  data\Mng2.data;

getnodes(Staff)(7)("persons");   // create ppl_array persons
for(i,0,length(persons))
{
  getleaves(persons[i])( property);
  name = getbykey(property)("Name");
  salary = getbykey(property)("Salary");
  //write#("name={0},salary={1}",name,salary);
  if (salary == 2000)
  {
    write#("[{0}]   {1}  {2}",persons[i], name, salary);
  }
}
 >rc data\mng2.scr
[Staff.Marketing.Clerks.Personal Data4]   Charlotte  2000
[Staff.Marketing.Clerks.Personal Data5]   Olivia  2000
[Staff.Marketing.Clerks.Personal Data5]   Olivia  2000
[Staff.Operations management.Clerks.Personal Data18]   Felix
                                                  2000
[Staff.Operations management.Clerks.Personal Data19]   James
                                                  2000
[Staff.Human Resource.Clerks.Personal Data21]   Sophia  2000
```

Structures

Preprocessor generates additional ppl-code for commands **definestruct, insertstruct and createstruct** so they used for non-interactive mode only (in scripts).

definestruct

Creates named block statement, that contains one or several objects - vars, arrays, storages and insertstructs. Struct named block cannot be empty, functions may be located inside **definestruct** or behind.
Variables and functions with first character "_" are private. (see example Examples\struct\TestStruct14.scr).

Format **scr**:

```
definestruct name
{
  array …;
  var …;
  storage…;
  insertstruct …;
  …
}
```

Example:

```
definestruct Room
{
  array computers[3];
  array tables[3];
}
```

Generated code in ppl-mode

```
definestruct
(Room
  (
  (array (computers[3]) )
  (array (tables[3]) )
  )
);
```

insertstruct

Inserts defined structure, previously defined by definestruct. It is possible to insert several instances.
Format **scr**:
insertstruct object_name [[size]] as struct_name

Example:

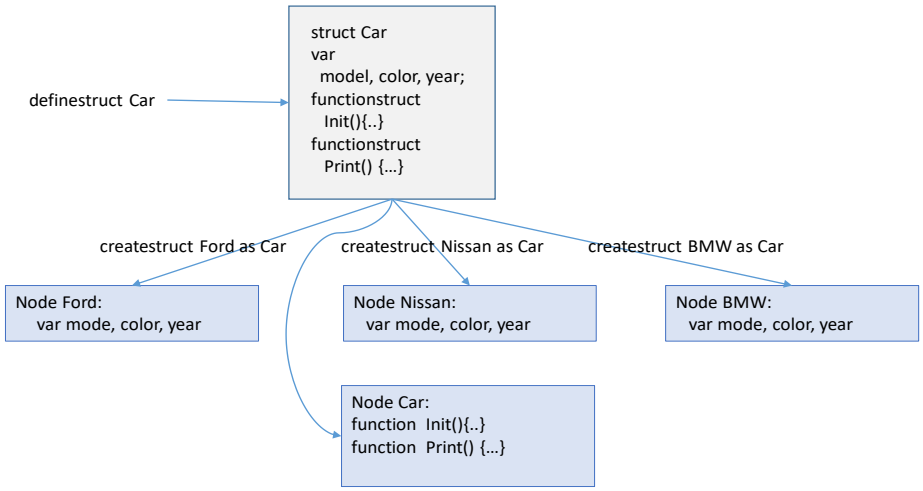
```
insertstruct rooms[2] as Room;
```

Generated ppl_code:

```
(createnode (rooms) )
(copynode (rooms)(Room)(2))
```

createstruct

Creates one or several instances in accordance with structure type, previously defined by definestruct.
Command createstruct creates per each instance of struct node with name of instance, that contains all objects except of functions.
Additionally for all instances it is created one time node with name of struct, that contains struct functions only, not struct objects.



Format **scr**: **createstruct name[[size]] as struct_name**

Example: create struct array

```
createstruct Group[2] as Person;
```

Generated ppl_code:

```
createstruct (Group)(Person)(2);
```

See examples\struct\teststr4.scr

Tree of above-created object:

```
>displaynodes;
-N4     Group           [Person]
---N5   0               [Node]
-----N6 rooms           [Rooms]
-------N7       0
---------N8     computers       [Array 3]
---------N8     tables          [Array 3]
-------N7       1
---------N8     computers       [Array 3]
---------N8     tables          [Array 3]
---N5   1               [Node]
-----N6 rooms           [Rooms]
-------N7       0
---------N8     computers       [Array 3]
---------N8     tables          [Array 3]
-------N7       1
---------N8     computers       [Array 3]
---------N8     tables          [Array 3]
```

To set data for structure members it is possible by this manner (see examples above):
(see file Examples\Struct\TestStruct4.scr)

```
definestruct Room
{
  array computers[3];
  array tables[3];
}
definestruct Person
{
  var Name;
  var Family;
  insertstruct rooms[2] as Room;
}
createstruct person[2] as Person;
//person.0.Name  = "John";
// index as var
var y = 0;
person.y.Name  = "John";
```

```
person.y.Family = "Deere";
var x = person.y.Name;          // right;
var x1 = person.0.Name;         // right
var x2 = get(person.y.Name);    // right
write#("x = {0}",x);
write#("x1 = {0}",x1);
write#("x2 = {0}",x2);
person.y.rooms.y.computers[0] = "HPE Cray EX";
person.y.rooms.y.computers[0] = "Asus";
person.y.rooms.y.computers[1] = "Sony";
person.y.rooms.y.computers[2] = "HP";
person.y.rooms.y.tables[0] = "IKEA 70x140";
person.y.rooms.y.tables[1] = "IKEA 70x160";
person.y.rooms.y.tables[2] = "IKEA 70x200";
d Person;
d Room;
dd person;
```

instancename

Returns the current structure instance name for using in structure functions.
Format:
instancename() | instancename

Example:

```
>write(instancename);
```

File examples\struct\teststruct10.scr

```
function Print()
{
  write("Print: Global Function");
}

definestruct Script
{
   var x;
   var y;
}
functionstruct Print(Script)
{
  write#("===={0}===={1}===",funcname,instancename);
  write#("\tthis.x = {0} this.y = {1}",this.x,this.y);
}

functionstruct Foo(Script)
{
   write#("===={0}===={1}===",funcname,instancename);
   this.Print();
}
createstruct script as Script;
script.x = 100;
script.y = "PPL";
script.Foo();
script.Print();

function Script.Print()
{
  write#("===={0}===={1}===",funcname,instancename);
  write#("Updated function Print   this.x = {0} this.y = {1}",
     this.x,this.y);

}

script.Print();
Print();
```

```
>rc examples\struct\teststruct10.scr
===Script.Foo===script===
===Script.Print===script===
        this.x = 100 this.y = PPL
===Script.Print===script===
        this.x = 100 this.y = PPL
Warning: [FuncCreateFunction] function [Script.Print] is updated
===Script.Print===script===
Updated function Print   this.x = 100 this.y = PPL
Print: Global Function
```

functionstruct

Adds function to structure.
Format is the same as format of function plus structure name.

Format **ppl:**
functionstruct are located behind **definestruct** in this format:

```
functionstruct
(
   function_name
   (struct name)
  parameter_list
  ( function body )
);
function_name | struct_name::=  identifier
parameter_list::= parameter [parameter_list]
parameter::=  (identifier) | (identifier[default value]) | empty
function body::= (statement1) [(statement2) (statementN)]
identifier::= [var] | [array]  | [storage]  |
        [struct <struct_name>] | [struct array <struct_name>]:<param_name>
```

Format **scr:**
functionstruct may be located inside **definestruct** or behind in this format:
For using inside the **definestruct**:

```
functionstruct  function_name
  (parameter_list)
  {
    function body
  }
```

For using behind the **definestruct** add struct_name:

```
functionstruct    function_name
  (struct name,parameter_list)
  {
    function body
  }

function_name | struct_name::=  identifier
parameter_list::= parameter, [parameter_list]
parameter::=
          identifier | identifier[default value] | identifier = default value | empty
function body::= statement1; [ statement2;   statement; ]
identifier::= [var] | [array] | [storage]:<name>
By default parameter type is var.
```

See examples:
Examples\struct\TestStruct11.scr (functions of struct are located behind struct)
Examples\struct\TestStruct12.scr (functions of struct are located inside struct)

Any function, belonging to structure by command **funcstruct,** may be replaced behind by command **function**.
Access to struct functions and any struct objects is provided via "**this**".

See example in Examples\Struct\TestStruct10.scr:

```
function Print()
{
  write("Global Function");
}

definestruct Script
{
   var x;
   var y;
}
functionstruct Print(Script)
{
  write#("===={0}===={1}===",funcname,instancename);
  write#("this.x = {0} this.y = {1}",this.x,this.y);
}

functionstruct Foo(Script)
{
   write#("===={0}===={1}===",funcname,instancename);
   this.Print();
}

createstruct script as Script;
script.x = 100;
script.y = "PPL";
script.Foo();
script.Print();
Print();

function script.Print()
{
  write#("===={0}===={1}===",funcname,instancename);
  write#("Updated function Print   this.x = {0} this.y =
          {1}",this.x,this.y);
}
script.Print();
```

```
===Script.Foo===script===
===Script.Print===script===
   this.x = 100 this.y = PPL
===Script.Print===script===
   this.x = 100 this.y = PPL
Warning: [FuncCreateFunction] function [Script.Print] is updated
===Script.Print===script===
Updated function Print   this.x = 100 this.y = PPL
Print: Global Function
```

Example of using delegates for functions of structures:

```
definestruct Person
{
   var name;
   var family;
   var age;
   functionstruct Init(n,f,a)
   {
      this.name = n;
      this.family = f;
      this.age = a;
   }
}
function Print2()
{
    write("global function: Print2");
}
definestruct Employee
{
   var position;
   insertstruct person as Person;
   functionstruct Print()
   {
     write( "struct function: Employee.Print");
     write#("    person: name={0} family={1} age={2}",
        this.person.name,this.person.family,this.person.age);
     write#("    position={0}",this.position);
   }
}
createstruct employee as Employee;
employee.position = "Manager";
call employee.person.Init("Johnny","Walker",40);

delegate myDlgt ();
dlgtinstance instance myDlgt;
dlgtset instance Print2;
dlgtcall instance();

dlgtset instance employee.Print;
dlgtcall instance();
```

dstruct

Displays contents of structure types.
Format **ppl**:
dstruct [(struct_name) [(data)]];
Format **scr**:
dstruct [struct_name [data]];

dd

Displays contents of structure instance (as well as "display") without functions.
Format: **dd instance_name**

Example:
see examples\struct\testCar.scr.

```
definestruct Car
{
  var model;
  var color;
  var year;
}
functionstruct Print(Car)
{
   write#("==={0}===",InstanceName);
   write#("model = {0},  color = {1}, year =
       {2}",this.model,this.color,this.year);
}
functionstruct Init(Car,m,c,y)
{
  this.model = m;
  this.color = c;
  this.year  = y;
}
createstruct Ford as Car;
Ford.Init("Mustang")("Red")("1969");
createstruct Nissan as Car;
Nissan.Init("Qashqai")("White")("2023");
Ford.Print();
Nissan.Print();
===Ford===
model = Mustang,  color = Red, year = 1969
===Nissan===
model = Qashqai,  color = White, year = 2023

>dd
-N2     NS
---N3   Global
-----L0 empty    (const)
```

```
-----N4 Ford            [Car]
-------L0       model   ["Mustang"]
-------L1       color   ["Red"]
-------L2       year    ["1969"]
-----N4 Nissan          [Car]
-------L0       model   ["Qashqai"]
-------L1       color   ["White"]
-------L2       year    ["2023"]
```

Arithmetic operators

+, -, *, /, ^, %, ++, --

These are binary operators.
Do not confuse with functions names in file **CommonFunctions.ppl**:
Sum, Sub, Mul, Div, Pow

Examples in ppl prefix notation:

```
+(x)(y)
write(+(1)(2)(3))      // Result: 6
*(+(x)(y))(-(z)(3))
```

Examples in scr infix notation:

```
>code scr;
> var z = x  + y + 1;
> var z  = (x  + y) * (z - 3);
```

Logical operators

<, <=, >, >=, ==, !=, &&, ||, xor

xor only for ppl mode
These are binary operators.
Do not confuse with functions names in file **CommonFunctions.ppl**:
LT, LE, GT, GE, EQ, NE, AND, OR,XOR
Examples in ppl prefix notation:

```
==(x)(y)
&&(==(x)(y))(==(z)(3))
```

Examples in scr infix notation:

```
x == y
(x == y) && (z == 3)
```

Conditional ?: operator

Condition ? if_true: if_false;

```
set x = 3 < 4 ? 1 : 2;
or
x = 3 < 4 ? 1 : 2;
```

Variables and Arrays

var

Creates a single variable in Global or in Local function scope. It will be error if name already exists (see recreate).
Format **ppl**:
var (name)| (name)(initial value)
name:= [node path]name
node path:= node. | node
initial value:= value| ppl expression
ppl expression:=value | prefix notation expression

Format **scr**:
var name
var name1,name2,name3…
var name|name = initial value
var name1,name2,name3… = init_value

name:= node path.name
node path:= node. | node
initial value:= value| scr expression
scr expression:= value | infix notation expression

Examples:

```
➢ file var.scr
code scr;
recreate yes;
showcode yes;
array y[] = {1,2,3};
var X;
var x = 0;
var t = "Hello!";
var n = 2;
var p = y[0];
var q = n;
var w = y[n];
var a,s,z=0;
var pi = Math.PI();
var OneRadian = 180/Math.PI();
d;

>rc var.scr;
>code scr;
code: scr
>recreate yes;
>showcode yes;
```

```
>array (y)(1)(2)(3);
>var (X);
>var (x) (0);
>var (t) ("Hello!");
>var (n) (2);
>var (p) (y[0]);
>var (q) (n);
>var (w) (y[n]);
>var (a) (0);
>var (s) (0);
>var (z) (0);
>var (pi) (Math.PI());
>var (OneRadian) (/ (180) (Math.PI(0)) );
>d;
-N1     NS
---N2   Global
-----L0       empty                        (const)
-----N3 y                                  [Array 3]
-------L0       #                                [1]
-------L1       #                                [2]
-------L2       #                                [3]
-----L1       X
-----L2       x                            [0]
-----L3       t                            [Hello!]
-----L4       n                            [2]
-----L5       p                            [1]
-----L6       q                            [2]
-----L7       w                            [3]
-----L8       a                            [0]
-----L9       s                            [0]
-----L10      z                            [0]
-----L11      pi                           [3.14159265358979323846]
-----L12      OneRadian                    [57.29577951308232]
---N2   Local
```

Do not use Math.Function in Math.Function:

```
>var x = Math.Sin(30/(180/Math.PI())) // error

var OneRadian = 180/Math.PI();        // correct
var x = Math.Sin(30/OneRadian);
```

const

Creates a single constant variable in Global or in Local function scope. It will be error if name already exists (see recreate).

Format **ppl**:
const (name)(initial value)
name:= [node.]name

initial value:= value| ppl expression
ppl expression:=value | prefix notation expression

Example:

```
>code ppl;
>const (x)(0);
>const (y[+(2)(3)]);
```

Format **scr**:
const name = initial value
const name1,name2,name3… = initial value

name:= node path.name
node path:= node. | node
initial value:= value| scr expression
scr expression:= value | infix notation expression

Examples:

```
>createnode N1;
>code scr;
>const N1.Greetings = "Hello"
>write(N1.Greetings)
Hello
>const radian = 180 / Math.PI();
>write(radian);
57.2958
```

setconst

Converts var with assigned value to const.
Format **ppl:**
setconst (var_name)
Format **scr:**
setconst var_name

Example:

```
>code scr;
>var x = 0;
>setconst x;
```

array

Creates single-dimensional array in Global or in Local function scope as a composite parent node and registers it to Scope Manager. It will be error if array with same name already exists (see recreate).

Format **ppl**:

array(name [length]) [(initial value)]
array(name)(1st item)(2nd item)…
name:= node path.name
node path:= node. | node
length:= value| ppl expression
initial value:= value| ppl expression
item:= value| ppl expression
ppl expression:=value | prefix notation expression

Examples:

```
>code ppl;
>var(x[10]);
>array(y[3]);
>array(y[/(x)(2)])(0);        // init by 0 all 5 elements
>array(y[x])(*(x)(3));        // init by 30 all 10 elements
>array(y)(1)(x)(+(1)(2));     // init 3 elements array = 1,10,3
```

Format **scr**:

array name[length];
array name [length] = initial value;
array name [] = {1st item, 2nd item,…};
name:= node path.name
node path:= node. | node
length:= value| scr expression
initial value:= value| scr expression
item:= value| scr expression
scr expression:=value | infix notation expression

Examples:

```
>code scr;
>array y[3];
>d
-N1     NS
---N2   Global
-----L0       empty                          (const)
-----N3 y                                    [Array 3]
-------L0       #
-------L1       #
-------L2       #
---N2   Local
```

```
>array y[1+2] = 0;          // init by 0 all 3 elements
>array y[] = {1,2,1+2};     // init 3 elements array = 1,2,3
>var x = 1;
>array y[x+2];
```

To access an array element you need to calculate index as a separate variable:

```
>code scr;
>var x = 1;
>array y[] = {1,2,3,4,5};
var index = x + 1;
>write(y[index]);        // or write(y[2])
```

```
Index may be used as expression in command set. Name of command
set may be omitted.
>set y[x + 1] = 100; // or y[x + 1] = 100;
```

Creation array with **length = 0**:

```
array arr;  or
array arr[]; or
array arr[0];
```

In the following sample array with **length = 0** is created preliminary and resized in function **Directory.GetDirectories** in accordance with real length:

```
>createnode Dir;
>array Dir.dir;
>call Directory.GetDirectories(Dir.dir, "c:\\users\\");
```

The following commands in format **scr** are used without parentheses in simple statement without equal sign.
They are used with parentheses in statements as arguments of other commands or in statements with equal sign on the right side:

array.push,
array.shift,
array.unshift,
array.remove,
array.insert,
array.slice,
array.concat,
array.clear.

Example:

```
 >code scr;
>array y[] = {1,2,3,4,5};
>array.push y 10;                  // simple statement
>write#( array.push (y)(20));     // argument of command write#
7
>writearray y row
1,2,3,4,5,10,20
```

Writing arguments in parentheses implies ppl-mode and prefix notation.

Example:

```
>code scr;
>array y[] = {1,2,3,4,5};
>array.remove (y)(1+1);                  // wrong
>array.remove (y)(+(1)(1));              // right
```

See array commands examples in **examples\ArrayFunc\Samples.scr**.

Additional array service see in file **CommonFunctions.ppl**:

CsvToArray, ArrayToCsv, ArrayIsExist, ArrayIndexOf, ArrayLastIndexOf, ArrayForEach.

resize

Changes length of array, all elements are saved in changed array.

Format **ppl**:

resize(array_name)(new length)
resize(array_name)(new length) (init_value)

Format **scr**:

resize array_name[new length]
resize array_name[new length] = init_value
resize(array_name)(new length) [(init_value)] (supported format ppl)

Example 1

```
>code scr;
>array y[5] = 0;
>resize y[10];
>d;
-N1     NS
---N2   Global
-----N3 y       [Array 10]
-------L0       #       [0]
-------L1       #       [0]
-------L2       #       [0]
-------L3       #       [0]
-------L4       #       [0]
-------L5       #
-------L6       #
-------L7       #
-------L8       #
-------L9       #
---N2   Local
>resize y[3];
>d;
-N1     NS
---N2   Global
-----N3 y       [Array 3]
-------L0       #       [0]
-------L1       #       [0]
-------L2       #       [0]
---N2   Local
```

If init_value is specified this value will be set in all elements of the new array. If init_value is not specified old values are saved in the new array.
Size of the new array may be equal zero, may be smaller or larger than the old one.

To resize storage use **sresize**.

array.push

Adds a new item to an array as last, returns a new size of array.
Format **ppl: array.push (array_name)(item_value) |**
or for key and value array (kvp)
array.push (array_name)(item_name)(item_value)
Format **scr:**
array.push array_name item_value |
array.push (array_name, item_value)|
array.push (array_name, item_name, item_value) |
array.push array_name, item_name, item_value |
array.push array_name index item_value |
array.push (array_name)(item_name)|
array.push (array_name)(item_name)(item_value)

It is possible to use variables or array items as item name or item value.

Example:

```
>code scr;
>array y;
>array.push (y,1);
>array.push (y,1 + 2);
>array.push (y,1 + array.min());
>writearray y row;
-----Array y-----
1, 3, 2
>var len = array.push(y)(1 + 2);     // error, it is PPL-forlamt
>var len = array.push(y)(+(1)(2));   // right
or
>array.push(y,1+2);
>var len = length(y);
```

array.pop

Returns the latest item value and removes item**.**
For kvp-array returns item name and item value, separated by comma and removes item.
If array is empty: returns string "**Empty**".
Format**: array.pop (array_name)**

Examples:
1.

```
>code scr
>array y[] = {1,2,3};
>var Result = "";
>Result = array.pop(y);    // Result = 3
>writearray y row;
-----Array y-----
1, 2
```

2.

```
>code scr
>array y[3];
>setkvp y[0] = one,1;
>setkvp y[1] = two,2;
>setkvp y[2] = three,3;
>var Result = "";
>Result = array.pop(y);    // return three,3
>write#("name={0}value={1}",
    gettoken(Result)(",")(0),gettoken(Result)(",")(1));
name=three value=3
>writearray y row;
-----Array y-----
1, 2
```

array.reverse

Reverses items order in array, returns a size of array;
Format **ppl: array.reverse (array_name)**
Format **scr: array.reverse array_name | array.reverse (array_name)**

array.shift

Removes the first item of the array, returns a new size of array.
Format **ppl: array.shift (array_name)**
Format **scr: array.shift array_name | array.shift (array_name)**

Example:

```
>code scr;
>array y[] = {1,2,3,4,5};
>array.shift y;
>writearray y row
-----Array y-----
2,3,4,5
> write#(array.shift(y))
2
>writearray  y row
-----Array y-----
3, 4
```

array.remove

Removes item by index, returns a new size of array.
Format **ppl:**
array.remove (array_name)(index)
Format **scr:**
array.remove array_name index |
array.remove array_name, index |
array.remove (array_name, index) |
array.remove (array_name) (index)

Example 1:

```
>array y[] = {1,2,3,4,5};
>write#("y.length = {0}",length(y))
y.length = 5
>write#("y.length = {0}",array.remove(y,1));
y.length = 4
>writearray y row
-----Array y-----
1, 3, 4, 5
```

It is possible to use variables or array items as array_name or index.
Example 2:

```
>array y[] = {1,2,3,4,5};
>var i = 0;
>var x = "y";
>array.remove(get(x),i);
>writearray y row;
-----Array y-----
2, 3, 4, 5
```

array.clear

Removes all items from array, returns 0.
Format **ppl: array.clear (array_name)**
Format **scr: array.clear array_name | array.clear (array_name)**

array.unshift

Adds a new item as first to an array, returns a new size of array.
Format **ppl:**

array.unshift (array_name)(item_value) |
array.unshift (array_name)(item_name)(item_value)

Format **scr:**

array.push array_name item_value |
array.push (array_name, item_value)|

array.push (array_name, item_name, item_value) |
array.push array_name, item_name, item_value |
array.push array_name index item_value |
array.push (array_name)(item_name)|
array.push (array_name)(item_name)(item_value)

It is possible to use variables or array items as index, item name or item value.

array.insert

Inserts item before item with index, returns a new size of array.
Format **ppl:**

array.insert (array_name)(index)(item_value) |
array.insert (array_name)(index)(item_name)(item_value)

Format **scr:**

array.insert (array_name,index, item_value) |
array.insert (array_name,index, item_name, item_value)

It is possible to use variables or array items as index, item name or item value:

```
>array y[] = {1,2,3,4,5}
>array r[] = {1,3}
>array.insert(y, r[1], r[0])
>writearray y row
-----Array y-----
1, 2, 3, 1, 4, 5
```

array.slice

Forms a slice [of specified length] out of the current array segment starting at the specified index and return new array length or "**Error**".

Format **ppl**:

array.slice (array_name)(index)|
array.slice(array_name)(index)(length)

Format **scr**:

array.slice array_name index|
array.slice (array_name, index)|
array.slice (array_name, index, length) |
array.slice array_name, index, length |
array.slice array_name index length |
array.slice (array_name)(index)|
array.slice(array_name)(index)(length)

It is possible to use variables or array items as index or length.

index starts from 0

Example:

```
>code scr;
>array y[] = {1,2,3,4,5};
>writearray y row;
-----Array y-----
1, 2, 3, 4, 5
>array.slice(y, 2);
>writearray y row;
-----Array y-----
3, 4, 5
>array.slice(y, 1, 2);
>writearray y row;
-----Array y-----
4, 5
```

array.sum

Returns the sum of all items in array. In case of error returns string "**Error**".
Format: **array.sum (array_name)**

Example:

```
>code scr;
>dbg yes;
>array y[] = {1,2,s,4,5,a};
Info: [CreateArrayFormat1] Global array [y] is created
>array.sum(y);
Error: [FuncArraySum] not digital data array y[2] = s
Error: [FuncArraySum] not digital data array y[5] = a
Error;
>recreate yes;
>array y[] = {1,2,3,4,5};
Info: [CreateArrayFormat1] Global array [y] is created
>array.sum(y);   // 15
```

array.copy

Clean contents of destination array (dst) and resize it in accordance with size of source array (src).Copies src array to dst array. In case of error returns string **"Error"**.
Format ppl : **array.copy (src array)(dst array)**
Format scr : **array.copy src_array dst_array |**
array.copy (src_array, dst_array) |
array.copy (src_array, dst_array) |
array.copy (src_array) (dst_array)

Example:

```
>code scr;
>array src[] = {1,2,3,4,5};
>array dst;   // dst array must be created before array.copy
>array.copy src dst;
           // or
           // array.copy (src, dst);
           // array.copy (src) (dst);
>writearray dst row;
-----Array dst-----
1, 2, 3, 4, 5
>writearray src row;
-----Array src-----
1, 2, 3, 4, 5
```

array.filtercopy

Copies src array to dst array except objects with value = 0. In case of error returns string **"Error"**.
Format ppl : **array.filtercopy (src array)(dst array)**
Format scr : **array. filtercopy src_array dst_array |**
array. filtercopy (src_array, dst_array) |
array. filtercopy src_array, dst_array |
array. filtercopy (src_array) (dst_array)

Example:

```
>code scr;
>array src[] = {1,0,3,0,5};
>array dst;   // dst array must be created before
array.filtercopy
>array.filtercopy src dst;
           // or
           // array.filtercopy (src, dst);
           // array.filtercopy (src) (dst);
>writearray dst row;
-----Array dst-----
1, 3, 5
```

```
>writearray src row;
-----Array src-----
1, 2, 3, 4, 5
```

array.min

Returns the minimum value in array. In case of error returns string **"Error"**.
Format: **array.min (array name)**
Example:

```
>code scr;
>dbg yes;
>array y[] = {10, 20, 5, 30};
>array.min(y);  // 5
```

array.max

Returns the maximum value in array. In case of error returns string **"Error"**.
Format: **array.max (array name)**

Example:

```
>code scr;
>dbg yes;
>array y[] = {10, 20, 5, 30};
>array.max(y);  // 30
```

array.average

Returns the mean value in array. In case of error returns string **"Error"**.
Format: **array. average (array name)**
Example:

```
>code scr;
>dbg yes;
>array y[] = {10, 20, 30, 40, 50};
>array.average(y);  // 30
```

array.first

Returns value of the first item in array. In case of error returns string **"Error"**.
Format: **array.first (array name)**
Example:

```
>code scr;
>dbg yes;
>array y[] = {100, 200, 300};
>array.first(y);   //  100
```

array.last

Returns value of the last item in array. In case of error returns string **"Error"**.
Format: **array.last (array name)**
Example:

```
>code scr;
>dbg yes;
>array y[] = {100, 200, 300};
>array.last(y);   // 300
```

array.concat

Concatenates several arrays to destination array, returns length of destination array or **"Error"** if one of arrays is absent.
Format **ppl**: **array.concat (array1)(array2)(...arrayN) (dst array)**
Format **scr**: **array.concat array1 array2 ...arrayN dst_array |**
array.concat (array1, array2, ...arrayN, dst_array) |
array.concat (array1)(array2)(...arrayN) (dst array)

Example:
1.

```
>code scr;
>array y[] = {1,2,3,4,5};
>array a[] = {10,20,30};
>array b[] = {100,200};
>array dst;
```

```
>array.concat y a b dst;   // simple statement
 //  or
 //>array.concat (y, a, b, dst);
 //>array.concat (y) (a) (b) (dst);

>writearray y row;
-----Array y-----
1, 2, 3, 4, 5
>writearray dst row;
-----Array dst-----
1, 2, 3, 4, 5, 10, 20, 30, 100, 200
```

2.

```
>code scr;
>array y[] = {1,2,3};
>array b[] = {4,5,6};
>array dst;
>var len = array.concat(y)(b)(dst);  // right side of statement
                                     // with equal sign
>writearray dst row;
-----Array dst-----
1, 2, 3, 4, 5, 6
>write#("length={0}",len);
length = 6
```

array.sort

Sort ppl_array in accordance with ascend or descend order.
Format **ppl**:

array.sort (array_name)(direction)

Format **scr**:

array.sort array_name, direction |
array.sort array_name direction |
array.sort (array_name, direction) |
array.sort (array_name)(direction)

direction = ascend | descend | "ascend" | "descend"

Example:

```
>code scr;
>array y[] = {1,30,2};
>array.sort (y,ascend);
>writearray y row;
-----Array y-----
1, 2, 30
```

See array samples in examples\arrayfunc.

Storage

Service of multi-dimensional arrays is realized by storage operators in mode ppl (parameters with prefix expressions in parentheses, but may be used also in mode scr (see Examples 3).

storage

Creates one-dimensional or multi-dimensional array with dimension from 1 to N in Global or in Local function scope and registers it to ScopeManager. It will be error if name already exists (see recreate).

Format **ppl**:
storage (name) (size dim1) |
storage (name) (size dim1) (size dim2)…
name:= node path.name
node path:= node. | node
length:= value| ppl expression
ppl expression:=value | prefix notation expression

Format **scr**:
storage storage_name [size dim1]
storage storage name [size dim1, size dim2 …]
storage (storage name) (size dim1)(size dim2) …

Examples:

```
1.
>code scr;
>recreate yes;
>storage x;
>storage x[2];   // 1-dimensional array
>storage (x)(2);
>storage x[2,3];// 2-dimensional array

>>var x = 3
>storage s[x + 1]
>swrite s
      0           0           0           0
```

```
storage s;  // storage will be resized in storage.ReadFile
storage.ReadFile (s)("examples\StorageLib\saved_10.csv")(",");
swrite s;
```

sinit

Init storage
Format **ppl**:
sinit (name)(initial value)
Format **scr**:
sinit name = initial value
sinit (name)(initial value)

initial value:= value| ppl expression

Examples:

```
>code scr;
>storage x[2,3];
>sinit x = 0;
```

sget

Gets value of element in storage
Format:
sget (name)(index1)(index2)…

Examples:

```
>storage s[3];
>var x = sget(s)(0); // get value of single-dimensional
                     //    array,index=0

 // get value of two-dimensional array,stor[0][0]
>storage s2[3,3];
>var x = sget(s2)(0)(0);
```

sset

Sets value for element in storage
Format **ppl**: **sset (name)(index1)(index2)…(value)**
Format **scr**: **sset name [index1,index2,…] = value**
In scr mode sset may be omitted.

Examples:

```
>code scr;
// set value of single-dimensional array,stor[0]= 1
>storage stor[3];
>sset stor[0] = 1;  // or stor[0] = 1;

// set value of two-dimensional array,stor[0][0] = 1
>storage stor[2,3];
>sset stor[0,0] = 1;
> var y = 1;
>sset stor[y,y] = 2; // it is possible to use var as index
>swrite stor 30;
        0         1         2

---------NS.Global.stor---------
[0]     1
[1]               2
```

swrite

Displays elements values of storage
Format **ppl**:

swrite(name)
swrite(name) (max_window_width)

Format **scr**:

swrite name | swrite (name)
swrite name max_window_width | swrite (name) (max_window_width)

default max_window_width = 100

Examples:

```
>code scr;
>storage s1[3]
>swrite s1
  0               0               0
>storage s2[3,4]
>swrite s2
  0               0               0               0
  0               0               0               0
  0               0               0               0


>storage s3[2,3,4]
>swrite s3
--- Slice [0] ---
  0               0               0               0
  0               0               0               0
  0               0               0               0

--- Slice [1] ---
  0               0               0               0
  0               0               0               0
  0               0               0               0
```

sinfo

Displays size of each dimension in storage and number of items
Format **ppl**:
sinfo(name)
Format **scr**:
sinfo name | sinfo (name)

Examples:

```
>storage ss[3,4,5];
>var x;
>x = sinfo (ss);
>d x;
x = [Storage s3 [3, 4, 5] [Total leaves: 60]]
```

sresize

Allows you to dynamically resize a `storage` component at runtime. It may be used to expand or shrink a 1D array, or to modify the length of specific rows in a multi-dimensional array, enabling the creation of jagged arrays.

Format **ppl**:
sresize(name)(ind1)(ind2)(indN)… (new size)
Format **scr**:

sresize name[index1, index2, ..., new_size]

- **name**: The name of the existing storage array.
- **index** (optional): The path to the specific row or dimension you want to resize. Omitted for 1D arrays.
- **new_size**: The last parameter in the brackets always specifies the new length of the target array/row.

Key Behaviors:

- **Safe Resizing:** Existing values within the new bounds are preserved.
- **Zero Initialization:** Any newly allocated elements are automatically initialized to `"0"`.
- **Bounds Protection:** The command includes strict boundary checks and will safely abort with an error message if an invalid path is provided.

Examples:

```
// 1D Array Resizing
storage s[2];
sresize s[5]; // Expands the array 's' to hold 5 elements.

// Jagged Array Resizing
storage jagged[3, 2];
sresize jagged[0, 5]; // Resizes only the 0th row to a length of
                      // 5 elements.
sresize jagged[1, 1]; // Shrinks the 1st row to a length of 1
                      // element.

//How to properly resize a one-dimensional array:
>storage s[2];
>sresize s[0,5];
Error: [FuncStorageResize] failed to resize. Invalid path or out
of bounds.
>sresize s[5];
>swrite(s);
    0           0           0           0           0
```

sdimsize

Returns the number of children (size) of the storage node reached by following a path of branch indices from the root.

Format **ppl**:
sdimsize(name)[(idx1)(idx2)...(idxN)][(out_var)]

Semantics:
- No indices -> size of the top-level dimension (axis 0), i.e. how many slices/branches the storage has at its root.
- Each additional (idxK) descends one level further, into the child at that index, and returns THAT node's size.
- Because storages can be jagged (see sresize), a path of the same length can return different sizes depending on which branch you follow -- there is no single global "size of axis N" once branches have been resized independently.
- If the last argument is not a number, it is taken as out_var and the result is also written into that variable (same effect as a separate assignment).

Limitation: dimension_index arguments must be literal integers or already-evaluated expressions; a math expression that can't be resolved to an int is not accepted here.

Error: if the path is invalid (out of bounds, or descends into a leaf), sdimsize reports an error instead of returning -1 silently.

Example (rectangular storage, s3[3,4,5]):

```
>storage s3[3,4,5];
>var x;
>x = sdimsize(s3);          // top level -- axis 0
>d x;
-L3       x                              [3]

>x = sdimsize(s3)(0);       // descend into branch 0 -- axis 1
>d x;
-L3       x                              [4]

>x = sdimsize(s3)(0)(0);    // descend into branch 0, then its
                            // branch 0 -- axis 2
>d x;
-L3       x                              [5]
```

Example (jagged storage, after independent sresize of branches):

```
>storage s[3,2,4];
>sresize s[0,4];   // slice 0 now has 4 rows
>sresize s[1,3];   // slice 1 now has 3 rows
>sresize s[2,5];   // slice 2 now has 5 rows

>var x;
>x = sdimsize(s);       // -> 3   (still 3 slices)
>x = sdimsize(s)(0);    // -> 4   (rows in slice 0)
>x = sdimsize(s)(1);    // -> 3   (rows in slice 1)
>x = sdimsize(s)(2);    // -> 5   (rows in slice 2)
>x = sdimsize(s)(1)(2); // -> 4   (columns in row 2 of slice 1)

>sdimsize(s3)(0)(0,x);   // out_var form: writes the result
                         // into x as well
```

Backup & Recovery

savedata | (sd)

Saves data from node to file with extension **.data**.
If node is root, all root contents will be saved.
Format **ppl**: **savedata | sd (filename.data) [(node)]**
Format **scr**: **savedata | sd filename.data [node]**

Default node: **NS.Global**

Example:

```
>storage s[3,3]
>array y[3]
>createnode Node1
>var Node1.x = 0
>sd "Examples\SaveGlobal.data"
(Global
  (empty)
    (s [Storage]
      (_shape_
        (# "3,3")
      )
      (row_0
        (# "0")
        (# "0")
        (# "0")
      )
      (row_1
        (# "0")
        (# "0")
        (# "0")
      )
      (row_2
        (# "0")
        (# "0")
        (# "0")
      )
    )
  (y [Array 3]
    (#)
    (#)
    (#)
  )
  (Node1 [Node]
    (x [0])
  )
)
```

readdata (rd)

Reads data from file file with extension **.data** to Nodes Configuration and NS.Global, **not to Local.**
Format **ppl**: **readdata | rd (filename.data)[(node)]**
readdata | rd filename.data

Format **scr**: **readdata | rd filename.data [node]**
Default node: **NS.Global**

Examples:

```
>code scr
>rd "Data\Colors.data";
>d;
-N  NS
---N  Global
-----N  Colors
-------L0       Black           [0]
-------L1       Blue            [9]
-------L2       Cyan            [11]
-------L3       DarkBlue        [1]
-------L4       DarkCyan        [3]
-------L5       DarkGray        [8]
-------L6       DarkGreen       [2]
-------L7       DarkMagenta     [5]
-------L8       DarkRed         [4]
-------L9       DarkYellow      [6]
-------L10      Gray            [7]
-------L11      Green           [10]
-------L12      Magenta         [13]
-------L13      Red             [12]
-------L14      White           [15]
-------L15      Yellow          [14]
```

Each item in this array has key (Black, Blue,…) and value(0,9,…), use getbykey and getbyvalue.

```
>createnode NODE
var filename = "data\Colors.data";
>rd filename NODE;
>d;
-N1     NS
---N2   Global
-----L0       empty                     (const)
-----N3 NODE                            [Node]
-------N4       Colors                          [Array 15]
---------L0       Black                         [0]
---------L1       Blue                          [9]
---------L2       Cyan                          [11]
---------L3       DarkBlue                      [1]
        .......
```

Control Flow

if, else

The meaning of the block "if-else" does not differ from the generally accepted.
See Base Concepts about using statements terminator ";" in if.

Format **ppl**:

```
if(condition)
(
  (statement)
  (statement)
  [(else
      (
        (statement)
         (statement)
      )
   )]
);
```

Here expression in prefix notation.
Statement in ppl mode.

Format **scr**:

```
if (condition)
{
    statement;
    statement;
  [else
  {
    statement;
    statement;
  }]
}
```

Here expression in infix notation.
Statement in ppl or scr mode.

Example ppl mode:

```
var(x[1]);
var(y[1]);
if (==(x)(y))
(
    (write(TRUE))
  (else
    (
      (write(FALSE))
    )
  )
);
TRUE
```

Examples in scr mode:

```
var x = 1;
var y = 1;
if ( x == y )
{
  write(TRUE);
}
else
{
  write(FALSE);
}
```

```
var x;
if(x == empty)
  write("x = empty");
```

If block in "if" or in "else" contains only one statement it is possible to omit { }:

```
code scr;
var x = 0;
if(x == 1)
  write(TRUE);
else
  write(FALSE);
FALSE
```

switch, case, default

switch statement – for select one from several case blocks to be executed.
About using statements terminator ";" in switch see Base Concepts.

Format **ppl**:

```
switch(expression)
(
    (case1) [(case2)…]
     (
        (statement)
        (statement)
         …
     )
     (caseN) …
      (
        (statement)
        (statement)
         …
      )
      …
      [ (default)
        (
            (statement)
            (statement)
      …
         )
       ]
);
```

Here expression in prefix notation.
Statements in ppl mode.

Format **scr**:

```
switch(expression)
{
   case <value>:
   case <value>:
     statement;
     statement;
     ...
  break;
  case <value>:
     statement;
     statement;
      ...
   break;
  [
  default:
    statement;
    statement;
```

```
    ...
    break;
  ]
```

}Here expression in infix notation.
Statement in r scr mode.

Examples:
Format **ppl**:

```
1.
switch(x)
(
  (1)(3)
  (
    (write("Cases 1 & 3"))
  )
);
2.
var (x[1]);
switch (x)
   (
   (1)(3)
   (
      (write("Case 1 & 3"))
   )
   (2)
   (
      (write("Case 2"))
   )
   (default)
   (
      (write("Default"))
   )
  );
   Case 1 & 3
```

```
3.Format scr
var x = 2;
switch(x)
{
   case 1: case 3:
       write("Case 1 & 3");
   break;
   case 2:
       write("Case 2");
   break;
   default:
      write("Default");
   break;
}
```

loop,do

Iteration block for ppl mode only.
About using statements terminator ";" in loop see Base Concepts.

Format **ppl**:

```
loop (iteration var)(begin)(end)[(step)]  or
 loop () //infinity loop
 (do
   (
     (statement)
     (statement)
       …
   )
 )
begin:= value|ppl expression
end:= value|ppl expression
step:= value|ppl expression
```

By default step = 1. Step may positive or negative.
Parameter end is set before entry to iteration block and may be changed by **setloopend** inside iteration block.
Statement in ppl mode.

Examples:

```
loop (i)(0)(10)(1)      // or loop (i)(10)(0)(-1)
(do
  (
     (write("i = {0}")(i))
  )
);
```

See infinity example – greatest common factor (gcf) calculation in for

setloopend

Changes "end" in loop (iteration var)(begin)(**end**)[(step)] inside iteration block
Format **ppl**:
setloopend(end_value)

Format **scr**:
setloopend end_value

Example:
1.

```
array arr[10] = 0;
for(j,0,length(arr))
{
  write#("j = {0}   length = {1}",j,length(arr));
  array.remove(arr)(0);
  setloopend length(arr);
}
writearray arr row;
j = 0   length = 10
j = 1   length = 9
j = 2   length = 8
j = 3   length = 7
j = 4   length = 6
-----Array arr-----
0, 0, 0, 0, 0
```

2. (see examples\lib\IsItemExist.scr)

```
// delete all repeated items from array
array arr[] = {a,1,2,a,2,3,4,4};
writearray arr row;
for(i,0,length(arr))
{
  for(j,i + 1,length(arr) )
  {
     if(arr[i] == arr[j])
     {
       array.remove (arr,j);
       setloopend length(arr);
     }
     setloopend length(arr);
  }
}
writearray arr row;
-----Array arr-----
a, 1, 2, a, 2, 3, 4, 4
-----Array arr-----
a, 1, 2, 3, 4
```

for

Iteration block for **scr mode** only.
About using statements terminator ";" in **"for"** see Base Concepts.

Format **scr**:

```
for(iteration var, begin, end [, step])  or
for()  // infinity for
{
   statement;
   statement;
   …
}
```

By default **step = 1**. Step may positive or negative.

Example:

```
var x;
for(i, 0, 10)
{
   x = i * 2;
   write#("x = {0}",x);
}
```

If block in "for" contains only one statement it is possible to omit { }:

```
for(i, 0, 10, 1)
  write#("i={0}",i);
```

Example with **infinity for**:

```
function gcd(x,y,z)
{
   if(isinteger (x) == "False")
   {
     write#("not integer value x={0}",x);
     return;
   }
   if(isinteger (y) == "False")
   {
     write#("not integer value y={0}",y);
     return;
   }
   for()   // infinity for
   {
     if (x > y)
     {
       x = x - y;       // or Sub(x)(x)(y);
     }
     if (x < y)
     {
       y = y - x;       // or Sub(y)(y)(x);
     }
     if (x == y)
     {
        z = x;
        return;
     }
   }
}
var z = 0;
var x = 14144;
var y = 26163;
call gcd(x,y,z);
write#(gcd = {0},z);
gcd = 17;
```

break

Exit from loop (ppl mode) or from for (scr mode) or end of case in switch block.

Example:

```
code scr;
for(i, 0, 4, 1)
{
  if (i == 2)
  {
    write#("i = {0} break",i);
    break;
  }
}
```

continue

Continue executing in loop (ppl mode) or in for (scr mode).

```
Example 1:
code ppl;
loop(x)(0)(5)(1)
 (do
   (
    (if (==(x)(3))
     (
       (write("x = {0} continue") (x) )
       (continue)
     )
    (write("x={0}") (x) )
  )
);

Example 2:
code scr;
for(x,0,5,1)
{
   if(x == 3)
   {
      write("x = {0} continue;",x);
      continue;
   }
   write#("x = {0}",x);
}
```

Input and Output

write

Writes the string value to the standard output stream.
String interpolation ($"x") is not supported. If string value begins with "Error:" it will be written in red.
Format:
write(value) | write(c# format)(value)(value)…
value:=value | ppl expression with prefix notation

Example:

```
>var  x = 0;
>write(x);
>write("{0}{1}")("x=")(x); // like c# write("{0}{1}")("x=",x);
//quote in string
>write("ppl\"language");  // ppl"language
//tab in string
>write("ppl\tlanguage");  // ppl language
//newline in string
>write("ppl\r\nlanguage");  // ppl
                            //language
>write(12col);
Error: [GetValue] non valid name [12col]
Error: [Traversal] [write] PrepareParameters return false
```

This operator is used in ppl and scr mode.

```
>code scr
>var x = 2*5;        // scr expression is calculated in var
>write("{0} {1}")( "x = " )(x)
>code ppl
>write("{0}")(*(2)(5))   // ppl prefix expression
>write(1+3)
Error: [funcWrite] name [1+3] write supports only prefix
 expression
>write#(1+3)    // use write# for infix expression
4
```

write#, writeline

Like as write, for mode scr only, each argument is not enclosed in parentheses.
Format **scr**:
write#(arg) | write#("c# format", arg1 , arg2,…)
writeline(arg) | writeline("c# format", arg1 , arg2,…)

arg:= <literal> | <scr expression with infix notation>

Examples:

```
>code scr;
>write#("{0} {1}", aaaa,1+3)
aaaa 4
>write#(1+2);
3
>write(1+2);   // not same result as with write#
Error: [funcWrite] name [1+2] does not exists or wrong math.
 expression
>write(+(1)(2));
3
>write#("1+2");
1+2
>write#("{0,-10}{1}\t{2}","PPL","Hello","World")
PPL    Hello World
```

writearray

Writes array contents to the standard output stream. By default writearray writes array elements into the column. Writearray writes array elements into the row by second argument **"row"**.

Format **ppl**:

writearray ([node.]array_name) [(row)] [mode]

Format **scr**:

writearray [node.]array_name [row] [mode]

or

writearray ([node.]array_name [,row] [mode])

mode = **true** | false

mode sets to write (**true**) or no array objects with value = 0; By default mode = **true.**

Examples:

1.

```
>code scr;
>array y[] = {1,2,3};
>writearray y;
-----Array y-----
[0]     1
[1]     2
[2]     3
>writearray y row
-----Array y-----
1, 2, 3
```

2.

```
>code scr;
>storage(x)(2)(3);
>sinit(x)(0);
// write the bottommost arrays in storage
>writearray (x.0.Row);
[0]     #       0
[1]     #       0
[2]     #       0
>writearray (x.1.Row, row);
0, 0, 0
```

3.

```
>code scr;
>array y[3];
>setkvp y[0] = one,1;
>setkvp y[1] = two,2;
>setkvp y[2] = three,3;
>writearray y row;
-----Array y-----
{one,1}, {two,2}, {three,3}
```

readline

Reads the next line of characters from the standard input stream. Result will be passed to calling operator.
Format**: readline() | readline**

Examples:

```
>var x;
>x = readline;
>Enter:
>>Hello
>d;
-N1     NS
---N1   Global
-----L0 x       [Hello]
```

Functions

Function library **CommonFunctions.ppl**, defined in file **Configuration.data** as **default_loaded_functions**, loads automatically and reloads when command **init** executes. It includes 2 types of functions:
-Mathematical and Logical functions,
-Array services and other functions.

Mathematical and Logical functions:

Sum (_result)(n1)[(n2)]
Sub (_result)(n1)[(n2)]
Mult (_result)(n1)[(n2)]
Div (_result)(n1)[(n2)]
Pow (_result, n1, n2)
PlusPlus (_result) | Plus1 (_result) // like c#: ++(var)
MinusMinus (_result) | Minus1 (_result) // like c#: --(var)
LT (_result, n1, n2)
LTEQ (_result, n1, n2)
GT (_result, n1, n2)
GTEQ (_result, n1, n2)
EQ (_result, n1, n2)
NOTEQ (_result, n1, n2)
AND (_result, n1, n2)
OR (_result, n1, n2)
XOR (_result, n1, n2)

These functions replace using prefix notations. Result are returned in 1st parameter and does not passed to the next command:

```
>var (z)(0);
>var(x)(1);
>set(z)(Sum(x)(1));     // error
>Sum(z)(x)(1));         // right
```

Examples:

```
1.
>code ppl;
>var(x)(1);
>var(z);
>Sum(z)(x)(1);  // z = x + 1
>Sum(x)(2);     // x = x + 2
>d
-N2     NS
---N3   Global
-----L4 x       [3]
-----L5 z       [2]
```

```
2. This sample returns wrong result:
>var(x)(5);
>Sum(x)(x)(2);
Info: using same args can lead to unpredictable results [Sum]
 [x]
>d x
-----Variables  and arrays-----
-L4     x       [5]
>Sum(x)(2);   // right, x = x + 2
```

```
3.
>code scr;
>call Sum(x,2,3);
>call Sum(x,2+3);
>Sum(x)(+(2)(3));
```

User may create own functions file, like **CommonFunctions.ppl**, and set it in file **Configuration.data** as **UserFunctionsN** or load it:

```
>rc user_functions.ppl|scr;
```

Files, defined in **Configuration**, load their function only and do not execute any command, commands in files, loaded by command rc (readcode) are executed one after another.

Array services and other functions:

CsvToArray (var:str, array:arr) – to fill array from string data separated by comma
ArrayToCsv (array:arr, var:str) – copy to string separated by comma data from array
ArrayIsExist (_result, array:arr, value) – return True/False if defined value exists in array
ArrayIndexOf (_result, array:arr, value) – return the index of the first occurrence within array or -1.
ArrayLastIndexOf (_result, array:arr, value) – return the index of the last occurrence within array or -1.

Example:

```
>array my_array;
>call CsvToArray("1,2,3,,1,5",my_array);
>writearray my_array row;

>var Result;
>call ArrayToCsv(my_array,Result);
>write#("Result = {0}",Result);

>call ArrayIndexOf  (Result, my_array, 3);
>write#("index = {0}",Result);

>call ArrayLastIndexOf  (Result, my_array, 1);
>write#("index = {0}",Result);
```

```
1, 2, 3, , 1, 5
Result = 1,2,3,,1,5
index = 2
index = 4
```

See **examples\ArrayFunc\SamplesFunc.scr**.

ArrayForEach(array:arr, array:callback_name) – calls a callback function once for each array element

Example: (examples\arrayfunc\foreach.scr)

```
function sumfunc(array:arr,var:i)
{
   result = result + arr[i];   // used reserved name result
}

delegate d2 (array:arr,var i);
dlgtinstance instance d2;
dlgtset instance sumfunc;
call ArrayForEach({1,2,3,4,5},instance);
write#("getresult = {0}",getresult);
result = 15
```

WindowSize() // is executed in Administrator mode only
width=120 height=30

function

Functions must be declared before being called.
Functions are saved in Tree **Functions** or in Tree **Global** for later call.
Functions return result via parameters, like a classic procedure, and via operator "return" (see Examples\ArrayFunc\mean.scr).

When function is called internal **var result** is created automatically**,** it may be used as any var, when function is ended value of var result received to caller and var result will be deleted.
When function is called **by name** (in mode **ppl** or **scr**) each argument must be enclosed in parentheses.
When function is called by command **'call'** (in mode **scr** only) each argument not must be enclosed in parentheses, but separated by comma.
Function uses data created inside, passed from calling function and data from NS.Global.
Data created in function are deleted when function will be finished.
The function receives parameters **by reference**. If the argument is a literal, then attempting to change it will result in an error message.

Limitations:

1. If argument value will be changed in function this argument may be used only one time when function is called(see example **Sum_wrong_result**).

2. Do not create function in function:

```
function f()        //error
{
   function s() {…};
}
```

Format **ppl**:

Statement terminator ';' does not follow after statements within function, but each statement is surrounded by parentheses.

```
function
(
   name
  parameter_list
  ( function body )
);
name::=  identifier
parameter_list::= parameter [parameter_list]
parameter::=  (identifier) | (identifier[default value]) | empty
function body::= (statement1) [(statement2) (statementN)]
identifier::= [var] | [array]  | [storage]  |
       [struct <struct_name>] | [struct array <struct_name>]:<param_name>
```

Format **scr**:

Statement terminator '**;**' always follows after each type of statements within function.

```
function     name
             (parameter_list)
          {
                function body
          }
name::=  identifier
parameter_list::= parameter, [parameter_list]
parameter::=
          identifier | identifier[default value] | identifier = default value | empty
function body::= statement1; [ statement2;   statement; ]
identifier::= [var] | [array] | [storage]:<name>
```

By default parameter type is var.

Examples:

1.function without parameters

```
>code scr
Function func ()
{
   write#("func");
}
```

```
2. function with default parameter
function func (n,m[10])   //= func (var:n,var:m[10])  or
                              //function func (n,m = 10)
{
   write#(funcname());
}
```

```
3. array parameter
function func (array: n)
{
  write#( funcname());
}
```

4. different argument types

```
function f(var: param)
{
  write#("{0}  {1}",getargname(param), param);
}

array y[] = {1,2,3,4,5};
f(y[0]);
var x = 0;
f(x);
storage s[2,2];
s[0,0] = 10;
f(s[0,0]);
y  1
x  0
s  10
```

5. call function from function

```
 code ppl;
function
(
  test2(n)
  (
     (write(n))
  )
);
function
(
 test()
 (
    (loop (i)(0)(5)(1)
     (do
      (
        (test2(i))
      )
    )
   )
 )
);
test();     // function call
```

6.

```
 code scr;   // (see  examples\scr\func.scr)
function sum_arr(array:n,array:m)
{
  for(i,0,length(n))
    write#("[{0}]  [{1}]" ,i, n[i] + m[i]);
}
d Functions.sum_arr;
array x[] = {1,2,3,4,5};
array y[] = {6,7,8,9,10};
```

```
sum_arr(x)(y);  // function call
// command "call" allows use arguments by this manner:
// call sum_arr({1,2,3,4,5},{6,7,8,9,10});
[0]   [7]
[1]   [9]
[2]   [11]
[3]   [13]
[4]   [15]
```

In the following example (see examples\scr\func4.scr) parameter index default value = 0 and this parameter may be omitted when the function is called.
Wen function is calling item array may be passed as 2 arguments (array, index)

7.

```
code scr;
function func (array:arr, var:index[0])
{
  write#("{0}[{1}] = {2}",getargname(arr),index, arr[index]);
}
array y[] = {1,2,3,4};
func(y);
call func(y);   // same as previous line
call func(y,0);
call func(y,1);
call func(y,2);
y[0] = 1
y[0] = 1
y[0] = 1
y[1] = 2
y[2] = 3
```

8. Call function from node saved in Tree Global

```
code ppl;
createnode New;
function
(
   New.func(name)     // public
   (
     (write(name))
    )
);
function
(
      New._hfunc(name)   // private
      (
        (write(name))
      )
)
```

9. Call function from node created in Global

```
code scr;
createnode N;
function N.f()
{
  write("Global.N.f - function");
};
N.f();
dn;
Global.N.f - function
-N2     NS
---N3   Global
-----N4 N                [Node]
-------N5       f                [function]
---------N6     #                [internal_block]
-----------N7   write
```

```
 // Call function from node created in Functions
createnode Functions.N;
function Functions.N.f()
{
  write("Functions.N.f - function");
};
Functions.N.f();
 Functions.N.f - function
```

10. Recursion

```
var tmp = 0;
function  rec(x)
{
    PlusPlus(tmp);
    write#("tmp={0}",tmp);
    if (tmp == x)
     {
       return;
     }
    rec(x);
}
rec(5);
```

11. warning when function updated

```
function f()
{
  write("function f");
}
f();
function f()
{
  write("updated function f");
}
f();
```

```
function f
Warning: [FuncCreateFunction] function [f] is updated
updated function f
```

12. Passing structure as function parameter

```
definestruct Room
{
  var x;
  array y[3];
}
// function f(struct Toom:r) - wrong struct_name causes error
function f(struct Room:r)
{
  r.x = 1;
  r.y[0] = "A";
}
createstruct R as Room;
f(R);
```

13. Passing structure array as function parameter

```
definestruct Room
{
  var x;
  array y[3];
}
function f(struct array Room:r)
{
  r.0.x = 1;
  r.0.y[0] = "A";
}
createstruct R[2] as Room;
f(R);
```

Examples with using public and private functions in Trees Functions and Global it is possible to find in directory Examples\Access.

14. **Array as parameter** (see Examples\ArrayFunc\mean.scr)

```
function mean2(array:arr)
{
   result = 0;   // result is internal var, created in function
   for(i,0,length(arr))
       result = result + arr[i];
   result = result/ length(arr);
   //return result/ length(arr);
}
call mean2({1,2,3,4,5,6,7,8,9,10});
write#("mean2 = {0}",getresult);

array arr[] = {1,2,3,4,5,6,7,8,9,10};
write#("mean2 = {0}",mean2(arr));
mean2 = 5.5
mean2 = 5.5
```

or array item:

```
function f(var: n)
{
   n = 5;
}
array N[3] = 0;
f(N[0]);
f(N[2]);

writearray N row;
-----Array [N]-----
5, 0, 5
```

15. function names - funcnames

```
function fora()
{
  write(funcname);
}
fora();

function defaulta()
{
  write(funcname);
}
defaulta();
```

16.

```
function f(x)
{
   write(x);
}
var c = "qqq";
f("v");   // literal
f(c);     // variable
f(v);     // error wrong argument name

>rc examples\CallFunc\func21.scr
v
qqq
Error: [FuncExecFunction] function [f] argument [v] not found
```

17. attempt to change literal argument

```
function f(var:c)
{
   write#(" c={0}",c);
   c = 20;
   write#("c={0}",c);

}
f(10);
c=10
Error: [FuncExecFunction]  attempt to change literal parameter
 [c]  arg  [10]
```

call

Command '**call**' invokes a function in mode **scr**, it is possible to use expression in infix notation as arguments, do not need to enclose in parentheses each argument when function is called.
Do not use '**call**' as function argument or on the right side of mathematical expression or as argument of any command.

Format **scr**:
call function_name(arg1,arg2,…)

Examples:

```
1.
>code scr
>var x = 0;
>call Sum(x,4 + 1);     // right: 2nd arg in infix notation
>d x
-----Variables  and arrays-----
-L1      x        [5]

>call Sum(x,-(5)(2));  // wrong: 2nd arg in prefix notation
                       // right: Sum(x)(-(5)(2))

2.Call function without arguments or with one not-expression
argument by command 'call' and without 'call':
call func(123);   same as  func(123);

3. To get return of function, called by 'call',
   run 'dbg yes' before:

>dbg yes;
>call Math.PI();
result = 3.141592653589793
// or
>debugppl no;
>call Math.PI();
>write(getresult);
3.141592653589793

4.To get return from function, when function is used as
argument do not use 'call', use ppl_notation:
>write#( Math.PI() );
 result = 3.141592653589793

5.
>write#(call Math.PI());     // error: call on the right side
>var x = call Math.PI();     // error
>var x = Math.PI();          // right
3.141592653589793
```

6. command "call"allows you to pass array by this manner:
call function_name({item1,item2,…},arg2,…)

Example: (see examples\CallFunc\funcarr2.scr and examples\CallFunc\funcarr2.ppl)

```
function SumArray(array:arr,var:r)
{
   for(i,0,length(arr))
     call Sum(r,arr[i]);
}

var R = 0;
call SumArray({1,2,3,4,5},R);
write#("R = {0}",R);
```

or:

```
array arr[] = {1,2,3,4,5};
call SumArray(arr,R);
```

Preprocessor generates the following ppl-code:

```
function
(
  SumArray (array:arr)(var:r)
  (
    (set (r)(0))
    (
      loop (i)(0)(length(arr))(1)
      (
        do
        (
          (Sum(r)(r)(arr[i]))
        )
      )
    )
  )
);
var (R);
array (arg_array0)(1)(2)(3)(4)(5 );
SumArray (arg_array0 )( R);
del arg_array0;
write ("R = {0}")(R);
```

```
array split_array;
call String.Split("1 2 3 4 5",{" "},split_array);
>writearray split_array row
-----Array [split_array]-----
1, 2, 3, 4, 5
```

return

Returns from function or exit from script, passes result from called function.

Format **ppl: return || return (statement in prefix notation)**

Format **scr: return | return (statement in infix notation)**

Examples:

1.

```
function f()
{
  for(i, 0, 5)   // step by default = 1
  {
     if (i == 2)
      return;
     write(i);
   }
}
f();
write("end of script");
0
1
```

2.

```
code scr;
function f()
{
  return 2 + 3;
}
f();
write(getresult);
5
```

getresult

Gets result of return passed from called function.

Format:

getresult () | getresult

Examples:

1. Return from function:

```
function sum(x,y)
{
   return x + y;
}
call sum(1,2);
write#("result = {0}",getresult);
result = 3
     or
write#("result = {0}",getresult(sum(1)(2)));
```

funclist

Displays loaded function names and their parameters from node Functions.

Format:

funclist| finclist()

Example:

```
>funclist;
-----Function List-----
Sum  (_result, n1, n2)
Sub  (_result, n1, n2)
Mult  (_result, n1, n2)
Div  (_result, n1, n2)
Pow  (_result, n1, n2)
PlusPlus  (_result)
MinusMinus  (_result)
LT  (_result, n1, n2)
LTEQ  (_result, n1, n2)
GT  (_result, n1, n2)
GTEQ  (_result, n1, n2)
EQ  (_result, n1, n2)
NOTEQ  (_result, n1, n2)
AND  (_result, n1, n2)
OR  (_result, n1, n2)
XOR  (_result, n1, n2)
```

funcname

Returns the current function name.
Format:
funcname() | funcname

Examples:

```
>write(funcname);
main
```

```
function MyFunction()
{
  write(funcname);
}
MyFunction();
MyFunction
```

argc

Returns number of arguments
Format:
argc() | argc

Example:

```
function Sum (_result,n1,n2 = "")
{
    write#("argc = {0}",argc());
    if (argc() == 2)
      _result = _result + n1;
    else
      _result = n1 + n2;
}
>code scr;
>var x = 0;
>call Sum(x,1);
>write#("x = {0}",x);
argc = 2
x = 1
```

getargname

Returns argument name (or argument value if it is literal) by parameter name
Format:
getargname (parameter_name)

Example:

```
function a(n)
{
 write#("param name=[{0}] arg name=[{1}]",
       getname(n),getargname(n));
}
var i = 0;
a(i);
param name=[n]  arg name=[i]
```

finally and failure blocks

finally block are called at the end of the successful completion of the script or functions.
failure block are called in case of emergency termination of the script or functions.
These blocks are added to the body of script or functions as additional nodes and have access to all variables in their scope.

Example:

```
finally  { write("script finally block"); }
failure  { write("script failure block"); }

function foo(var: x)
{
  finally
  {
    var str1 = "function finally block";
    //writ(str1);
    //Error: [Traversal] [foo] [finally] wrong cmd or function
    //   name [writ]
    write(str1);
  }

  failure
  {
    var str2 = "function failure block";
    write(str2);
  }

  if(isdigits (x) == True)
  {
     write("digits");
     return;
  }
  else
  {
     write("not digits");
     return;
  }
}
foo("a");
```

```
not digits
function finally block
script finally block
```

Delegates and callbacks

There are 4 operators for using delegates:
delegate – creation delegate
dlgtinstance – creation delegate instance
dlgtset - setting the function to delegate instance
dlgtcall – call function by delegate instance

delegate

delegate is created as an array whose elements define the method parameters**.**
Prefix "**delegate_**" **is added to delegate name** for internal using.
Format **ppl**:
delegate (<delegate name>) (param1) (param2)...
param:= <var_name> | var:<var_name> | array:<array_name> |
storage:<array_name> | struct [array] <struct_name>: < var_name >
Format **scr**:
delegate <delegate name> (param1, param2,…)

Example (**scr-mode**):

```
>delegate  MyDelegate (var:x,array:arr);
Generated ppl code:
>d
-N2     NS
---N3   Global
-----L0       empty                        (const)
-----N4 delegate_MyDelegate                [Array 2]
-------L0       #                                [var:x]
-------L1       #                                [array:arr]
```

dlgtinstance

dlgtinstance is created as an array with 2 elements, first is delegate name, second is empty and will be set by dlgtset. Delegate parameters types must be matched types of function parameters . Prefix **"dlgtinstance_"** is added to delegate instance name for internal using.
Format **ppl**:
dlgtinstance (<delegate instance name>)(<delegate name>)
Format **scr**:
dlgtinstance <delegate instance name> <delegate name>

Example(**scr-mode**):

```
>delegate d2 (var:x,array:arr);
>dlgtinstance instance d2;
>d;
-N2     NS
---N3   Global
-----L0 empty   (const)
-----N4 delegate_d2     [Array 2]
-------L0       #       [var:x]
-------L1       #       [array:arr]
-----N4 dlgtinstance_instance     [Array 2]
-------L0       #       ["delegate_d2"]
-------L1       #
```

dlgtset

dlgtset sets function name as second element in **dlgtinstance** array.
Format **ppl**:
dlgtset (delegate instance name)(function name)
Format **scr**:
dlgtset delegate instance name function name

Example (**scr-mode**):

```
// see previous example with delegate and dlgtinstance
function f1(var:x,array:arr)
{
   write#("x = [{0}]",x);
   writearray arr;
}
>dlgtset instance f1;
```

dlgtcall

dlgtcall calls function defined in dlgtset.
Format **ppl**:
dlgtcall(delegate instance name)(arg1)(arg2)(arg3)…
Format **scr**:
dlgtcall delegate instance name(arg1,arg2,arg3,…)

Example:

```
// see previous examples with delegate, dlgtinstance and dlgtset
>var z = "qqq";
>array y[] = {1,2,3};
>dlgtcall instance (z,y);
x = [qqq]
-----Array arr-----
[0]     1
[1]     2
[2]     3
```

See samples with delegates - Examples\delegates*.scr

callback

callback invokes synchronous callback method.

Format **ppl**:

callback (callback name)(arg1)(arg2)(arg3)…

Format **scr**:

callback callback name (arg1,arg2,arg3,…)

Example: (see Examples\delegates\callback.scr)

```
function cb1(var:n)
{
  write#("=====(0)=====",funcname);
  write#("n = (0)",n);}

function cb2(var:n)
{
  write#("=====(0)=====",funcname);
  write#("n = (0)",n);}

function f(array:x,var:str)
{
  callback x(str);
}

delegate d2 (var:n);
dlgtinstance instance d2;
dlgtset instance cb1;
call f(instance,"PPL");
dlgtset instance cb2;
call f(instance,"PPL");
function cb1 PPL
function cb2 PPL
```

VS Code support

Enhance your ForestPPL development experience by enabling custom syntax highlighting in Visual Studio Code. You can easily add the official syntax rules to your editor using the provided extension package.

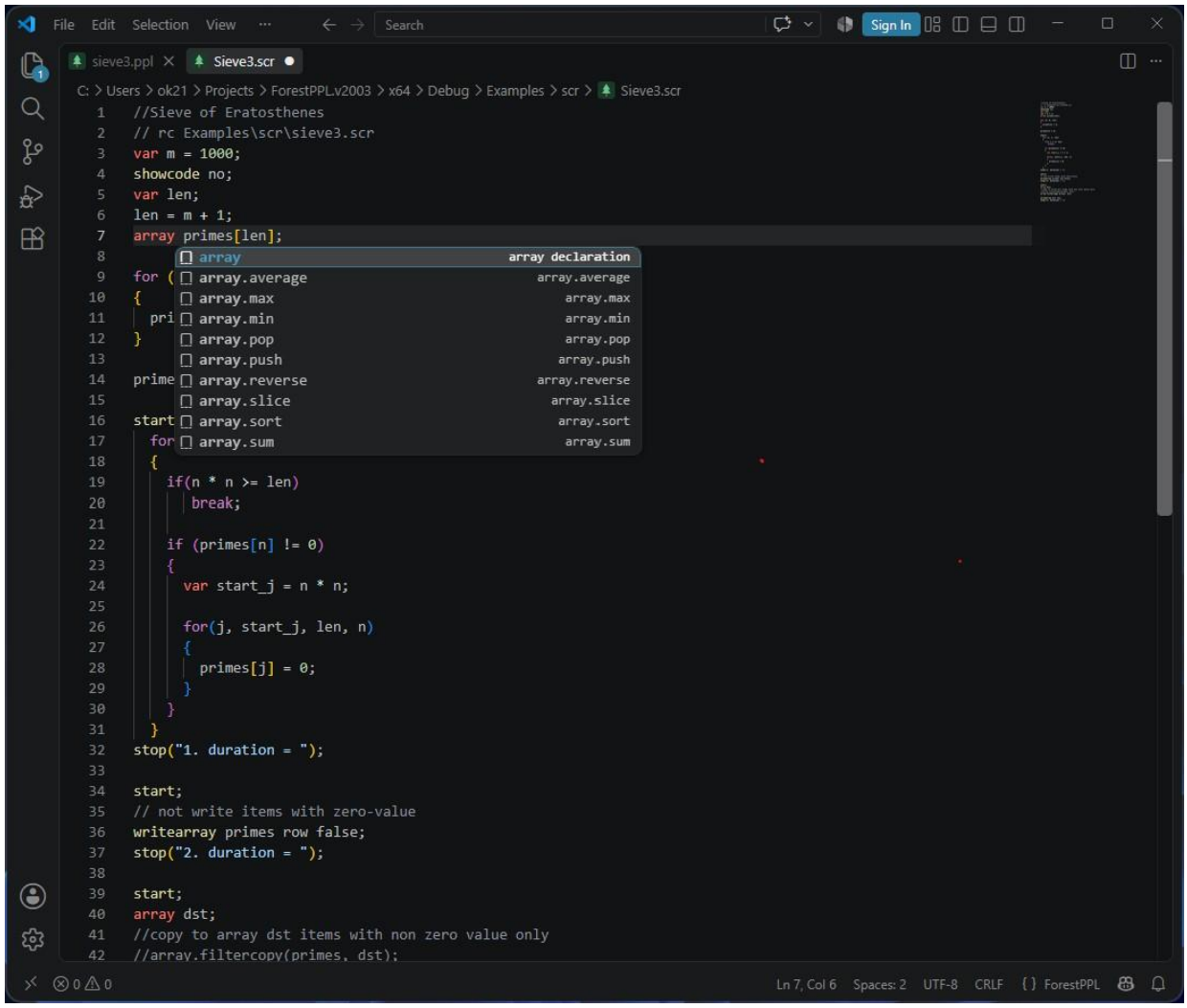
Installing the ForestPPL Extension via UI

- Open Visual Studio Code.
- Navigate to the Extensions view by clicking the square icon in the Activity Bar on the left side of the window (or press Ctrl+Shift+X).
- Click the Views and More Actions menu (the three dots ... located at the top right corner of the Extensions panel).
- Select Install from VSIX... from the dropdown menu.
- Locate and select the **VS.Code\forestppl-vscode-0.3.0.vsix** file on your local machine, then click Install.
- Restart VS Code if prompted.

Alternative Installation via Command Line If you prefer using the terminal, you can install the extension directly by running the following command in the folder where your file is located:

code --install-extension VS.Code\forestppl-vscode-0.3.0.vsix

Once the installation is complete, Visual Studio Code will automatically recognize your ForestPPL script files. The editor will apply the dedicated color scheme and syntax highlighting to your commands, keywords, and variables, making it significantly easier to read, navigate, and debug your code.


File Edit Selection View ···
Search
Sign In
sieve3.ppl
Sieve3.scr
C: > Users > ok21 > Projects > ForestPPL.v2003 > x64 > Debug > Examples > scr > Sieve3.scr
//Sieve of Eratosthenes
// rc Examples\scr\sieve3.scr
var m = 1000;
showcode no;
var len;
len = m + 1;
array primes[len];
array array declaration
array.average array.average
array.max array.max
array.min array.min
array.pop array.pop
array.push array.push
array.reverse array.reverse
array.slice array.slice
array.sort array.sort
array.sum array.sum
for (
{
pri
}
prime
start
for
{
if(n * n >= len)
break;
if (primes[n] != 0)
{
var start_j = n * n;
for(j, start_j, len, n)
{
primes[j] = 0;
}
}
}
stop("1. duration = ");
start;
// not write items with zero-value
writearray primes row false;
stop("2. duration = ");
start;
array dst;
//copy to array dst items with non zero value only
//array.filtercopy(primes, dst);
Ln 7, Col 6 Spaces: 2 UTF-8 CRLF { } ForestPPL

Additional functionalities

The following below-mentioned additional DLLs with C++ functionalities are added and this list will be expanded. Custom DLLs use functions from the C++ STL and internal PPL functions.
To get list of methods of additional loaded DLLs:
<DLLname>.help

For using user's library it is needed to set it in Configuration.data or to add by command **import** in program.

Use Utility **CodeGen.exe** to create code for additional DLLs. Arguments of additional DLLs use **prefix math.notation, only Math library uses infix math. notation in scr-mode, this feature is added to preprocessor**:
x = Math.Sqrt(1+3) + Math.Max(2)(3*5);

Math

Methods:

Max	**E**	**PI**
Min	**Exp**	
BigMul	**Floor**	
Sqrt	**Log**	
Round	**Log10**	
Abs	**Pow**	
Acos	**Sign**	
Asin	**Sin**	
Atan	**Tan**	
Atan2	**Truncate**	
Ceiling	**Tanh**	
Cos	**Cosh**	
DivRem	**Sinh**	

Limitation: Do not use Math.Function in Math.Function:

```
var x = Math.Sin(30/(180/Math.PI())) // error
```

To get short help for every method in Math.DLL:
>Math.help[(method name)];

Returns the larger of two double-precision floating-point numbers:
Math.Max(double d1)(double d2)

Returns the smaller of two double-precision floating-point numbers:
Math.Min(double d1)(double d2)

Produces the full product of two 32-bit numbers:
Math.BigMul(Int32 n1)(Int32 n2)

Returns the square root of a specified number: **Math.Sqrt(double d1)**

Rounds a double-precision floating-point value to a specified number:
Math.Round (double value)[(Int32 digits)]

Returns the absolute value of a double-precision floating-point number:
Math.Abs(double value)

Returns the angle whose cosine is the specified number**: Math.Acos(double d)**
Returns the angle whose sine is the specified number: **Math.Asin(double d)**

Returns the angle whose tangent is the specified number**: Math..Atan(double d)**

Returns the angle whose tangent is the quotient of two specified numbers:
Math.Atan2(double d1)(double d2)

Returns the smallest integral value greater than or equal to the specified number:
Math.Ceiling(double d)

Returns the cosine of the specified angle: **Math.Cos(double d)**

Returns the quotient and remainder, separated by '; ' as result:
Math.DivRem(Int64 n1)(Int64 n2)
Example:

```
write#("result: {0}",Math.DivRem(9)(2));
result: 4;1
```

Represents the ratio of the circumference of a circle to its diameter: **Math.PI()**

Represents the natural logarithmic base: **Math.E()**

Returns e raised to the specified power: **Math.Exp(double value)**

Returns the largest integral value less than or equal to the specified number:
Math.Floor(double value)

Returns the logarithm of a specified number: **Math.Log(double value)**

Returns the base 10 logarithm of a specified number: **Math.Log10(double value)**

Returns a specified number raised to the specified power:
Math.Pow(double value)(double power)

Returns an integer that indicates the sign of a double-precision floating-point number:
Math.Sign(double value)

Returns the sine of the specified angle: **Math.Sin(double value)**

Returns the tangent of the specified angle: **Math.Tan(double value)**

Calculates the integral part of a number: **Math.Truncate(double value)**

Returns the hyperbolic tangent of the specified angle: **Math.Tanh(double value)**

Returns the hyperbolic cosine of the specified angle: **Math.Cosh(double value)**

Returns the hyperbolic sine of the specified angle: **Math.Sinh(double value)**

String

Methods:

Compare	**Replace**
Concat	**DeleteEndOfLine**
Contains	**StartsWith**
Format	**Substring**
IndexOf	**ToCharArray**
LastIndexOf	**ToLower**
Insert	**ToUpper**
Remove	**Trim**
Split	**Char**
SplitCsv	

To get short help for every method in String.DLL:
>String.help[(method name)];

Returns signed int as string: **String.Compare(stringA)(stringB)**

Returns concatenation of several strings: **String.Concat(string1)(string2)**...

Returns true|false: **String.Contains(string)(specified substring)**

Converts the value of objects to string based on the formats specified and returns result:
String.Format(format)(string1)(string2)...
Example:

```
String.Format("{0} {1}")("qwe")("zxc")
qwe zxc
```

Returns a new string in which a specified number of characters from the current string are deleted:
String.Remove(string)(startIndex)(number of deleted symbols)
Example:
>rc examples\lib\StringRemove.scr

```
import String;
array primes[] = {1,2,3,5,7};
var output = "";
for(i,0,length(primes),1)
{
  output = String.Concat(output)(primes[i])(",");
}
var index = length(output) - 1;
output = String.Remove(output)(index)(1);//remove the latest ','
write(output);
1,2,3,5,7
```

Returns a new string in which one or all occurrences of a specified Unicode character
or string in the current string are replaced with another specified Unicode character or string:
String.Replace(string)(old value)(new value)[mode]
mode:=one|last|all
by default: **all**

```
>dbg yes
>String.Replace("Hello Helo")("l")("f")
Heffo Hefo
>String.Replace("Hello Helo")("l")("f")(one)
Heflo Helo
>String.Replace("Hello Helo")("l")("f")(last)
Hello Hefo
>String.Replace("Hello Helo")("l")("f")(all)
Heffo Hefo
>String.Replace("Hello Helo")("l")("f")(www)
Error: [String.FuncReplace] invalid mode [www]. Expected 'one',
'last', or 'all'
```

Determines whether this string instance starts with the specified character:
Returns **True**|**False**:
String.StartsWith(string)(value)

Retrieves a substring from this instance. The substring starts at a specified character position and has a specified length:
String.Substring(string)(startIndex)[(length)]
Examples:

```
>var r = String.Substring("123asd")(0)(3);
r = 123
>var r = String.Substring("123asd")(3);
r = asd
```

Writes the characters in this instance to a Unicode character array:
String.ToCharArray(string)("ppl_chars_array")
node_of PPL_chars_array is string in quotes or value of variable.

```
Example:
>array chars;
>String.ToCharArray("qwerty")("chars"));
>writearray chars row;
importlist-----Array chars-----
q,w,e,r,t,y
```

Returns a copy of this string converted to lowercase: **String.ToLower(string)**

Returns a copy of this string converted to uppercase: **String.ToUpper(string)**

Returns a new string in which all leading and trailing occurrences of a set of specified characters from the current string are removed:
String.Trim(string)[(trim chars string)]

```
>String.Trim(" abcde")(" ae");
bcd
```

Returns one character from string: **String.Char(string)(index)**

Returns the resized string array that contains the substrings in this instance that are delimited by elements of a specified string array or in special string var, string array must be created before with size = 0:
String.Split(string)("ppl_array_separators")(ppl_array_result) or
String.Split(string)("var_separator")(ppl_array_result)
It is possible to use **comma** instead of **','**
space instead of **' '**
tab instead of **'\t'**
String.Split does not skip separators between quotes.

Example:

```
>var text;
>text = File.ReadAllText("Data\test.csv");
>d text;
-----L1       text    [<1 2 3 4 5 6 7 8 9 10>]
>array separator[] = {comma,tab,space};
>array split_text_array;
>var len = length(text) - 2;
>text = String.Substring(text)(1)(len);
>d text;
-----L1       text    [1 2 3 4 5 6 7 8 9 10]
>String.Split(text)(separator)(split_text_array);
>writearray split_text_array row
-----Array [split_text_array]-----
1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

Returns the resized string array that contains the substrings in this instance that are delimited by separator of a specified string var, string array must be created before with size = 0 (array <name>;).
If value with separator is surrounded by quotes it doesn't split.
(see example\lib\splitcsv.scr):

array array_result;
String.SplitCsv(string)("var_separator")(array_result)
It is possible to use **comma** instead of **','**
space instead of **' '**
tab instead of **'\t'**

String.SplitCsv skips separators between quotes.

Example:

Returns string from File.ReadAllText without EndOfLine: **DeleteEndOfLine(string)**

```
>rc Examples\Lib\FilesplitCsv.scr

var text = File.ReadAllText("examples\lib\splitcsv.txt");
array splitcsv_text;   // array with results
call String.SplitCsv(text, ",", splitcsv_text);
writearray splitcsv_text row;
call File.WriteAllText(text,"examples\lib\splitcsv_copy.txt");
```

Reports the zero-based index of the first occurrence of the specified string in this instance:
String.IndexOf (string)(value)[(start_index)][(count)]

Reports the zero-based index of the last occurrence of the specified string in this instance:
String.LastIndexOf (string)(value)[(start_index)][(count)]

Example (see examples\lib\IsUniqueSymbol.scr)

```
function IsUnique(str,res)
{
  var char;
  for(i,0,length(str))
  {
   char = String.Char(str)(i);
   if( String.LastIndexOf (str)(char) != i)
   {
     res = False;
     return;
   }
  }
  res = True;
}

var Result;
call IsUnique("abcdef",Result);
write(Result);
call IsUnique("1234567",Result);
write(Result);
call IsUnique("abcABC",Result);
write(Result);
call IsUnique("abcadef",Result);
write(Result);
True
True
True
False
```

Returns a new string in which a specified string is inserted at a specified index position in this instance:
String.Insert (string)(start index)(string to insert)

Directory

Methods:

GetFiles
GetDirectories
SetCurrentDirectory
GetCurrentDirectory
GetParent
CreateDirectory
Exists
Delete

To get short help for every method in Directory.DLL:
> Directory.help[(method name)];

Writes the names of files (including their paths) in the specified directory to node_of_PPL_array, created before,its previous content will be cleared:
array PPL_array;
Directory.GetFiles(PPL_array)(path)
path = var | string_literal
Example:

```
function WriteFilesInDirectory (array:arr,var:dir)
{
  write#("[{0}] [{1}]", getargname(arr),getargname(dir));
  call Directory.GetFiles(getargname(arr),dir);
  writearray arr;
}
var c = "c:\\";
array files;
call WriteFilesInDirectory (files,c);
```

```
[files] [c]
-----Array [arr]-----
[0]     c:\appverifUI.dll
[1]     c:\devlist.txt
[2]     c:\DumpStack.log
[3]     c:\DumpStack.log.tmp
[4]     c:\Finish.log
[5]     c:\hiberfil.sys
[6]     c:\logUploaderSettings.ini
[7]     c:\logUploaderSettings_temp.ini
[8]     c:\pagefile.sys
[9]     c:\swapfile.sys
[10]    c:\vfcompat.dll
```

Writes the names of directories (including their paths) in the specified directory to **PPL_array**, created before,its previous content will be cleared:

array PPL_array;
Directory.GetDirectories(PPL_array)(path)

Example:

```
>import Directory;
>array dir;
>Directory.GetDirectories(dir)("c:\Users");
```

Sets the current working directory to the specified directory:
Directory.SetCurrentDirectory(path)

Gets the current working directory: **Directory.GetCurrentDirectory()**

Returns parent fullname: **Directory.GetParent(path)**

Returns CreationTime: **Directory.CreateDirectory(path)**

Returns **True** or **False** : **Directory.Exists(path)**

Deletes the specified directory and any subdirectories and files in the directory
Returns **True** or **False**: **Directory.Delete(path)**

Queue

Methods:

Create	**Peek**	
Count	**Clear**	
Write	**Contains**	
Enqueue	**AddArray**	
Dequeue	**ToArray**	**Delete**

To get short help for every method in Queue.DLL:
Queue. help[(method name)]

Creates Queue object: **Queue.Create(queue name)**
Examples:

```
>import Queue;
>Queue.Create("q");
>var name = "Q";
>Queue.Create(name);
```

Returns the number of elements actually contained in Queue: **Queue.Count(queue name)**

Writes queue names or all elements from the specified queue to the standard output stream:
Queue.Write() or Queue.Write(name)

Adds an object to the end of the **Queue: Queue.Enqueue(queue name)(string)**
To add empty string use keyword empty.

Removes and returns the object at the beginning of the Queue:
Queue.Dequeue(queue name)
return **nan** if queue is empty

Returns the object at the beginning of the Queue without removing it:
Queue.Peek(queue name)
return **nan** if queue is empty

Removes all objects from the Queue: **Queue.Clear(queue name)**

Determines whether an element is in the Queue, returns "True" or "False":
Queue.Contains(queue name)(string)

Adds PPL array to the **Queue: Queue.AddArray(ppl_array) (queue name)**

Writes all elements from Queue to the PPL array created before with size = 0:

```
array  ppl_array;
```

Queue.ToArray(queue name) (ppl_array)

Delete all Queue objects: **Queue.Delete();**

Examples:
Note the difference between value as variable and value as string literal (also for **Stack** and **Dictionary**)

```
>import Queue
Imported [Queue]
>var x = 0;
>array r[] = {1,2,3};
>Queue.Create("Q")
>call Queue.Enqueue("Q",x)
>call Queue.Enqueue("Q","x")
>call Queue.Enqueue("Q",r)
>call Queue.Enqueue("Q","r")
>Queue.Write(Q)
===Q===
        0
        x
        Array 3
        r
```

It is possible to save in **Queue** (also in **Stack** and **Dictionary**) lines of code in ppl-format to be performed using **eval**:

```
>import Queue
Imported [Queue]
>Queue.Create("Q")
>var y = 1
>Queue.Enqueue("Q")("var (x[0]);Sum(x)(y)(2);d x;")
>Queue.Write("Q")
===Q===
        var (x[0]);Sum(x)(y)(2);d x;
>eval(Queue.Dequeue("Q"))
-----Variables  and arrays-----
-L0       x                              [3]
```

See samples of code with Queue methods in **examples\lib\Queue.scr**

Stack

Methods:

Create	**Peek**	
Count	**Clear**	
Write	**Contains**	
Push	**AddArray**	
Pop	**ToArray**	**Delete**

To get short help for every method in Stack.DLL:
>Stack.help[(method name)]

Creates Stack object: **Stack.Create(stack name)**
Examples:

```
>import Stack;
>stack.Create("st");
>var name = "NewStack";
>Stack.Create(name);
```

Returns the number of elements actually contained in Stack: **Stack.Count(stack name)**

Writes stack names or all elements from the specified stack to the standard output stream:
Stack.Write() or Stack.Write(stack name)

Inserts an object at the top of the stack: **Stack.Push(stack name)(string)**
To insert empty string use keyword **empty**.

Removes and returns the object at the top of the Stack:
Stack.Pop(stack name)
return **nan** if stack is empty

Returns the object at the top of the Stack without removing it:
Stack.Peek(stack name)
return **nan** if stack is empty

Removes all objects from the Stack: **Stack.Clear(stack name)**

Determines whether an element is in the Stack, returns **True** or **False**:
Stack.Contains(stack name)(string)

Adds PPL array to the Stack: **Stack.AddArray) (PPL array)(stack name)**

Writes all elements from Stack to the PPL array created before with size = 0:
array ppl_array;

Stack.ToArray(stack name) (ppl_array)

Delete all Stack objects: **Stack.Delete();**

Examples:

```
>dbg yes;
>import Stack
Imported [Stack]
>Stack.Create("s")
>Stack.Push("s")("one")
>Stack.Push("s")("two")
>Stack.Push("s")("three")
>Stack.Pop("s")
result = three
>Stack.Pop("s")
result = two
>Stack.Pop("s")
result = one
>Stack.Pop("s")
result = nan
```

See samples of code with Stack methods in **examples\lib\Stacks.scr**

Dictionary

This library uses functions from **std::unordered_map**.

Methods:

Create	**Clear**
Delete	**Write**
Count	**ContainsKey**
Add	**Remove**
GetValue	**AddArray**
SetValue	**ToArray**

To get short help for every method in Dictionary.DLL:
>**Dictionary.help[(method name)]**

Creates Dictionary object: **Dictionary.Create(dictionary name)**

Deletes all Dictionary objects: **Dictionary.Delete()**
or deletes one object from Dictionary: **Dictionary.Delete(dictionary_name)**

Example:

```
>import Dictionary;
>Dictionary.Create("dict1");
>var name = "dict2";
>Dictionary.Create(name);
array y[] = {"dict3",0,1};
>Dictionary.Create(y[0]);
>Dictionary.Delete("dict1");   // delete dict1
>Dictionary.Delete();          // delete dict2 and dict3
```

Returns number of dictionaries or number of elements in specified dictionary:
Dictionary.Count() | Dictionary.Count(dictionary name)

Adds the specified key and value to the Dictionary:
Dictionary.Add(dictionary name)(key)(value)

Example:

```
>import Dictionary;
>Dictionary.Create("dict");
>call Dictionary.Add("dict","one",1+1);
>call Dictionary.Add("dict","two",3);
>call Dictionary.Add("dict","three",4);
>call Dictionary.Count("dict");  // => 3
>call Dictionary.Count();        // => 1
>Dictionary.Write("dict");
```

Dictionary.GetValue
Get value from Dictionary by key
return **True** or **False**:
Dictionary.GetValue(dictionary name)(key)(var name for result)

Example:

```
>import Dictionary
>Dictionary.Create("dict")
>call Dictionary.Add("dict","one",1);
>call Dictionary.Add("dict","two",2);
>call Dictionary.Add("dict","three",3);
>array y[3]
>var x
>call Dictionary.GetValue("dict","one",x)
>write#("x = {0}",x)
x = 1
```

Dictionary.SetValue
set value in specified dictionary for specified key, if key exists
return **True or False:**
Dictionary.SetValue(dictionary name)(key)(value)

Dictionary.Write
Writes dictionary names only: **Dictionary.Write()**
or
all elements from the specified Dictionary to the standard output stream:
Dictionary.Write(dictionary name)

Removes all objects from the Dictionary: **Dictionary.Clear(dictionary name)**

Determines whether the Dictionary contains the specified key, returns **True** or **False**:
Dictionary.ContainsKey(dictionary name)(key)

Removes the value with the specified key from the Dictionary:
Dictionary.Remove(dictionary name)(value)
Adds PPL array to the Dictionary:
Dictionary.AddArray(PPL array) ("dictionary name")

```
>import Dictionary
Imported [Dictionary]
>Dictionary.Create("D")
>array y[3]
>y[0] = "one",1
>y[1] = "two",2
```

```
>y[2] = "three",3
call >Dictionary.AddArray (y,"D")
>Dictionary.Write("D")
Dictionary.Write [D]
        one     1
        two     2
        three   3
```

Write all elements from Dictionary to new PPL array created before with size = 0:

```
array ppl_array;
```

Dictionary.ToArray(dictionary name) (ppl_array)

Examples of code with Dictionary methods in **examples\lib\Dictionary.scr**

Convert

Methods:

StringToInt32Array
StringToHexArray
HexToBin
BinToHex
IntToHex
HexToInt
IntToBin
BinToInt

To get short help for every method in Convert.DLL:
>Convert.help[(method name)];

String characters converts to int32 array created before with size = 0 and resized in **Convert.StringToInt32Array** with size of string_chararacters:
Convert.StringToInt32Array(string_chararacters)(Int32 _ppl_array)

String characters converts to hex array created before with size = 0 and resized in **Convert.StringToHexArray** with size of string_chararacters:
Convert.StringToHexArray(string_chararacters)(Hex_ppl_array)

All below mentioned methods convert data in accordance with method name and return:
Returns string bin: **Convert.HexToBin(string with hex value)**
Returns string hex: **Convert.BinToHex(string with bin value)**
Returns string hex: **Convert.IntToHex(string with Int32 value)**
Returns string Int32: **Convert.HexToInt(string with hex value)**
Returns string bin: **Convert.IntToBin(string with Int32 value)**
Returns string Int32: **Convert.BinToInt(string with bin value)**

Examples:
See Examples\lib\Convert.scr

```
>array Int32;
>Convert.StringToInt32Array("12345")("Int32");
Info [CreateArrayFormat2] Global array [Int32] is created
>writearray Int32;
Result:
-----Array Int32-----
[0]     49
[1]     50
[2]     51
[3]     52
[4]     53
>array Hex;
>Convert.StringToHexArray("12345")("Hex");
```

```
>writearray Hex;
Result:
-----Array Hex-----
[0]     31
[1]     32
[2]     33
[3]     34
[4]     35
```

Examples:

```
>dbg yes
>Convert.HexToBin(16);
result = 10110
>Convert.BinToHex(1111111)
result = 7F
>Convert.IntToHex(256)
result = 100
```

File

Methods:

ReadAllText	**ReadAllLines**
WriteAllText	**WriteAllLines**
Exists	**Delete**

To get short help for every method in **File.DLL**:
>File.help[(method name)];

Returns all contents of text file:**File.ReadAllText(filename)**

Creates a new file, write the contents to the file, and then closes the file:
File.WriteAllText(var_ppl)(filename)

Determines whether the specified file exists, returns True or False: **File.Exists(filename)**

Returns string array with lines of text file:**File.ReadAllLines(filename)(ppl_array)**

Creates a new file, writes one or more strings to the file, and then closes the file:
File.WriteAllLines(ppl_array)(filename)

Deletes the specified file: **File.Delete(filename)**
Example:

```
>rc examples\lib\File.scr
```

```
var text;

//examples\lib\split.txt
//1,2,3,4,5,6,7,8,9,10,
//11,12,13,14,15,16,17,18,19,20

text = File.ReadAllText("examples\lib\split.txt");
write(text);
call File.WriteAllText(text,"examples\lib\Copy.txt");
var b = File.Exists("examples\lib\Copy.txt");
write(b);
if (b == True)
  File.Delete("examples\lib\Copy.txt");
b = File.Exists("examples\lib\Copy.txt");
write(b);

array arr[] = {"One","Two","Three"};
call File.WriteAllLines(arr,"examples\lib\Copy2.txt");
array arr2;
call File.ReadAllLines("examples\lib\copy2.txt",arr2);
writearray arr2;
```

```
File.Delete("examples\lib\copy2.txt");


Result:
1,2,3,4,5,6,7,8,9,10,
11,12,13,14,15,16,17,18,19,20
True
False
-----Array arr2-----
[0]     One
[1]     Two
[2]     Three
```

Random

The Random library use the Mersenne Twister pseudorandom number generator (PRNG).

Methods:

Create	**Delete**
Next	**Write**
NextToArray	

To get short help for every method in **Random.DLL**:
>Random.help[(method name)];

Creates Random object: **Random.Create(random_name)(type)(from)(to)**
type:= "int" | "real"

Returns a random integer: **Random.Next(random_name)**

Creates random numbers and writes them to the of a specified **ppl_array** created before with size = 0:
Random.NextToArray(random name)(ppl_array)(quantity of random elements)

Deletes created random_name or all random_names:
Random.Delete(random_name) | Random.Delete()

Writes random_names: **Random.Write()**

Example:
File examples\lib\random.scr

```
import Random;
call Random.Create("intR","int",1,10);
write(Random.Next("intR"));
write(Random.Next("intR"));
write(Random.Next("intR"));

array int_arr;
call Random.NextToArray("intR",int_arr,10);
writearray int_arr row;

call Random.Create("dblR","real",1,10);
write(Random.Next("dblR"));
write(Random.Next("dblR"));
write(Random.Next("dblR"));

array double_arr;
call Random.NextToArray("dblR",double_arr,10);
```

```
writearray double_arr row;

write("===Random.Delete===[dblR]");
Random.Delete("dblR");
Random.Write();

write("===Random.Delete===[all]");
Random.Delete();
Random.Write();
Result:
Imported [Random]
9
2
10
-----Array int_arr-----
9, 2, 10, 10, 3, 7, 4, 1, 6, 3
2.219293
8.515077
9.719810
-----Array double_arr-----
2.98931, 3.7735, 5.92499, 2.69544, 9.93593, 9.96815, 9.70925,
7.53255, 9.82999, 1.98876
===Random.Delete===[dblR]
0       intR    int
===Random.Delete===[all]
```

Console

For using in scr or ppl files, not for console input.

Methods:

ForegroundColor	**Beep**
BackgroundColor	**Clear**
ForegroundPromptColor	**SetCursorPosition**
DefaultColors	**GetCursorPosition**
Write	
WindowWidth	**WindowHeight**

To get short help for every method in **Console.DLL**:
>Console.help[(method name)];

Sets the foreground color of the console:
Console.ForegroundColor(color)
Sets the background color of the console:
Console.BackgroundColor(color)
Sets the prompt foreground color of the console**:**
Console.ForegroundPromptColor(color)
Sets the default foreground, background and ForegroundPromptColor color of the console:
Console.DefaultColors()

Writes the text representation of the specified value or values to the standard output stream: **Console.Write [(format)](string)(string)**
Plays the sound of a beep through the console speaker:
Console.Beep (frequency)(duration)
frequency - 37 to 32767 hertz
duration - msec
Clears the console buffer and corresponding console window of display information:
Console.Clear()
Sets the position of the cursor:
Console.SetCursorPosition(left column cursor position)(top row cursor position)
Gets the position of the cursor**:**
Console.GetCursorPosition ()
Returns '**left column cursor position, top row cursor position'**

Only for Administrator mode:
Run function **WindowSize() (see file Functions\CommonFunctions.ppl)**
>WindowSize()
width=120 height=30
Get/Set **WindowWidth(value) and WindowHeight(value)**

```
>Console.WindowHeight(20)          // set
>write(Console.WindowHeight())     // get
20
```

See examples in examples\Console.

Vector

Using Vector and Matrix libraries significantly increases program performance.
Methods:

Vector.Create ("vector_name")(length)(type)
type:= double|float|decimal|bool|int|uint|long|ulong|string
types in accordance with value types in
https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/built-in-types
Vector.Get(vector_name)(index)
Vector.Set(vector_name)(index)(value)
Vector.Add("vector_name")(ppl_array)

Vector.Write("vector_name")[("row"|"col"|"col0")]
Write to line - **"row"**
Write not nullable data to column - **"col"**
Write all data to column - **"col0"**
by default = **"row"**

Vector.WriteNames()
Vector.Delete("vector_name")
Vector.DeleteAll()
Vector.Sort ("vector_name")(order)
order:= "ascend"| "descend"

To get short help for every method in **Vector.DLL**:
>Vector.help[(method name)];

Examples:

```
>import Vector;
>array v[] = {1,2,3,4,5};
>call Vector.Create ("V",5, "int");
>call Vector.Add("V",v);
> call Vector.Write("V");
=====vector V=====
1       2       3       4       5
>call Vector.Set("V",0,0);
>call Vector.Write("V","col");   // write in column
=====vector V=====
[1]       2
[2]       3
[3]       4
[4]       5
>call Vector.Delete("V");
```

See examples **in examples\MatrixVector.**

Matrix

Methods:

Matrix.Create ("matrix_name")(rows)(columns)(type)
type:= double|float|decimal|bool|int|uint|long|ulong|string
types in accordance wth value types in
https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/built-in-types
Matrix.Get("matrix_name")(index row)(index_column)
Matrix.Set("matrix_name")(index row)(index_column)(value)
Matrix.AddArrayToRow ("matrix_name")(row)(""ppl_array"")
Matrix.AddArrayToColumn ("matrix_name")(column)(""ppl_array"")
Matrix.Write to line - "row"
Matrix.Write not nullable data to column - "col"
Write all data to column - "col0"
Write("matrix_name")[("row"|"col"|"col0")]
Matrix.WriteNames()
Matrix.Delete("matrix_name")
Matrix.DeleteAll()
Matrix.Rotate("matrix_name")("cw" | "ccw") - for square matrix only

Set column_index for sorting:
Matrix.Sort("matrix_name")(column index| order)
order:= ascend | descend

To get short help for every method in Matrix.DLL:
>**Matrix.help[(method name)]**

Example:

```
import Matrix;
call Matrix.Create("D",3,3,"double");
array r11[] = {1,2,3};
array r12[] = {4,5,6};
array r13[] = {7,8,9};

call Matrix.AddArrayToRow("D",0,"r11");
call Matrix.AddArrayToRow("D",1,"r12");
call Matrix.AddArrayToRow("D",2,"r13");


// Rotation CCW
call Matrix.Write("D");
call Matrix.Rotate("D","ccw");
write("===ccw===");
Matrix.Write("D");
```

```
// Rotation CW
call Matrix.Rotate("D","cw");
write("===cw===");
Matrix.Write("D");
Matrix.Delete("D");
```

Result:

```
=====matrix D=====
[0]     1       2       3
[1]     4       5       6
[2]     7       8       9
===ccw===
=====matrix D=====
[0]     3       6       9
[1]     2       5       8
[2]     1       4       7
===cw===
=====matrix D=====
[0]     1       2       3
[1]     4       5       6
[2]     7       8       9
```

```
>import Matrix;
>call Matrix.Create("D",3,3,"double");
>array r11[] = {1,2,3};
>array r12[] = {4,5,6};
>array r13[] = {7,8,9};
>call Matrix.AddArrayToColumn("D",0,r11);
>call Matrix.AddArrayToColumn("D",1,r12);
>call Matrix.AddArrayToColumn("D",2,r13);
>Matrix.Write("D");
>call Matrix.Sort("D",0,"descend");
>Matrix.Write("D");
>Matrix.DeleteAll();
code: scr
Imported [Matrix]
=====matrix D=====
[0]     1       4       7
[1]     2       5       8
[2]     3       6       9
=====matrix D=====
[0]     3       6       9
[1]     2       5       8
[2]     1       4       7
```

See examples in **examples\MatrixVector.**

DataFrame

DataFrame is a table with named columns, columns may be defined with different types,number of rows and columns is not limited.

DataFrame Architecture in ForestPPL (Under the Hood)

When you create or load a **DataFrame i**n ForestPPL, the engine builds a highly optimized Abstract Syntax Tree (AST) structure designed for both execution speed and memory efficiency. The engine uses a hybrid approach, separating global script variables from internal metadata.

Here is how a DataFrame is structurally organized inside the engine:

- **Root Node (DataFrame):** The parent object representing the table (e.g., `DF1`). This node is explicitly registered in the engine's global cache (`ScopeManager`), allowing the script to find the table instantly via an **O(1)** or highly optimized lookup.
- **Settings Node (Settings):** A private, internal branch directly attached to the root. It acts as a container for all table metadata, such as `RowsLength`, `PrintEmptyRows`, and specific column formats (`Type` and `Width`). To maximize performance and prevent polluting the global memory space, `Settings` and its children are **not** registered in the global cache. The core engine accesses them directly via rapid tree traversal.
- **Data Columns (Arrays):** The actual data vectors (e.g., `DF1.Name`, `DF1.Quantity`). These are stored as dedicated array nodes under the root. Because scripts often need direct and fast access to these columns, they are registered in the global cache alongside the root node.

Why this matters for performance:

By keeping internal settings encapsulated within the AST and only exposing the Root and Column arrays to the global **ScopeManager**, ForestPPL maintains a minimal memory footprint. This architecture guarantees that even with hundreds of loaded dataframes, variable lookups remain lightning-fast.

Methods:

SetRow	**AddRowData**
SetColumn	**GetRowArray**
Write	**GetSliceRowArray**
Save	**GetColumnArray**
ReadFile	**GetSliceColumnArray**
Init	**SetCell**
List	**GetCell**
Create	**CellName**
InsertRows	**SetWidth**
AddRows	**SetType**

InsertColumns
AddColumns
RemoveRows
RemoveColumns
ClearColumns
SetWidthForAll
SetTypeForAll
Sort
Reverse
SelectRows
UnselectRows
GetRowsLength
GetColumnLength
GetWidth
GetType
SetPrintEmptyRows
GetPrintEmptyRows
SetRowSelectedFrom
GetRowSelectedFrom
SetRowSelectedTo
GetRowSelectedTo
SetResizeStep
GetResizeStep
GetColumnNames
Delete
SetArrayToRow
SetArrayToColumn

To get short help for every method in **DataFrame.DLL**:
>**DataFrame.help[(method name)]**

DataFrame.Init – to add this call in first .scr file only, if project contains several .scr-files.
Format: **DataFrame.Init()**

DataFrame.Create - creates node df_name, node Settings and arrays per each column

Formats:
DataFrame.Create() - creates DataFrame "DF" with 26x26 columns with following names: A,B,C,… BA,BB,BC,…ZA,…ZZ and 1000 rows

DataFrame.Create(Name) - creates DataFrame "Name" with 26x26 columns with following names: A,B,C,… BA,BB,BC,…ZA,…ZZ
and 10 rows.
Name := <character literal> | <value of var | array_item>

Format:
DataFrame.Create(Name)(rows length) - creates DataFrame "Name" with 26 columns with following names: A,B,C,…Z and "rows length" rows

DataFrame.Create(Name)(rows length)(column1)(column2)…(columnN) - creates DataFrame "Name" with N columns and "rows length" rows

Examples:

```
>import DataFrame
> var name = "OK"
>DataFrame.Init()
>DataFrame.Create(name)(2)("A")("B")
```

```
>import DataFrame
>DataFrame.Init()
>DataFrame.Create("D")(2)("A")("B")
>d
-N2      NS
---N3    Global
-----L0 empty    (const)
-----N4 DF               [DataFrame]
-------N5        Settings
---------L0      RowSelectedFrom
---------L1      RowSelectedTo
---------L2      PrintEmptyRows   [yes]
---------L3      RowsLength       [2]
---------L4      ResizeStep         [10]
---------L5      AType    [Text]
---------L6      AWidth   [12]
---------L7      BType    [Text]
---------L8      BWidth   [12]
-------N5        A                [Array 2]
---------L0      #
---------L1      #
-------N5        B                [Array 2]
---------L0      #
---------L1      #
```

DataFrame.Settings.RowSelectedFrom and **DataFrame. Settings.RowSelectedTo** sets selected rows and used in **DataFrame.Read**, **DataFrame.Save**, **DataFrame.Write.**

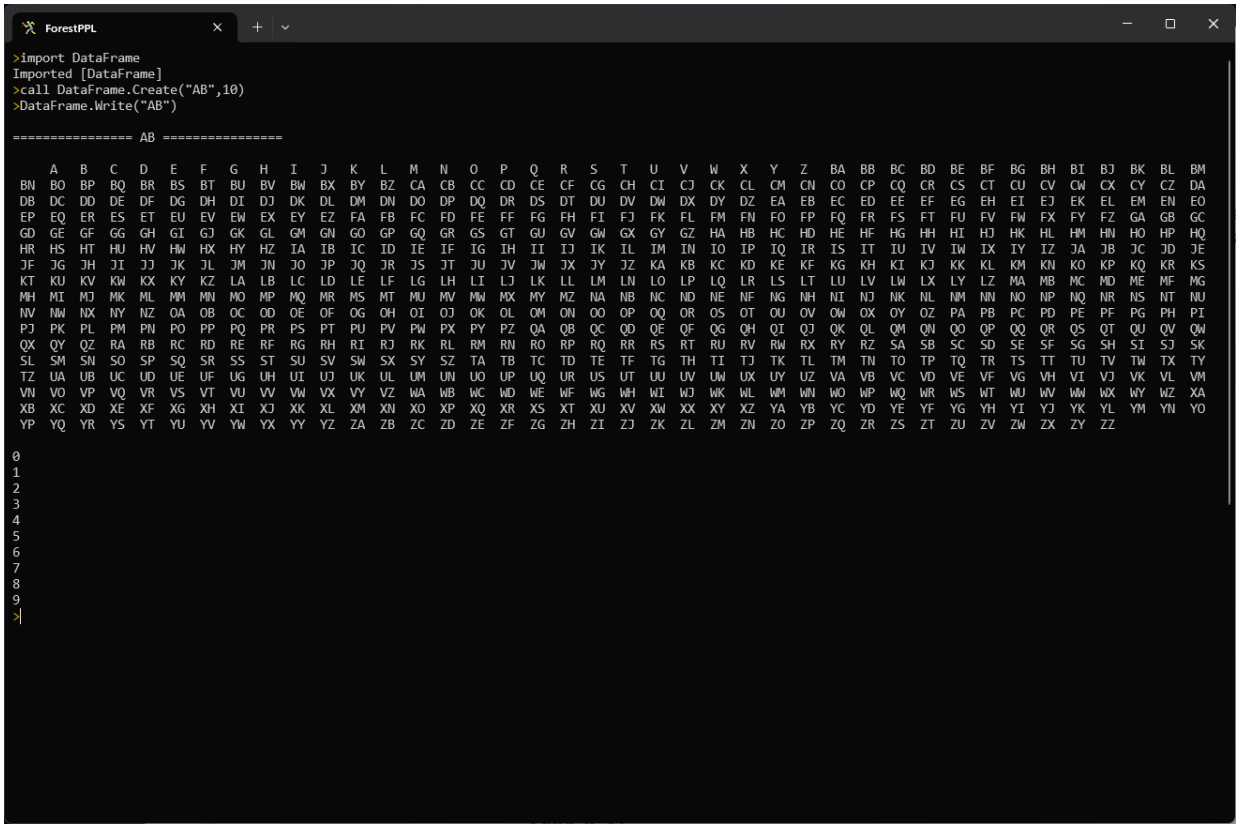
DataFrame.Settings.PrintEmptyRows sets to present or not empty rows and used in **DataFrame.Write**.
DataFrame. Settings.RowsLength is set automatically as number of rows.

DataFrame. Settings.ResizeStep sets number of allocated rows and used in **DataFrame Create** and **DataFrame AddRows**.

DataFrame. Settings.<column name>Type sets column type and used in **DataFrame**.**Read** and **DataFrame**.**Sort**.

DataFrame. Settings.<column name>Width sets column width and used in Write.

1. Example:



2. Example

```
>import DataFrame
>call DataFrame.Create("MyDataFrame",2," One","Two","Three")
-----Variables  and arrays-----
-N4      MyDataFrame      [Node]
---N5    Settings        [Node]
---N5    One               [Array 2]
---N5    Two               [Array 2]
---N5    Three             [Array 2]
>DataFrame.Write(MyDataFrame)
Result:
     One          Two          Three
0
1
```

DataFrame.SetRow and **DataFrame.SetColumn** return **false** if **columnType = Number** and value is not digital

DataFrame.SetRow - sets values for each column for defined row from array,variables or literal.

Format:
DataFrame.SetRow(df_name)(row index)(ppl_array)
DataFrame.SetRow(df_name)(row index)(literal1) | (var1) (literal2) | (var2) …

DataFrame.SetColumn - sets values for each element for defined column from array,variables or literal.

Format ppl:
DataFrame.SetColumn(df_name)(column) (ppl_array)
DataFrame.SetColumn(df_name)(column)(literal1) | (var1) (literal2) | (var2) …

Examples:

```
import DataFrame;
DataFrame.Init();
call DataFrame.Create
     ("DF",5,"City","Population","Area","Elevation");

write#("=====SetRow=====");
// Setting by data
call DataFrame.SetRow("DF",0,"Ljubljana",279631,163.8,295,0,0);
      // Warning: out of bounds 0,0
call DataFrame.Write("DF");

// Setting by array
array Kranj[] = {"Kranj",37941,148,386};
call DataFrame.SetRow("DF",1,Kranj);
//call DataFrame.Write("DF");
// Setting by vars
var x = "Kranj";
var y = 37941;
call DataFrame.SetRow("DF",1,x,y);
call DataFrame.Write("DF");
```

DataFrame.Write - displays DataFrame full contents or only contents of defined columns

Format:
DataFrame.Write[(df_name)] [("column")("column")…]

To display contents of specific rows call before **DataFrame.Write**:
DataFrame. RowSelectedFrom (df_name)(row index)
DataFrame. RowSelectedTo (df_name)(row index)
or by method SelectRows.
DataFrame.SelectRows(df_name)(row index from)(row index to)

DataFrame.Save -saves full contents of DataFrame in file format **".csv"** or **".data"** or defined columns only in format **".csv"**

Format:
DataFrame.Save(df_name)("filename.csv|.data")[(column)(column)]…
DataFrame.Save(df_name)(" filename.data")

DataFrame.ReadFile - reads full contents of DataFrame from file in format **".csv"** or **" .data"** or contents of defined columns from file in format **".csv" only**

Format:
(1) DataFrame.ReadFile(df_name) (filename.csv)[(column)(column)]…
or
(2) DataFrame.ReadFile(filename.data)
Format (2) deletes existing data and creates new Dataframe or several DataFrames with names from file.data

Example:

```
File df.data
(DF [DataFrame]
 (Settings
    (RowSelectedFrom)
    (RowSelectedTo)
    (PrintEmptyRows [yes])
    (RowsLength [3])
    (ResizeIncrement [10])
    (NameType [Text])
    (NameWidth [12])
    (QuantityType [Text])
    (QuantityWidth [12])
   )
 (Name [Array 3]
   (# ["CARS"])
   (# ["TREES"])
   (# ["ROADS"])
 )
 (Quantity [Array 3]
   (# [20])
   (# [30])
   (# [40])
 )
)
>import DataFrame;
>call DataFrame.ReadFile("examples\DataFrame\df.data");
>call DataFrame.Write("DF");
```

Result:

```
     Name         Quantity

0    "CARS"       20
1    "TREES"      30
2    "ROADS"      40
```

DataFrame.InsertRows - inserts number of rows before index

Format:
DataFrame.InsertRows(df_name)(index)[(number of rows)]
By default number of added rows= 1

DataFrame.AddRows - adds number of empty rows at the end

Format:
DataFrame.AddRows([df_name])[(number of rows)]
By default df_name = DF
By default number of added rows: **DataFrame.Settingd.ResizeatedIncrement = 10**

DataFrame.InsertColumns - insert one or several named columns before defined column

Format:
DataFrame.InsertColumns(df_name))(defined column1)(column2)(column3)…

DataFrame AddColumns - adds number of named columns at the end

Format:
DataFrame.AddColumns(df_name)(column1)(column2)…

DataFrame RemoveRows – removes rows

Format:
DataFrame.RemoveRows(df_name) – removes all rows
DataFrame.RemoveRows(df_name)[(number_of_row) – removes 1 row
DataFrame.RemoveRows(df_name)(number_from)(*) - removes rows between number_from to the end of rows
DataFrame.RemoveRows(df_name)(number_from)(number_to) - removes rows between number_from – number_to

DataFrame.RemoveColumns - removes named columns and their Type and Width in **Settings**

Format: **DataFrame.RemoveColumns(df_name)[(column1)(column2)…]**

DataFrame.RemoveColumns(df_name) - removes all columns and their Type and Width in **Settings**:

DataFrame.ClearColumns - clears contents of all DataFrame or defined columns:
Format: **DataFrame.ClearColumns(df_name)[(column1)(column2)…]**

DataFrame.SetWidthForAll - sets same width for all columns to display DataFrame by **DataFrame.Write**

Format: **DataFrame.SetWidthForAll [(df_name)] (width)**
default name – DF
default with = 12

Example:

```
>DataFrame.SetWidthForAll(14)
```

DataFrame.SetTypeForAll - sets same type for all columns

Format: **DataFrame.SetTypeForAll [(df_name)] [("Text" | "Number")]**
default name – DF
default type = Text

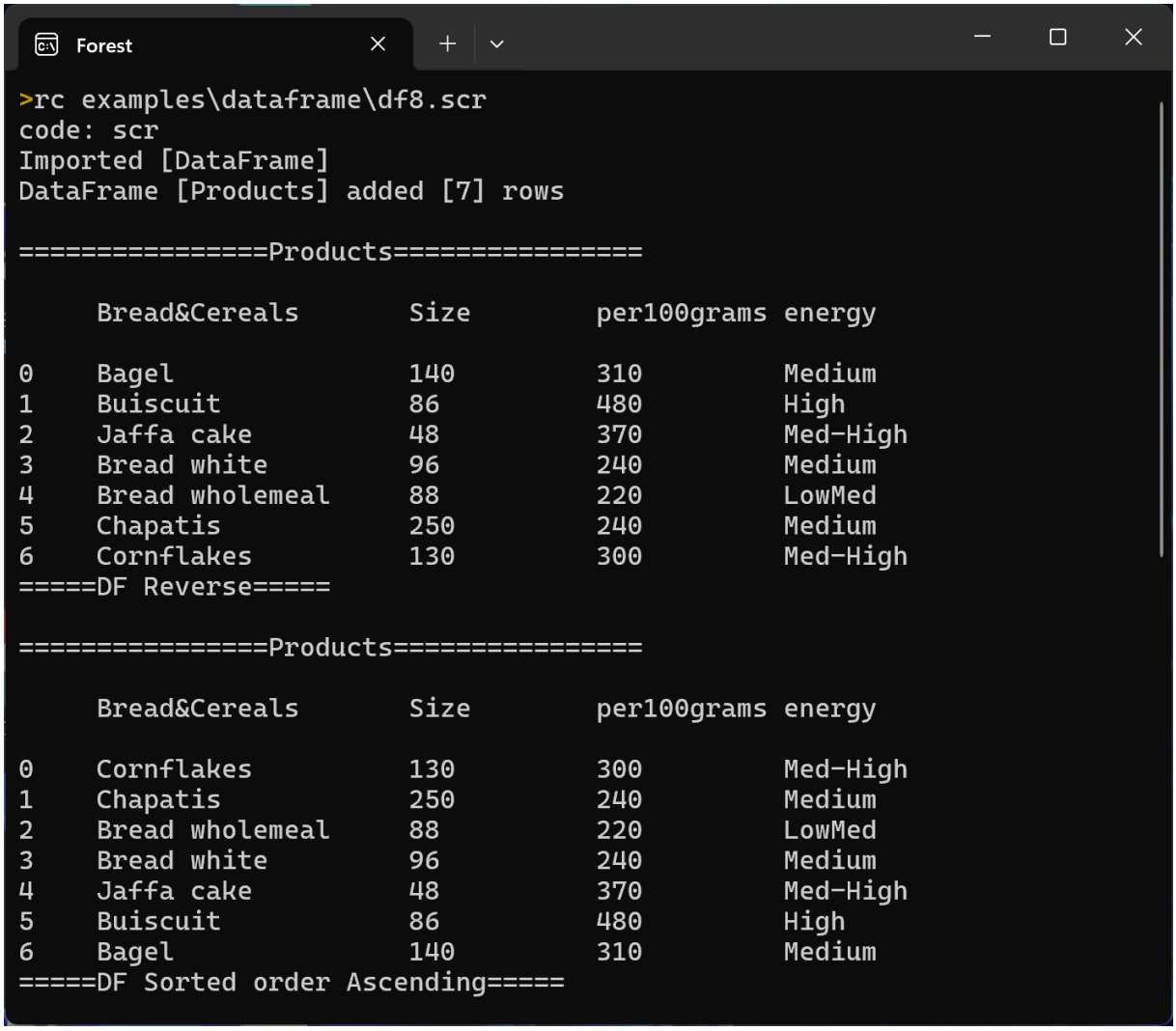
```
Example:
>DataFrame.SetTypeForAll("Number")
```

DataFrame.Sort - sorts all contents by specified column in ascending or descending order
Format: **DataFrame.Sort(df_name)("ascend"| "descend")(column)**

Example:

```
File: examples\DataFrame\products.csv
Bagel,140,310,Medium
Buiscuit ,86,480,High
Jaffa cake,48,370,Med-High
Bread white,96,240,Medium
Bread wholemeal,88,220,LowMed
Chapatis,250,240,Medium
Cornflakes,130,300,Med-High

Program: examples\DataFrame\df8.scr
import DataFrame;
DataFrame.Create(Products)(0)(Bread&Cereals)(Size)(per100grams)
                           (energy);
set Products.Settings.Bread&CerealsWidth = 20;
DataFrame.ReadFile(Products)(examples\DataFrame\products.csv);
DataFrame.Write(Products);
DataFrame.Sort(Products) ("ascend")(per100grams);
DataFrame.Write(Products);
```



See examples df8.scr and df9.scr(wrong results for digital data when Type = Text).

DataFrame.Reverse - reverses all DataFrame contents by columns

Format: **DataFrame.Reverse(df_name)**

DataFrame.SelectRows

Format:
Select one row only: **DataFrame.SelectRows(df_name)(select_from)**
Select rows select_from – to end: **DataFrame.SelectRows(df_name)(select_from)(*)**
Select rows between select_from – select_to:
DataFrame.SelectRows(df_name)(select_from)(select_to)

DataFrame.UnSelectRows

Format: **DataFrame.UnSelectRows(df_name)**

DataFrame.GetRowsLength – returns number of rows

Format: **DataFrame.GetRowLength(df_name)**

DataFrame.GetColumnsLength – returns number of columns

Format: **DataFrame. GetColumnsLength (df_name)**

DataFrame.AddRowData – adds row with data per columns at the end

Format: **DataFrame.AddRowData(df_name)(ppl_array)**

DataFrame.GetRowArray – copies data from defined row to ppl array

Format:
DataFrame.GetRowArray(df_name)(index row) (ppl array)

DataFrame.GetSliceRowArray - copies dataframe data, defined by row,column_from [and column_to] to ppl array

Format:
To the end of rows:
DataFrame. GetSliceRowArray (df_name)(row)(column_from)(ppl array)
or
DataFrame. GetSliceRowArray (df_name)(row)(column_from)(column_to)(ppl array)

DataFrame.GetColumnArray - copies data from defined column to array

Format:
DataFrame.GetColumnArray(df_name)(column) (ppl array)

DataFrame.GetSliceColumnArray – copies dataframe data, defined by column, row_from [and row_to] to array

Format:
To the end of columns:
DataFrame. GetSliceColumnArray (df_name)(column)(row_from)(ppl array)
or
DataFrame. GetSliceColumnArray (df_name)(column)(row_from)(row_to) (ppl array)

Example:

```
File: Examples\DataFrame\df12.scr
recreate yes;
import DataFrame;
DataFrame.Init();
call DataFrame.Create("Table",0,"A","B","C","D","E");
call DataFrame.SetType("Table","A", "Number");
call DataFrame.ReadFile("Table","examples\dataframe\Table.csv");
DataFrame.Write("Table");

write("=====DataFrame.GetColumnArray=====");
array arr_column_A;
call DataFrame.GetColumnArray("Table","A",arr_column_A);
writearray arr_column_A row;

write("=====DataFrame.GetRowArray=====");
array arr_row_0;
call DataFrame.GetRowArray("Table",0,arr_row_0);
writearray arr_row_0 row;

write("=====DataFrame.GetSliceRowArray=====");
array arr_A2;
call DataFrame.GetSliceRowArray("Table",0,"A",arr_A2);

writearray arr_A2 row;

array arr_A3;
call DataFrame.GetSliceRowArray("Table",1,"B","D",arr_A3);
writearray arr_A3 row;

write("=====DataFrame.GetSliceColumnArray=====");
array arr_A4;
call DataFrame.GetSliceColumnArray("Table","B",1,4,arr_A4);
writearray arr_A4 row;
```

```
array arr_A5;
call DataFrame.GetSliceColumnArray("Table","B",0,arr_A5);
writearray arr_A5 row;

del all;
printscope;
>rc examples\dataframe\df12.scr
Result:
code: scr
Imported [DataFrame]

================ Table ================

     A           B           C           D           E

0    1           2           3           4           5
1    10          20          30          40          50
2    100         200         300         400         500
3    1000        2000        3000        4000        5000
4    10000       20000       30000       40000       50000
=====DataFrame.GetColumnArray=====
-----Array [arr_column_A]-----
1, 10, 100, 1000, 10000
=====DataFrame.GetRowArray=====
-----Array [arr_row_0]-----
1, 2, 3, 4, 5
=====DataFrame.GetSliceRowArray=====
-----Array [arr_A2]-----
1, 2, 3, 4, 5
-----Array [arr_A3]-----
20, 30, 40
=====DataFrame.GetSliceColumnArray=====
-----Array [arr_A4]-----
20, 200, 2000, 20000
-----Array [arr_A5]-----
2, 20, 200, 2000, 20000

=========== State of SCOPE MANAGER ===========
Depth [0]
 (Global)
 <-- TOP (Current of Context)
    <no variables>
```

DataFrame. SetCell - sets value for cell ,defined by name or by column name and row index

Format:
DataFrame.SetCell(df_name)(column name)(index row)(value)
or
DataFrame.SetCell(df_name.cell_name)(value)

DataFrame.GetCell - returns value of cell, defined by name or by column name and row index

Format:
DataFrame.GetCell(df_name)(column name)(index row)
or
DataFrame.GetCell(df_name.cell_name)

DataFrame.CellName - sets name for defined cell

Format: **DataFrame.CellName(df_name)(column name)(index row)(cell name)**

DataFrame.SetWidth - sets Width value for defined column name in **Settings**

Format: **DataFrame. SetWidth (df_name)(column name)(value)**

DataFrame.SetType - sets Type value for defined column name in **Settings**

Format: **DataFrame. SetType (df_name)(column name)("Text "| "Number")**

DataFrame.GetWidth - returns Width for defined column name in **Settings**

Format: **DataFrame. GetWidth (df_name)(column name)**
Example:

```
>import DataFrame
>DataFrame.Create("DF")(2)( "A")("B")
>DataFrame.SetWidth("DF")("A")(15)
>write(DataFrame.GetWidth("DF")("A"))
15
```

DataFrame.GetType - returns Type for defined column name in **Settings**

Format: **DataFrame. GetType(df_name)(column name)**
Example:

```
>import DataFrame
>DataFrame.SetType("DF")("A")("Number")
>write(DataFrame.GetType("DF")("A"))
Number
```

DataFrame . SetPrintEmptyRows - sets PrintEmptyRows value in **Settings**

Format: **DataFrame.SetPrintEmptyRows (df_name)(value)**
value:= "yes"| "no"

DataFrame.GetPrintEmptyRows - returns PrintEmptyRows value in **Setting**

Format: **DataFrame.GetPrintEmptyRows (df_name)**
Example:

```
>import DataFrame;
>DataFrame.Create(DF)(2)(A)(B);
>DataFrame.SetPrintEmptyRows(DF)("yes");
>write(DataFrame.GetPrintEmptyRows(DF));
yes
```

DataFrame.SetRowSelectedFrom - sets RowSelectedFrom value in **Settings**

Format: **DataFrame. SetRowSelectedFrom (df_name)(index_row_from)**

DataFrame.GetRowSelectedFrom - returns RowSelectedFrom value in **Setting**

Format: **DataFrame. GetRowSelectedFrom (df_name)**

DataFrame.SetRowSelectedTo - sets RowSelectedTo value in **Settings**

Format: **DataFrame. SetRowSelectedTo (df_name)(index_row_to)**

DataFrame.GetRowSelectedTo – returns RowSelectedTo value in **Settings**

Format: **DataFrame. GetRowSelectedTo (df_name)**

Example:

```
>code scr;
>import DataFrame;
>call DataFrame.Create("DF",2, "A","B");
```

```
>call DataFrame.SetRowSelectedFrom("DF",0);
>call DataFrame.SetRowSelectedTo("DF",1);
>write(DataFrame.GetRowSelectedFrom("DF"));
result: 0
>write(DataFrame.GetRowSelectedTo("DF"));
1
```

DataFrame.SetResizeStep - sets ResizeStep value in **Settings**

Format: **DataFrame.SetResizeStep(df_name)(value)**

DataFrame.GetResizeStep - returns ResizeStep value in **Settings**

Format: **DataFrame.GetResizeStep(df_name)**
Example:

```
>import DataFrame;
>DataFrame.Create("DF")(2)("A")("B");
>DataFrame.SetResizeStep("DF")(100);
>write(DataFrame.GetResizeStep("DF"));
100
```

DataFrame.GetColumnNames – returns string with column names, separated by comma

Format:
DataFrame.GetColumnNames(df_name)
Example:

```
>import DataFrame;
>DataFrame.Init();
>call DataFrame.Create("Table",5,"A","B","C","D","E");
>write(DataFrame.GetColumnNames("Table"));
A,B,C,D,E
```

DataFrame.List – displays all created DataFrame names, their settings and cell names

Format: **DataFrame.List()**
Example:

```
>import DataFrame;
>DataFrame.Init();
>call DataFrame.Create("Table1",10, "A","B","C","D","E");
>call DataFrame.CellName("Table1.A",0, "A0");
```

```
>call DataFrame.CellName("Table1","B",0,"B0");
>call DataFrame.SetWidthForAll("Table1",10);
>call DataFrame.List();
=====Table1=====
  Settings
        RowSelectedFrom =
        RowSelectedTo =
        PrintEmptyRows = yes
        RowsLength = 10
        ResizeIncrement = 10
        AType = Text
        AWidth = 10
        BType = Text
        BWidth = 10
        CType = Text
        CWidth = 10
        DType = Text
        DWidth = 10
        EType = Text
        EWidth = 10
  defined cell names in DataFrame [Table1]
        name             column    row
        A0              A        0
        B0              B        0
```

DataFrame.Delete – delete all from DataFrame

Format **ppl**:
DataFrame.Delete()
DataFrame.Delete(name)
DataFrame.Delete(name1)(name2)…

Format **scr**:
DataFrame.Delete() or call DataFrame.Delete()
DataFrame.Delete(name) or call DataFrame.Delete(name)
DataFrame.Delete(name1)(name2)… or
call DataFrame.Delete(name1,name2,…

Important Note:

```
del all;              // delete  data from DataFrame, all arrays
                      // and variables out of DataFrame
DataFrame.Delete();   // delete  data from DataFrame only
```

SetArrayToRow – write data to row of df_name from array only
unlike SetRow, that works with vars, array and raw data.

Format **ppl:**
DataFrame.SetArrayToRow (df_name)(row)(array_name)

Format **scr:**
Call DataFrame.SetArrayToRow (df_name,row,array_name)

SetArrayToColumn - write data to column of df_name from array only
Unlike SetColumn, that works with vars, array and raw data.

Format **ppl:**
DataFrame.SetArrayToColumn (df_name)(column)(array_name)

Format **scr:**
Call DataFrame.SetArrayToColumn (df_name,column,array_name)

Both functions trim excess elements and display a Warning.
Examples:

```
>rc examples\dataframe\df27.scr
recreate yes;
import DataFrame;
DataFrame.Init();

write("======DataFrame.SetArrayToRow======");
call DataFrame.Create("DF",2, "Language","Complexity");
array arr[] = {"Assembler","High"};
call DataFrame.SetArrayToRow("DF",0,arr);
array arr[] = {"C","Medium","Professional"};
call DataFrame.SetArrayToRow("DF",1,arr);      // warning
DataFrame.Write("DF");
//array arr[] = {"C++","Medium","Professional"};
//call DataFrame.SetArrayToRow("DF",2,arr);      // error

call DataFrame.ClearColumns("DF");
call DataFrame.Write("DF");

write("======DataFrame.SetArrayToColumn======");
array arr[] = {"C","C++"};
call DataFrame.SetArrayToColumn("DF","Language",arr);

array arr[] = {"C","C++","ForestPPL"};
call DataFrame.SetArrayToColumn("DF","Language",arr);   // warning

array arr[] = {"High","Medium"};
call DataFrame.SetArrayToColumn("DF","Complexity",arr);
//array arr[] = {"Asembler","C","C++"};
//call DataFrame.SetArrayToColumn("DF","Creator",arr);     // error
```

```
call DataFrame.Write("DF");
======DataFrame.SetArrayToRow======
Warning: [DataFrame.FuncSetArrayToRow] DataFrame [DF] array size (3)
does not match columns count (2).
Data will be truncated or partially filled.

================ DF ================
     Language    Complexity

0    Assembler   High
1    C           Medium

================ DF ================
     Language    Complexity

0
1
======DataFrame.SetArrayToColumn======
Warning: [DataFrame.FuncSetArrayToColumn] DataFrame [DF] array size
(3) does not match rows count (2).
Data will be truncated or partially filled.

================ DF ================
     Language    Complexity

0    C           High
1    C++         Medium
```

Examples of using DataFrame methods in directory: examples\DataFrame
df1.scr – df27.scr,AddressBook2.scr, AddressBook.scr

StorageLib

StorageLib manipulates collections created with the storage command. A storage is either a plain list (1-D) or an N-dimensional array of any depth (for example s[2][3][2], loaded from a CSV file with a _shape_ header). Every function can work on the whole storage, on a sub-array, or on a single row, depending on how many index parentheses you write after the storage name.

Storage names are written without quotes. The only exception is the string form "s[1][2]", which names a sub-array in one argument (see **Indices and levels**).

General call form:

```
storage.Func(name)[(index)...](own parameters)
```

Read **Indices and levels** first: it explains how the interpreter decides which parentheses are indices and which are the function's own parameters, with worked examples.

Methods:

Push	**Copy**	**IndexOf**
PushRow	**Filtercopy**	**Contains**
Pop	**Min**	**CountOf**
Reverse	**Max**	**Join**
Shift	**Average**	**Split**
Remove	**First**	**Unique**
Clear	**Last**	**Shuffle**
Unshift	**Concat**	**Product**
Insert	**Sort**	**Median**
Slice	**Save**	
Sum	**ReadFile**	

To get short help for every method in StorageLib.DLL:
>storage.help[(method name)];

Indices and levels

The call form

```
storage.Func(name)[(index)...](own parameters)
```

Every function has a fixed number of **own parameters** (value, position, target variable, ...). The interpreter counts them **from the right**. Everything between the storage name and those own parameters is an **index**. So the same function name can address the whole storage, a slice, or a row, only by the number of parentheses.

0 own parameters: PushRow, Reverse, Clear, Sort, Unique, Shuffle

```
storage.Sort(s)(0)(1)
```

1 own parameter: Push, Unshift, Pop, Shift, Remove, First, Last, Sum, Min, Max, Average, Product, Median, Copy, Filtercopy

```
storage.Sum(s)(1)(2)(res)
```

2 own parameters: Insert (position, value), IndexOf / Contains / CountOf (value, target), Join (delimiter, target)

```
storage.IndexOf(s)(0)(1)(4)(res)
```

3 own parameters: Slice (start, end, target_storage)

```
storage.Slice(s)(1)(0)(2)(q)
```

Data used in the examples

File examples/StorageLib/test_nd.csv describes a 3-D array 2 x 3 x 2 holding the numbers 1..12:

```
_shape_,2,3,2
1,2,3,4,5,6,7,8,9,10,11,12
```

The same data as nested rows (the last dimension is the **row**: the list of plain values):

```
s[0] = [[1, 2],      s[1] = [[7, 8],
        [3, 4],              [9, 10],
        [5, 6]]              [11, 12]]
```

```
import StorageLib;
var res;
storage s;
storage.ReadFile(s)("examples/StorageLib/test_nd.csv")(",");
```

Worked example: why is the sum 23?

```
storage.Sum("s[1][2]")(res);    // res = 23
storage.Sum(s)(1)(2)(res);      // res = 23 (identical)
storage.Sum(s)(1)(res);         // res = 57
storage.Sum(s)(res);            // res = 78
```

"s[1][2]" does not mean "the element number 2 of s[1]". It names the **row** s[1][2]: first index 1 selects slice s[1], second index 2 selects the third row of that slice, i.e. [11, 12]. Sum adds **all values below the addressed node**, so the result is 11 + 12 = **23**, not 12 and not 11.
The same call written as storage.Sum(s)(1)(2)(res) is parsed like this: Sum has one own parameter (the target variable), so (res) is the target; the parentheses (1)(2) between s and (res) are indices. Dropping one index moves up one level: s[1] is the whole second slice (7+8+9+10+11+12 = 57), and no index at all means the whole storage (78).
Remember that a statistic never returns a single element. To read one value use First / Last on a row: storage.First(s)(1)(2)(res) gives 11, storage.Last(s)(1)(2)(res) gives 12.

Variables as indices

An index may be a number or a variable. Variables are evaluated before the call:

```
var i = 1;
storage.Sum(s)(i)(res);         // same as s[1] -> 57
```

String form "s[1][2]"

The quoted form is equivalent to writing the indices in parentheses. It is the only way to pass a sub-array where a function expects a storage name and there is no room for indices, e.g. both sources of Concat:

```
storage c2;
storage.Concat("s[0]")("s[1]")(c2);   // 6 rows, slice 0 then 1
storage.Sum(c2)(res);                 // 78
```

Slice and indices together

Slice has three own parameters, so in storage.Slice(s)(1)(0)(2)(q) the indices are only (1); (0)(2)(q) are start, end and target. The result is rows 0 and 1 of s[1] (end is exclusive):

```
storage q;
storage.Slice(s)(1)(0)(2)(q);   // q = [[7, 8], [9, 10]]
storage.Sum(q)(res);            // 34
```

Rules and errors

Which level does each function work on?

Row level only: Push, Unshift, Insert, Pop, Shift, First, Last, IndexOf, Sort, Unique. The indices must lead to a row (the last dimension, plain values). On an upper level the call fails: "target holds sub-arrays: add indices down to the last dimension".

Any level (acts on the children): Remove, Clear, Reverse, Shuffle, PushRow. Works on the addressed node itself: Remove(s)(1)(0) removes the whole row 0 of s[1]; Reverse(s) reverses the order of the two slices; Clear(s)(0) empties slice 0.

Whole subtree (all values): Sum, Min, Max, Average, Product, Median, Contains, CountOf, Join. Takes every value below the addressed node in row-major order, however deep. Non-numeric values are ignored by the statistics.

Copies: Copy, Slice, Filtercopy, Concat. Produce a new storage. Copy/Slice/Concat keep the shape (deep copy); Filtercopy always produces a flat 1-D list. Both Concat sources must be on the same level.

Numbers are parsed strictly

Values are kept as text and converted only when a function needs a number. The conversion is strict: the **whole** string must be a number (sign, decimal point and exponent are allowed: -3, +2.5, 1e3). "12abc", "" and "abc" are not numbers. Sum, Min, Max, Average, Product and Median **skip** such values; if no value is numeric the result is 0.

```
storage t;
storage.Split("5;12abc;7;;3")(";")(t);
storage.Sum(t)(res);       // 15  (5 + 7 + 3; "12abc" and "" are skipped)
storage.Average(t)(res);   // 5   (average of the three numeric values)
```

Results are written without trailing zeros: 6.5, 189, 6.333333.

Sort and Unique

Sort orders **numerically** when every value of the row is a number; if at least one value is not a number the whole row is sorted as **text** (so 10 comes before 9). The sort is stable. Unique removes duplicates by exact text comparison, keeping the first occurrence: 1 and 1.0 are different values.

```
storage a;
storage.Split("10;9;100;2")(";")(a);
storage.Sort(a);                // 2 9 10 100   (all numeric ->
                                   numeric order)
storage m;
storage.Split("10;9;abc;2")(";")(m);
storage.Sort(m);                // 10 2 9 abc   (mixed -> text
                                   order)
```

Errors

A failed StorageLib call prints a message Error: [storage.Func] ... and stops the running script. Typical messages:

```
Error: [storage.Sum] storage [x] not found.
Error: [storage.Push] target holds sub-arrays: add indices down
        to the last dimension or use storage.PushRow
Error: [storage.PushRow] target is the last dimension, use
        storage.Push
Error: [storage.Insert] position out of bounds.
Error: [storage.Slice] invalid start or end index.
Error: [storage.Pop] storage [s] is empty.
Error: [storage.Sum] target variable [r] not found.
Error: [storage.Concat] sources are on different levels
```

Add var declarations for target variables before the call, and check indices against the shape (_shape_ header of the CSV).

Methods

The examples below use the 3-D array s (values 1..12) loaded in Indices and levels, the 1-D storage a, and the variables res, txt and d declared with the following lines. Every example starts from a freshly loaded s and a freshly filled a; the results are printed with write.

```
import StorageLib;
var res;
var txt;
var d = ", ";
storage s;
storage a;
storage.ReadFile(s)("examples/StorageLib/test_nd.csv")(",");
```

Shows short help for every method of StorageLib or for one method.

storage.help[(method name)]

Example:

```
>import StorageLib
>storage.help(Clear)
```

```
Imported [StorageLib]
storage.Clear(storage_name)[(index)...] - removes all elements.
```

Adds a value to the end of a **row**. For a 1-D storage no index is needed; for an N-D storage the indices must lead to the last dimension. To add a whole row use PushRow.

storage.Push(storage_name)[(index)...](value)

Example:

```
storage b;
storage.Push(b)(5);
storage.Push(b)(3);
storage.Join(b)(d)(txt);
write(txt);
5, 3
storage.Push(s)(0)(1)(99);          // row s[0][1]
storage.Join(s)(0)(1)(d)(txt);
write(txt);
3, 4, 99
storage.Push(s)(5);
Error: [storage.Push] target holds sub-arrays: add indices down
 to the last dimension or use storage.PushRow
```

Adds a new sub-array at the end of the addressed level. The new sub-array has the same shape as the previous last sibling and is filled with zeros. Rows may then be of different lengths (ragged arrays).

storage.PushRow(storage_name)[(index)...]

Example:

```
storage.PushRow(s)(0);              // slice 0 gets a 4th row
storage.Sum(s)(0)(res);
write(res);
storage.Join(s)(0)(3)(d)(txt);
write(txt);
21
0, 0
storage.PushRow(s)(0)(1);
Error: [storage.PushRow] target is the last dimension, use
 storage.Push
```

Removes the last value of a **row** and stores it in target_var.

storage.Pop(storage_name)[(index)...](target_var)

Example:

```
storage.Split("9;3;7")(";")(a);
storage.Pop(a)(res);
write(res);
storage.Join(a)(d)(txt);
write(txt);
7
9, 3
storage.Pop(s)(1)(2)(res);          // row s[1][2] = 11 12
write(res);
```

```
12
```

Reverses the order of the children of the addressed node (values of a row, rows of a slice or slices of the whole array).

storage.Reverse(storage_name)[(index)...]

Example:

```
storage.Split("9;3;7")(";")(a);
storage.Reverse(a);
storage.Join(a)(d)(txt);
write(txt);
7, 3, 9
storage.Reverse(s)(0)(1);          // row s[0][1] = 3 4
storage.Join(s)(0)(1)(d)(txt);
write(txt);
4, 3
storage.Reverse(s);                // swaps the two slices
storage.Sum(s)(0)(res);
write(res);
57
```

Removes the first value of a **row** and stores it in target_var.

storage.Shift(storage_name)[(index)...](target_var)

Example:

```
storage.Split("9;3;7")(";")(a);
storage.Shift(a)(res);
write(res);
storage.Join(a)(d)(txt);
write(txt);
9
3, 7
storage.Shift(s)(0)(0)(res);
write(res);
storage.Join(s)(0)(0)(d)(txt);
write(txt);
1
2
```

Removes the child at position of the addressed node. At row level it removes one value, at an upper level a whole sub-array.

storage.Remove(storage_name)[(index)...](position)

Example:

```
storage.Split("9;3;7")(";")(a);
storage.Remove(a)(0);               // first element
storage.Join(a)(d)(txt);
write(txt);
3, 7
storage.Remove(s)(1)(0);            // row 0 of slice 1 (7 8)
storage.Sum(s)(1)(res);
write(res);
```

```
42
storage.Remove(s)(0)(9);
Error: [storage.Remove] position out of bounds.
```

Removes all children of the addressed node.
storage.Clear(storage_name)[(index)...]
Example:

```
storage.Clear(s)(0);                // slice 0 becomes empty
storage.Sum(s)(res);
write(res);
57
```

Adds a value to the beginning of a **row**.
storage.Unshift(storage_name)[(index)...](value)
Example:

```
storage.Split("9;3;7")(";")(a);
storage.Unshift(a)(1);
storage.Join(a)(d)(txt);
write(txt);
1, 9, 3, 7
storage.Unshift(s)(0)(0)(0);        // row s[0][0] = 1 2
storage.Join(s)(0)(0)(d)(txt);
write(txt);
0, 1, 2
```

Inserts a value at position of a **row**. position may equal the row length (append).
storage.Insert(storage_name)[(index)...](position)(value)
Example:

```
storage.Split("9;3;7")(";")(a);
storage.Insert(a)(1)(5);
storage.Join(a)(d)(txt);
write(txt);
9, 5, 3, 7
storage.Insert(s)(1)(2)(1)(0);      // row s[1][2] = 11 12
storage.Join(s)(1)(2)(d)(txt);
write(txt);
11, 0, 12
```

Copies the children from start up to (not including) end of the addressed node into target_storage. end larger than the length is clamped; a negative start becomes 0. Shape below the cut is preserved.
storage.Slice(storage_name)[(index)...](start)(end)(target_storage)
Example:

```
storage.Split("9;3;7")(";")(a);
storage b;
storage.Slice(a)(0)(2)(b);
storage.Join(b)(d)(txt);
write(txt);
```

```
9, 3
storage q;
storage.Slice(s)(1)(0)(2)(q);   // rows 0 and 1 of slice 1
storage.Sum(q)(res);
write(res);
storage.Join(q)(d)(txt);
write(txt);
34
7, 8, 9, 10
```

Sum of all numeric values below the addressed node. Non-numeric values are skipped; an empty result is 0. See **Indices and levels** for why "s[1][2]" gives 23.

storage.Sum(storage_name)[(index)...](target_var)

Example:

```
storage.Sum(s)(res);          write(res);
storage.Sum(s)(1)(res);       write(res);
storage.Sum(s)(0)(1)(res);    write(res);
storage.Sum("s[1][2]")(res);   write(res);
78
57
7
23
```

Deep copy of the addressed (sub)array into a new storage, shape preserved.

storage.Copy(source)[(index)...](destination)

Example:

```
storage s0;
storage.Copy(s)(s0);               // full backup
storage.Sum(s0)(res);
write(res);
storage row;
storage.Copy(s)(0)(1)(row);         // row s[0][1]
storage.Join(row)(d)(txt);
write(txt);
78
3, 4
```

Copies every value below the addressed node into a **flat** 1-D storage, leaving out the "empty" values "", 0 and 0.0.

storage.Filtercopy(source)[(index)...](dest)

Example:

```
storage zz; storage fz;
storage.Split("0;5;0;7")(";")(zz);
storage.Filtercopy(zz)(fz);
storage.Join(fz)(d)(txt);
write(txt);
5, 7
storage.Filtercopy(s)(fz);        // the 3-D array flattened
storage.Join(fz)(d)(txt);
```

```
write(txt);
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
```

Minimum of the numeric values below the addressed node.
storage.Min(storage_name)[(index)...](target_var)
Example:

```
storage.Min(s)(res);      write(res);
storage.Min(s)(1)(res);   write(res);
1
7
```

Maximum of the numeric values below the addressed node.
storage.Max(storage_name)[(index)...](target_var)
Example:

```
storage.Max(s)(res);      write(res);
storage.Max(s)(0)(res);   write(res);
12
6
```

Average of the numeric values below the addressed node (non-numeric values are not counted).
storage.Average(storage_name)[(index)...](target_var)
Example:

```
storage.Average(s)(res);           write(res);
storage.Average(s)(0)(2)(res);     write(res);
6.5
5.5
```

Reads the first value of a **row** without removing it.
storage.First(storage_name)[(index)...](target_var)
Example:

```
storage.First(s)(1)(2)(res);       // row s[1][2] = 11 12
write(res);
11
```

Reads the last value of a **row** without removing it.
storage.Last(storage_name)[(index)...](target_var)
Example:

```
storage.Last(s)(1)(2)(res);        // row s[1][2] = 11 12
write(res);
12
```

Joins two storages (or sub-arrays, written as "s[1]") into target. Both sources must be on the same level, otherwise "sources are on different levels".
storage.Concat(storage1)(storage2)(target)
Example:

```
storage cc;
```

```
storage.Concat(s)(s)(cc);              // two copies of s
storage.Sum(cc)(res);
write(res);
156
storage c2;
storage.Concat("s[0]")("s[1]")(c2);    // 6 rows: slice 0, then
slice 1
storage.Sum(c2)(res);
write(res);
78
storage.Concat("s[0]")("s[0][1]")(c2);
Error: [storage.Concat] sources are on different levels
```

Sorts a **row** in ascending order: numerically when all values are numbers, otherwise as text. Stable.

storage.Sort(storage_name)[(index)...]

Example:

```
storage.Split("10;9;100;2")(";")(a);
storage.Sort(a);
storage.Join(a)(d)(txt);
write(txt);
2, 9, 10, 100
storage.Split("10;9;abc;2")(";")(a);
storage.Sort(a);                       // not all numbers: sorted as
                                       //   text
storage.Join(a)(d)(txt);
write(txt);
10, 2, 9, abc
storage.Sort(s);
Error: [storage.Sort] target holds sub-arrays: add indices down
      to the last dimension
```

Saves the storage to a CSV file (N-D storages get a _shape_ header line).

storage.Save(storage_name)(filename.csv)[(delimiter)]

If the delimiter is omitted, values are written one per line and rows are separated by an empty line. Only the .csv extension is accepted. The file can be loaded back with ReadFile.

Example:

```
storage.Save(s)("out.csv")(",");
File out.csv:
_shape_,2,3,2
1,2,,3,4,,5,6
7,8,,9,10,,11,12
```

Loads a CSV file. A first line _shape_,2,3,2 makes an N-dimensional storage; without it a flat list is read.

storage.ReadFile(storage_name)(filename.csv)[(delimiter)]

The delimiter is "," by default. A first line _shape_,2,3,2 restores an N-dimensional storage; without it a flat list (one row) or a table (several rows) is read. Backslashes in the path (examples\StorageLib\file.csv) also work on Linux.

Example:

```
storage t;
storage.ReadFile(t)("examples/StorageLib/test_nd_2d.csv")(",");
storage.Sum(t)(res);
write(res);
21
```

Position of the first occurrence of value in a **row**, or -1.

storage.IndexOf(storage_name)[(index)...](value)(target_var)

Example:

```
storage.IndexOf(s)(0)(1)(4)(res);   write(res);
storage.IndexOf(s)(0)(1)(9)(res);   write(res);
1
-1
```

Searches **all values below** the node; target_var becomes True or False.

storage.Contains(storage_name)[(index)...](value)(target_var)

Example:

```
storage.Contains(s)(12)(txt);         write(txt);
storage.Contains(s)(0)(12)(txt);      write(txt);
storage.Contains(s)(1)(2)(12)(txt);   write(txt);
True
False
True
```

Counts occurrences of value among all values below the node.

storage.CountOf(storage_name)[(index)...](value)(target_var)

Example:

```
storage.CountOf(s)(5)(res);    write(res);
storage.CountOf(s)(99)(res);   write(res);
1
0
```

Joins all values below the node into one string.

storage.Join(storage_name)[(index)...](delimiter)(target_var)

Example:

```
storage.Join(s)(d)(txt);
write(txt);
storage.Join(s)(0)(1)(d)(txt);
write(txt);
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
3, 4
```

Splits a string into a flat 1-D storage.

storage.Split(source_string)(delimiter)(target_storage)

Example:

```
storage b;
storage.Split("10;20;30")(";")(b);
storage.Join(b)(d)(txt);
write(txt);
10, 20, 30
```

Removes duplicates from a **row**, keeping the first occurrence (exact text comparison).

storage.Unique(storage_name)[(index)...]

Example:

```
storage.Split("3;7;9;3")(";")(a);
storage.Unique(a);
storage.Join(a)(d)(txt);
write(txt);
3, 7, 9
```

Randomizes the order of the children of the addressed node.

storage.Shuffle(storage_name)[(index)...]

Only the order of the children of the addressed node changes; the values inside them are untouched.

Example:

```
storage.Split("9;3;7")(";")(a);
storage.Shuffle(a);                 // the order is random
storage.Sum(a)(res);
write(res);
19
```

Product of the numeric values below the addressed node.

storage.Product(storage_name)[(index)...](target_var)

Example:

```
storage.Product(s)(0)(0)(res);   // row s[0][0] = 1 2
write(res);
2
```

Median of the numeric values below the addressed node.

storage.Median(storage_name)[(index)...](target_var)

Example:

```
storage.Median(s)(res);       write(res);
storage.Median(s)(1)(res);    write(res);
6.5
9.5
```

See file examples\StorageLib\TestND.scr (all functions on 1-D and N-D data) and examples\StorageLib\TestAll.scr.

NumPPL

NumPPL is a library of numeric functions for ForestPPL, in the style of NumPy. It works on ordinary PPL storages (for example storage a[2,3];) and adds what the storage and Math libraries do not have: creating arrays, changing their shape, element-wise arithmetic with broadcasting, simple statistics and basic linear algebra.
Functions that already exist are not repeated. Use Math.* (Abs, Sqrt, Exp, Log, Sin, Cos, ...) for scalar mathematics, storage.* (Sum, Min, Max, Average, Product, Median, Sort, Copy, ...) for working with the elements of a storage, and Random.* for random numbers.

Methods:

Zeros	**Sub**	**CumSum**
Ones	**Mul**	**ArgSort**
Full	**Div**	**Dot**
Eye	**Pow**	**Det**
Arange	**Clip**	**Inv**
Linspace	**Var**	**Solve**
Reshape	**Std**	**Trace**
Transpose	**ArgMin**	**Norm**
Add	**ArgMax**	

To get short help for every method in NumPPL.DLL:
>NumPPL.help[(method name)];

Getting started

Import the library once at the beginning of a script. All functions are then called with the prefix NumPPL.

```
import NumPPL;
```

Every argument is written in its own parentheses. Arguments in square brackets are optional.

```
NumPPL.Name(arg1)(arg2)...(destination);
```

General rules

The destination is always the last argument. It must exist before the call (storage r; or var r;) and its previous content is replaced completely. A source may also be the destination.

Where the result goes. An array is written to a storage. A single number (Det, Trace, Norm, Var, Std, ArgMin, ArgMax, and Dot of two vectors) is written to a var.

Operands. An operand of Add, Sub, Mul, Div, Pow, Clip, Dot and Solve is a storage, a number, or a variable holding a number.

Sources. A source must be a whole storage (not a part such as s[1]), rectangular and numeric.

Broadcasting. Shapes are compared from the right; an axis of size 1 is stretched to match the other operand, as in NumPy.

Errors. A function that fails prints Error: [NumPPL.Name] reason and stops the script.

Number format. Numbers are written with the shortest text that reads back as the same value (0.1, 1e-07, inf, nan).

In the examples below, the result of swrite is shown as plain rows of values. The real output also contains a header line with the name of the storage.

Creating arrays

Creates an array of the given shape filled with 0. Every dimension must be an integer >= 1.

NumPPL.Zeros(d1)[(d2)...](dst)

Example:

```
storage m;
NumPPL.Zeros(2)(3)(m);
swrite m;
0 0 0
0 0 0
```

Creates an array of the given shape filled with 1.

NumPPL.Ones(d1)[(d2)...](dst)

Example:

```
storage m;
NumPPL.Ones(2)(2)(m);
swrite m;
1 1
1 1
```

Creates an array of the given shape filled with the specified value.

NumPPL.Full(value)(d1)[(d2)...](dst)

Example:

```
storage m;
NumPPL.Full(7)(2)(2)(m);
swrite m;
7 7
7 7
```

Creates an identity matrix with n rows and n columns, or n rows and m columns when m is given (ones on the main diagonal, zeros elsewhere).

NumPPL.Eye(n)[(m)](dst)

Example:

```
storage c;
NumPPL.Eye(2)(c);
swrite c;
1 0
0 1
```

Creates a one-dimensional array start, start+step, start+2*step, ... while the value is strictly less than stop (strictly greater than stop when step is negative). The default step is 1.

NumPPL.Arange(start)(stop)[(step)](dst)

The step must be a non-zero finite number, and the range must not be empty.

Example:

```
storage a;
NumPPL.Arange(1)(6)(a);          // 1 2 3 4 5
swrite a;
NumPPL.Arange(5)(0)(-1)(a);      // 5 4 3 2 1
swrite a;
1 2 3 4 5
5 4 3 2 1
```

Creates a one-dimensional array of num evenly spaced values. Both ends, start and stop, are included.

NumPPL.Linspace(start)(stop)(num)(dst)

num must be an integer >= 1.

Example:

```
storage v;
NumPPL.Linspace(0)(1)(5)(v);
swrite v;
0 0.25 0.5 0.75 1
```

Changing the shape

Gives the same values a new shape. The values are taken in row-major order. One dimension may be -1; it is calculated from the number of values. The total number of values must not change.

NumPPL.Reshape(src)(d1)[(d2)...](dst)

Unlike a resize, nothing is added or lost. To add or remove elements use the storage functions.

Example:

```
storage v; storage m;
NumPPL.Arange(1)(7)(v);          // 1 2 3 4 5 6
NumPPL.Reshape(v)(2)(3)(m);
swrite m;
NumPPL.Reshape(v)(-1)(2)(m);     // -1 becomes 3
swrite m;
1 2 3
4 5 6
1 2
3 4
5 6
```

Reverses the order of the axes. For a matrix this exchanges rows and columns; for an array of rank 3, element [i][j][k] becomes [k][j][i].

NumPPL.Transpose(src)(dst)

The source and the destination may be the same storage.

Example:

```
storage v; storage m; storage mt;
NumPPL.Arange(1)(7)(v);
NumPPL.Reshape(v)(2)(3)(m);
NumPPL.Transpose(m)(mt);
swrite mt;
1 4
2 5
3 6
```

Element-wise arithmetic

Adds two operands element by element. An operand is a storage or a number (or a variable holding a number); at least one operand must be a storage. Shapes are matched as in NumPy (broadcasting): they are aligned from the right and an axis of size 1 is stretched.

NumPPL.Add(a)(b)(dst)

Example:

```
storage a; storage b; storage x;
NumPPL.Arange(0)(3)(a);                  // 0 1 2
NumPPL.Arange(10)(30)(10)(b);            // 10 20
NumPPL.Reshape(b)(2)(1)(b);              // a column
NumPPL.Add(b)(a)(x);
swrite x;
10 11 12
20 21 22
```

Subtracts b from a element by element, with the same operand and broadcasting rules as Add.

NumPPL.Sub(a)(b)(dst)

Example:

```
storage a; storage x;
NumPPL.Arange(1)(5)(a);          // 1 2 3 4
NumPPL.Sub(a)(1)(x);
swrite x;
0 1 2 3
```

Multiplies two operands element by element (not a matrix product; for that use Dot).

NumPPL.Mul(a)(b)(dst)

Example:

```
storage a; storage x;
NumPPL.Arange(0)(3)(a);          // 0 1 2
NumPPL.Mul(a)(2)(x);
swrite x;
0 2 4
```

Divides a by b element by element, with the same operand and broadcasting rules as Add.

NumPPL.Div(a)(b)(dst)

Example:

```
storage a; storage x;
NumPPL.Arange(1)(5)(a);          // 1 2 3 4
NumPPL.Div(a)(2)(x);
swrite x;
0.5 1 1.5 2
```

Raises a to the power b element by element, with the same operand and broadcasting rules as Add.

NumPPL.Pow(a)(b)(dst)

The destination may be the source: the result is written back into the same storage.

Example:

```
storage v;
NumPPL.Linspace(0)(1)(5)(v);
NumPPL.Pow(v)(2)(v);
swrite v;
0 0.0625 0.25 0.5625 1
```

Limits every value to the range [lo, hi]. Values below lo become lo, values above hi become hi.

NumPPL.Clip(a)(lo)(hi)(dst)

lo must not be greater than hi.

Example:

```
storage a; storage x;
NumPPL.Arange(1)(6)(a);          // 1 2 3 4 5
NumPPL.Clip(a)(2)(4)(x);
swrite x;
2 2 3 4 4
```

Statistics

Calculates the population variance of all values of a storage (the sum of squared deviations from the mean divided by the number of values). The result is written to a variable.

NumPPL.Var(src)(var)

Example:

```
storage a; var r;
NumPPL.Arange(1)(6)(a);
NumPPL.Var(a)(r);
write(r);
2
```

Calculates the population standard deviation (the square root of Var). The result is written to a variable.

NumPPL.Std(src)(var)

Example:

```
storage a; var r;
NumPPL.Arange(1)(6)(a);
NumPPL.Std(a)(r);
write(r);
1.4142135623730951
```

Returns the flat (row-major) index of the first minimum value. The result is written to a variable.

NumPPL.ArgMin(src)(var)

Example:

```
storage a; var r;
NumPPL.Arange(1)(6)(a);
NumPPL.ArgMin(a)(r);
write(r);
0
```

Returns the flat (row-major) index of the first maximum value. The result is written to a variable.

NumPPL.ArgMax(src)(var)

Example:

```
storage a; var r;
NumPPL.Arange(1)(6)(a);
NumPPL.ArgMax(a)(r);
write(r);
4
```

Calculates the running (cumulative) sum over the flattened array. Each element of the result is the sum of all elements up to and including it.

NumPPL.CumSum(src)(dst)

Example:

```
storage a; storage x;
NumPPL.Arange(1)(6)(a);
NumPPL.CumSum(a)(x);
swrite x;
1 3 6 10 15
```

Returns the flat indices that would sort the array in ascending order. The sort is stable, and NaN values are placed last. The source array itself is not changed (to sort the values in place use storage.Sort).

NumPPL.ArgSort(src)(dst)

Example:

```
storage b; storage x;
NumPPL.Arange(5)(0)(-1)(b);     // 5 4 3 2 1
NumPPL.ArgSort(b)(x);
swrite x;
4 3 2 1 0
```

Linear algebra

Calculates the dot product. Vector . vector gives a number, which is written to a variable. Matrix . matrix, matrix . vector and vector . matrix give a storage. The inner dimensions must match.

NumPPL.Dot(a)(b)(dst)

Example:

```
storage v; storage m; storage mt; storage g;
NumPPL.Arange(1)(7)(v);
NumPPL.Reshape(v)(2)(3)(m);    // 1 2 3 ; 4 5 6
NumPPL.Transpose(m)(mt);
NumPPL.Dot(m)(mt)(g);         // M * M^T
swrite g;
14 32
32 77
```

Calculates the determinant of a square matrix. The result is written to a variable. The determinant of a singular matrix is 0.

NumPPL.Det(a)(var)

Example:

```
storage a; storage m; var r;
NumPPL.Arange(1)(5)(a);
NumPPL.Reshape(a)(2)(2)(m);    // 1 2 ; 3 4
NumPPL.Det(m)(r);
write(r);
-2
```

Calculates the inverse of a square matrix (Gauss-Jordan elimination with partial pivoting). An error is reported if the matrix is singular.

NumPPL.Inv(a)(dst)

Calculations are done in double precision, so the last digits may show rounding noise: -1.9999999999999996 instead of -2. Compare results with a tolerance, not for exact equality.

Example:

```
storage a; storage m; storage x;
NumPPL.Arange(1)(5)(a);
NumPPL.Reshape(a)(2)(2)(m);    // 1 2 ; 3 4
NumPPL.Inv(m)(x);
swrite x;
-2 1
1.5 -0.5    (up to rounding)
```

Solves the linear system A x = b. A is a square matrix; b is a vector or a matrix with the same number of rows as A.

NumPPL.Solve(a)(b)(dst)

The result may show rounding noise in the last digits, as for Inv.

Example:

```
storage a; storage b; storage m; storage x;
NumPPL.Arange(1)(5)(a);
NumPPL.Reshape(a)(2)(2)(m);   // 1 2 ; 3 4
NumPPL.Arange(5)(7)(b);      // 5 6
NumPPL.Solve(m)(b)(x);
swrite x;
-4 4.5   (up to rounding)
```

Calculates the trace of a square matrix: the sum of the main diagonal. The result is written to a variable.

NumPPL.Trace(a)(var)

Example:

```
storage a; storage m; var r;
NumPPL.Arange(1)(5)(a);
NumPPL.Reshape(a)(2)(2)(m);   // 1 2 ; 3 4
NumPPL.Trace(m)(r);
write(r);
5
```

Calculates the Euclidean norm of a vector, or the Frobenius norm of a matrix: the square root of the sum of squares of all values. The result is written to a variable.

NumPPL.Norm(a)(var)

Example:

```
storage a; var r;
NumPPL.Arange(1)(6)(a);
NumPPL.Norm(a)(r);
write(r);
7.416198487095663
```

Complete example: solving a linear system

The system 1x + 2y = 5, 3x + 4y = 6 is solved and checked.

```
import NumPPL;
storage a; storage b; storage m; storage x; storage chk;
var det;

NumPPL.Arange(1)(5)(a);
NumPPL.Reshape(a)(2)(2)(m);     // coefficients: 1 2 ; 3 4
NumPPL.Arange(5)(7)(b);         // right-hand side: 5 6

NumPPL.Det(m)(det);             // -2: the system has a unique
solution
write(det);
NumPPL.Solve(m)(b)(x);          // x = -4, y = 4.5
swrite x;
NumPPL.Dot(m)(x)(chk);          // check: m * x should give 5 6
swrite chk;
```

Because of floating-point rounding the check may print a value such as 4.999999999999999 instead of 5.

Error messages

Each message below was produced by a separate script. A failing function stops the script, so only the first error is reported.

```
NumPPL.Zeros(0)(x);
Error: [NumPPL.Zeros] dimension [0] must be an integer >= 1
NumPPL.Zeros(2)(nope);
Error: [NumPPL.Zeros] target variable [nope] not found or is not
 a storage.
NumPPL.Arange(0)(5)(0)(x);
Error: [NumPPL.Arange] step must be a non-zero finite number
NumPPL.Arange(5)(0)(x);
Error: [NumPPL.Arange] the range is empty
NumPPL.Linspace(0)(1)(0)(x);
Error: [NumPPL.Linspace] num must be an integer >= 1
NumPPL.Add(a)(b)(x);   // lengths 3 and 4
Error: [NumPPL.Add] shapes cannot be broadcast together
NumPPL.Add(2)(3)(x);
Error: [NumPPL.Add] at least one operand must be a storage
NumPPL.Add(a)("abc")(x);
Error: [NumPPL.Add] [abc] is neither a storage nor a number
NumPPL.Reshape(a)(4)(x);   // 3 values
Error: [NumPPL.Reshape] cannot reshape 3 values to this shape
NumPPL.Reshape(a)(-1)(-1)(x);
Error: [NumPPL.Reshape] only one dimension may be -1
NumPPL.Clip(a)(5)(1)(x);
Error: [NumPPL.Clip] lo must not be greater than hi
NumPPL.Var(5)(r);
Error: [NumPPL.Var] the source must be a storage
NumPPL.Dot(a)(b)(r);   // lengths 3 and 4
Error: [NumPPL.Dot] shapes do not match: 3 vs 4
NumPPL.Det(m)(r);   // 2x3 matrix
Error: [NumPPL.Det] the argument must be a square matrix
NumPPL.Inv(s)(x);   // singular matrix
Error: [NumPPL.Inv] the matrix is singular
NumPPL.Solve(s)(b)(x);   // wrong length of b
Error: [NumPPL.Solve] b must be a vector or a matrix with 2 rows
```

The interpreter may print its own line before the library message. For example, a destination variable that does not exist first gives "variable [nope] not found in scope".

Requests

Requests is an HTTP client library for ForestPPL in the style of the Python requests package. It is a user library built with CodeGen (like Queue) and implemented on top of libcurl. It sends GET, POST, PUT, PATCH, HEAD and any other HTTP request, follows redirects, decodes compressed responses, reads JSON by path, saves and downloads files, and supports headers, Basic and Bearer authentication, timeouts, TLS verification and proxies.

Like Queue, objects are named: every request stores its response under a name you choose, and the accessors (Status, Text, Header, JsonGet ...) read it back by that name. A request function returns the HTTP status code, so assign it to a variable: s = Requests.Get("r")(url);

The library needs libcurl. Windows (MS Dev Studio): vcpkg install curl:x64-windows, add the include directory, link libcurl.lib and put libcurl.dll (with the DLLs it depends on) next to Requests.dll. Linux / WSL2: sudo apt install libcurl4-openssl-dev and link with -lcurl. Load the library with import Requests; or add (UserImportN [Requests]) to Configuration.data.

Example:

```
import Requests;
var s;
s =
Requests.Get("r")("https://api.github.com/repos/okogosov/ForestPPL");
s = Requests.JsonGet("r")("full_name");
write(s);
okogosov/ForestPPL
```

Methods:

Get	**Text**	**ClearHeaders**
Post	**Header**	**SetAuth**
PostJson	**Headers**	**SetBearer**
Put	**Url**	**SetVerify**
Patch	**Elapsed**	**SetUserAgent**
Head	**Error**	**SetProxy**
Request	**JsonGet**	**Write**
Download	**Save**	**Count**
Status	**SetTimeout**	**Remove**
Ok	**SetHeader**	**Delete**

To get short help for every method in Requests.DLL:
>Requests.help[(method name)];

Test server for the examples

Most examples below run against a small local test server, so that they work without internet access and give the same results every time. It is the file examples/requests/requests_server.py:

```
python requests_server.py 8099
```

The server listens on http://127.0.0.1:8099 and offers the endpoints /get, /anything, /headers, /json, /status/<code>, /redirect/<n>, /basic-auth/<user>/<password>, /bearer, /delay/<seconds>, /resp-headers and /download. Its replies are JSON documents that echo the request, so the examples can show what was really sent. Every example starts with import Requests; and declares var s;

Requests - each stores the response under response_name and returns the status code

Sends an HTTP GET request, stores the response under response_name and returns the HTTP status code (0 if the request failed before a response arrived). Up to 10 redirects are followed and compressed (gzip/deflate) responses are decoded.

Requests.Get(response_name)(url)

A response name that is already in use is replaced by the new response.

Example:

```
s = Requests.Get("r")("http://127.0.0.1:8099/get?x=1&y=two");
write(s);
s = Requests.JsonGet("r")("args.y");
write(s);
200
two
```

Sends an HTTP POST request with form data (Content-Type: application/x-www-form-urlencoded) and returns the status code.

Requests.Post(response_name)(url)(data)

data is a string of the form "key=value&key2=value2".

Example:

```
s =
Requests.Post("p")("http://127.0.0.1:8099/anything")("name=Ann&age=30
");
write(s);
s = Requests.JsonGet("p")("form.name");
write(s);
s = Requests.JsonGet("p")("form.age");
write(s);
200
Ann
30
```

Sends an HTTP POST request with the body json_string and Content-Type: application/json, and returns the status code.

Requests.PostJson(response_name)(url)(json_string)

Quotes inside a PPL string are written as \" and reach the server unchanged.

Example:

```
var js;
js = "{\"name\":\"Ann\",\"tags\":[\"x\",\"y\"]}";
s =
Requests.PostJson("j")("http://127.0.0.1:8099/anything")(js);
write(s);
s = Requests.JsonGet("j")("json.name");
write(s);
s = Requests.JsonGet("j")("json.tags[1]");
write(s);
s = Requests.JsonGet("j")("headers.Content-Type");
write(s);
200
Ann
y
application/json
```

Sends an HTTP PUT request with the body data and returns the status code.

Requests.Put(response_name)(url)(data)

Example:

```
s = Requests.Put("u")("http://127.0.0.1:8099/anything")("v=1");
s = Requests.JsonGet("u")("method");
write(s);
s = Requests.JsonGet("u")("body");
write(s);
PUT
v=1
```

Sends an HTTP PATCH request with the body data and returns the status code.

Requests.Patch(response_name)(url)(data)

Example:

```
s =
Requests.Patch("u")("http://127.0.0.1:8099/anything")("w=2");
s = Requests.JsonGet("u")("method");
write(s);
s = Requests.JsonGet("u")("body");
write(s);
PATCH
w=2
```

Sends an HTTP HEAD request (only the headers are received, the body is empty) and returns the status code.

Requests.Head(response_name)(url)

Example:

```
s = Requests.Head("h")("http://127.0.0.1:8099/get");
write(s);
s = Requests.Header("h")("Content-Type");
write(s);
200
```

```
      application/json
```

Sends a request with any HTTP method (DELETE, OPTIONS, ...) and returns the status code. The method name is written in any case and is converted to upper case; the data is optional.

Requests.Request(response_name)(method)(url)[(data)]

A method name that contains anything but letters is rejected with "wrong method".

Example:

```
      s =
Requests.Request("d")("delete")("http://127.0.0.1:8099/anything/7");
      write(s);
      s = Requests.JsonGet("d")("method");
      write(s);
      s = Requests.JsonGet("d")("path");
      write(s);
      200
      DELETE
      /anything/7
```

Downloads the url straight into a file and returns the status code. Unlike Get, no response is stored, so the method also suits large and binary files.

Requests.Download(url)(filename)

A transport error (refused connection, timeout, DNS, TLS) is reported as an error message and stops the script.

Example:

```
      s =
Requests.Download("http://127.0.0.1:8099/download")("data.bin");
      write(s);
      200
```

Response accessors

Returns the HTTP status code of the stored response, or 0 if the request failed before a response arrived.

Requests.Status(response_name)

Example:

```
      s = Requests.Get("e")("http://127.0.0.1:8099/status/404");
      s = Requests.Status("e");
      write(s);
      404
```

Returns True if the request succeeded (no transport error) and the status code is below 400, otherwise False.

Requests.Ok(response_name)

Example:

```
      s = Requests.Get("e")("http://127.0.0.1:8099/status/404");
      s = Requests.Ok("e");
```

```
write(s);
s = Requests.Get("e")("http://127.0.0.1:8099/get");
s = Requests.Ok("e");
write(s);
False
True
```

Returns the body of the stored response as text.

Requests.Text(response_name)

The body is kept as raw bytes in a string. Text of a binary response is not meaningful: use Download for binary files.

Example:

```
s = Requests.Get("e")("http://127.0.0.1:8099/status/404");
s = Requests.Text("e");
write(s);
status 404
```

Returns the value of a response header. The name is case-insensitive; a header that occurs several times is returned joined with ", ". If the header is absent the result is nan.

Requests.Header(response_name)(header_name)

Example:

```
s = Requests.Get("rh")("http://127.0.0.1:8099/resp-headers");
s = Requests.Header("rh")("x-test");
write(s);
s = Requests.Header("rh")("X-Multi");
write(s);
s = Requests.Header("rh")("X-Nope");
write(s);
abc
one, two
nan
```

Returns all headers of the stored response as text, one "Name: value" per line (the headers of the final response, after redirects).

Requests.Headers(response_name)

The set of headers depends on the server; the output below is an excerpt.

Example:

```
s = Requests.Get("rh")("http://127.0.0.1:8099/resp-headers");
s = Requests.Headers("rh");
write(s);
Content-Type: text/plain
Content-Length: 7
X-Test: abc
X-Multi: one
X-Multi: two
```

Returns the final URL of the response, i.e. the address reached after all redirects.

Requests.Url(response_name)

Example:

```
      s = Requests.Get("rd")("http://127.0.0.1:8099/redirect/3");
// three redirects
      s = Requests.Url("rd");
      write(s);
      http://127.0.0.1:8099/get
```

Returns the duration of the request in milliseconds.

Requests.Elapsed(response_name)

The value differs from run to run.

Example:

```
      s = Requests.Get("r")("http://127.0.0.1:8099/get");
      s = Requests.Elapsed("r");
      write(s);
      2.871
```

Returns the transport error text (timeout, refused connection, DNS, TLS ...), or an empty string if the request succeeded.

Requests.Error(response_name)

A network failure is not a script error: Get returns 0, Ok returns False and Error tells the reason. The text comes from libcurl and differs between versions and systems.

Example:

```
      s = Requests.Get("x")("http://127.0.0.1:1/");
      write(s);
      s = Requests.Ok("x");
      write(s);
      s = Requests.Error("x");
      write(s);
      0
      False
      Failed to connect to 127.0.0.1 port 1 after 0 ms: Couldn't
connect to server
```

Returns a value from the JSON body of the stored response by path, or nan if the path does not exist (or the body is not JSON). Strings are returned decoded, numbers, true, false and null as text, objects and arrays as raw JSON text.

Requests.JsonGet(response_name)(path)

Path syntax: a.b[0].c - keys separated by dots, array elements by an index in square brackets.

Example:

```
      s = Requests.Get("js")("http://127.0.0.1:8099/json");
      s = Requests.JsonGet("js")("name");                    write(s);
      s = Requests.JsonGet("js")("version");                 write(s);
      s = Requests.JsonGet("js")("ok");                      write(s);
      s = Requests.JsonGet("js")("tags[2]");                 write(s);
      s = Requests.JsonGet("js")("nested.items[0].title");  write(s);
      s = Requests.JsonGet("js")("nope");                    write(s);
      Forest
```

```
5
true
c
one
nan
```

Writes the body of the stored response to a file and returns True.

Requests.Save(response_name)(filename)

Example:

```
s = Requests.Get("d")("http://127.0.0.1:8099/json");
s = Requests.Save("d")("saved.json");
write(s);
True
```

Session settings - they apply to every following request

Sets the timeout in seconds for the following requests (connection and whole request). The default is 30; 0 means no timeout. A request that runs out of time returns status 0.

Requests.SetTimeout(seconds)

seconds must be a non-negative integer.

Example:

```
Requests.SetTimeout(1);
s = Requests.Get("t")("http://127.0.0.1:8099/delay/3");
write(s);
s = Requests.Ok("t");
write(s);
Requests.SetTimeout(30);
s = Requests.Get("t")("http://127.0.0.1:8099/delay/1");
write(s);
0
False
200
```

Sets a header that is sent with every following request. A header with the same name (in any case) is replaced.

Requests.SetHeader(name)(value)

The name must not be empty and must not contain a colon.

Example:

```
Requests.SetHeader("X-Custom")("hello");
s = Requests.Get("r")("http://127.0.0.1:8099/headers");
s = Requests.JsonGet("r")("headers.X-Custom");
write(s);
Requests.SetHeader("x-custom")("replaced");
s = Requests.Get("r")("http://127.0.0.1:8099/headers");
s = Requests.JsonGet("r")("headers.X-Custom");
write(s);
hello
replaced
```

Removes all headers that were set with SetHeader.

Requests.ClearHeaders()

Example:

```
Requests.SetHeader("X-Custom")("hello");
Requests.ClearHeaders();
s = Requests.Get("r")("http://127.0.0.1:8099/headers");
s = Requests.JsonGet("r")("headers.X-Custom");
write(s);
nan
```

Turns on HTTP Basic authentication for the following requests. An empty user turns it off. Setting Basic authentication clears a Bearer token.

Requests.SetAuth(user)(password)

Example:

```
s = Requests.Get("a")("http://127.0.0.1:8099/basic-
auth/user/pw");
write(s);
Requests.SetAuth("user")("pw");
s = Requests.Get("a")("http://127.0.0.1:8099/basic-
auth/user/pw");
write(s);
s = Requests.JsonGet("a")("authenticated");
write(s);
Requests.SetAuth("")("");            // switch it off
401
200
true
```

Sends the header Authorization: Bearer token with the following requests. An empty token turns it off. Setting a token clears Basic authentication.

Requests.SetBearer(token)

An Authorization header set with SetHeader has priority over the token.

Example:

```
s = Requests.Get("b")("http://127.0.0.1:8099/bearer");
write(s);
Requests.SetBearer("tok123");
s = Requests.Get("b")("http://127.0.0.1:8099/bearer");
write(s);
s = Requests.JsonGet("b")("token");
write(s);
401
200
tok123
```

Turns the verification of TLS certificates on or off for the following https requests. The default is yes.

Requests.SetVerify(yes|no)

Also accepted: true|false and 1|0. Turn the verification off only for tests, for example with a self-signed certificate: the connection is no longer protected against impersonation.

Example:

```
Requests.SetVerify(no);
s = Requests.Get("t")("https://localhost:8443/");
Requests.SetVerify(yes);
```

Sets the User-Agent header. The default is ForestPPL-Requests/1.0.

Requests.SetUserAgent(text)

Example:

```
s = Requests.Get("r")("http://127.0.0.1:8099/headers");
s = Requests.JsonGet("r")("headers.User-Agent");
write(s);
Requests.SetUserAgent("MyAgent/2");
s = Requests.Get("r")("http://127.0.0.1:8099/headers");
s = Requests.JsonGet("r")("headers.User-Agent");
write(s);
ForestPPL-Requests/1.0
MyAgent/2
```

Sets a proxy server for the following requests. An empty string turns the proxy off.

Requests.SetProxy(url)

Example:

```
Requests.SetProxy("http://proxy.example.com:3128");
s = Requests.Get("r")("https://api.github.com/");
Requests.SetProxy("");              // direct connection again
```

Stored responses

Writes to the standard output: the names of all stored responses (without a parameter), or the details of one response - status, final URL, transport error, headers and body.

Requests.Write([response_name])

If no response is stored, the warning "table of responses is empty" is printed. In the output below the Server and Date headers sent by the server are omitted.

Example:

```
s = Requests.Get("rh")("http://127.0.0.1:8099/resp-headers");
s = Requests.Get("r")("http://127.0.0.1:8099/get");
Requests.Write();
Requests.Write("rh");
rh
r
===rh===
    status  : 200
    url     : http://127.0.0.1:8099/resp-headers
    header  : Content-Type: text/plain
    header  : Content-Length: 7
    header  : X-Test: abc
```

```
header   : X-Multi: one
header   : X-Multi: two
body     : headers
```

Returns the number of stored responses.

Requests.Count()

Example:

```
s = Requests.Get("a")("http://127.0.0.1:8099/get");
s = Requests.Get("b")("http://127.0.0.1:8099/get");
s = Requests.Count();
write(s);
2
```

Removes one stored response.

Requests.Remove(response_name)

Example:

```
s = Requests.Get("a")("http://127.0.0.1:8099/get");
s = Requests.Get("b")("http://127.0.0.1:8099/get");
Requests.Remove("a");
s = Requests.Count();
write(s);
1
```

Deletes all stored responses. The session settings (timeout, headers, authentication, ...) are kept.

Requests.Delete()

Example:

```
s = Requests.Get("a")("http://127.0.0.1:8099/get");
Requests.Delete();
s = Requests.Count();
write(s);
0
```

Behaviour worth knowing

A network failure is not a script error. Get returns 0, Ok is False and Error has the reason (refused connection, timeout, DNS, TLS). Wrong usage - an unknown response name, a wrong number of parameters, a bad method - prints Error: [...] and stops the script, like in the other libraries.

Response names. Using an existing name again replaces that response. Names are written in quotes.

Request parameters. Parameters can be string literals or variables. Strings with quotes inside use \" in a PPL string.

Binary data. Bodies are kept as raw bytes in a string. Use Download or Save for binary files.

Error messages

Wrong usage prints a message and stops the script. Typical messages:

```
s = Requests.Get("r");
Error: [Requests.FuncGet] wrong number of parameters != 2
s = Requests.Get("")("http://...");
Error: [Requests.FuncGet] empty response name
s = Requests.Get("r")("");
Error: [Requests.FuncGet] empty url
s = Requests.Status("no_such");
Error: [Requests.FuncStatus] wrong name [no_such]
s = Requests.Request("r")("GE1")(url);
Error: [Requests.FuncRequest] wrong method [GE1]
s = Requests.Save("r")("no_dir/x.txt");
Error: [Requests.FuncSave] cannot open file [no_dir/x.txt]
Requests.SetTimeout("abc");
Error: [Requests.FuncSetTimeout] timeout must be a non-negative
 integer, got [abc]
Requests.SetHeader("a:b")("x");
Error: [Requests.FuncSetHeader] wrong header name [a:b]
Requests.SetVerify("maybe");
Error: [Requests.FuncSetVerify] expected yes|no, got [maybe]
Requests.ClearHeaders(1);
Error: [Requests.FuncClearHeaders] wrong number of parameters !=
                          0
```

Tests

The folder examples/requests contains scripts that check themselves: they print PASS ... or FAIL ... lines and a final line RESULT: PASS n FAIL m. They use the local test server, so no internet access is needed.

requests_basic.scr - Get, accessors, status codes, redirects, Head, response headers, Write / Remove / Delete / Count

requests_post.scr - Post, PostJson (with the quote round trip), Put, Patch, Request

requests_auth_headers.scr - SetHeader, ClearHeaders, SetUserAgent, SetAuth, SetBearer

requests_json.scr - JsonGet paths and missing values

requests_files.scr - Save and Download

requests_errors.scr - refused connection, timeout, wrong usage

requests_common.scr - shared check / summary helpers, included with rc

Copy the folder to <ForestPPL dir>/examples/requests, start the server and run the scripts:

```
python requests_server.py 8099
rc examples\requests\requests_basic.scr
rc examples\requests\requests_post.scr
```

Structure of User's DLL

Utility **CodeGen.exe** generates files .h and .cpp in accordance with structure of User's DLL, described in configuration file.

Example:

Configuration file **Stack.cfg**

```
ProjectName = Stack
Methods = Create,Delete,Count,Write,Push,Pop,Peek,Clear,Contains,
          AddArray,ToArray
```

Run **CodeGen.exe Stack.cfg**

CodeGen.exe generates 2 files: Stack.cpp and Stack.h

Stack.cpp

```
#include "pch.h"
#include "..\PPL\PPL.h"
#include "..\PPL\Component\Component.h"
#include "..\PPL\Processing\processing.h"
#include "Stack.h"

using namespace std;
namespace PPLNS
{
  static Stack* STACKInstance = nullptr;

  void Stack_CreateInstance(PPL* ppl)
  {
    STACKInstance = new Stack(ppl);
    STACKInstance->AddToKeywordDictionary();
  }

 Stack::Stack(PPL* ppl)
  {
    this->ppl = ppl;
    keyword_dict = new unordered_map<string, function<bool(const
vector<string>&, string&, Composite*)>>;
  }
  //=========================================================
  void Stack::AddToKeywordDictionary()
  {
    //help_dict is created in BaseClass
    AddKeyword("help", BaseClass::FuncHelp);
    AddKeyword("Create", FuncCreate);
    AddKeyword("Delete", FuncDelete);
    AddKeyword("Count", FuncCount);
    AddKeyword("Write", FuncWrite);
    AddKeyword("Push", FuncPush);
    AddKeyword("Pop", FuncPop);
    AddKeyword("Peek", FuncPeek);
    AddKeyword("Clear", FuncClear);
    AddKeyword("Contains", FuncContains);
```

```
    AddKeyword("AddArray", FuncAddArray);
    AddKeyword("ToArray", FuncToArray);

    help_dict->insert({ "help", "\tStack.help([name])" });
    help_dict->insert({ "Create", "\t..." });
    help_dict->insert({ "Delete", "\t..." });
    help_dict->insert({ "Count", "\t..." });
    help_dict->insert({ "Write", "\t..." });
    help_dict->insert({ "Push", "\t..." });
    help_dict->insert({ "Pop", "\t..." });
    help_dict->insert({ "Peek", "\t..." });
    help_dict->insert({ "Clear", "\t..." });
    help_dict->insert({ "Contains", "\t..." });
    help_dict->insert({ "AddArray", "\t..." });
    help_dict->insert({ "ToArray", "\t..." });
    for (const auto pair : *keyword_dict)
    {
      string key = "Stack." + pair.first;
      ppl->processing->keyword_dict->insert({ key, pair.second });
    }
    ppl->ImportList.insert({ "Stack", this });
  }
  //============================================================
  bool Stack::FuncCreate(const vector<string>& parameters, string&
result, Composite* node) { return true; }
  bool Stack::FuncDelete(const vector<string>& parameters, string&
result, Composite* node) { return true; }
  bool Stack::FuncCount(const vector<string>& parameters, string&
result, Composite* node) { return true; }
  bool Stack::FuncWrite(const vector<string>& parameters, string&
result, Composite* node) { return true; }
 bool Stack::FuncPush(const vector<string>& parameters, string&
result, Composite* node) { return true; }
  bool Stack::FuncPop(const vector<string>& parameters, string&
result, Composite* node) { return true; }
  bool Stack::FuncPeek(const vector<string>& parameters, string&
result, Composite* node) { return true; }
  bool Stack::FuncClear(const vector<string>& parameters, string&
result, Composite* node) { return true; }
  bool Stack::FuncContains(const vector<string>& parameters, string&
result, Composite* node) { return true; }
  bool Stack::FuncAddArray(const vector<string>& parameters, string&
result, Composite* node) { return true; }
  bool Stack::FuncToArray(const vector<string>& parameters, string&
result, Composite* node) { return true; }
}
```

Stack.h

```
/************************************************************
*This code generated by Application CodeGen.exe             *
*for creation User DLL                                      *
*Add  Project Reference PPL.Lib                             *
*Add  pch.h from Project PPL                                *
*Author: Oscar Kogosov, email: ok21@hotmail.com             *
*You must not remove this notice from this software.        *
************************************************************/
#ifndef STACK_H
#define STACK_H

#ifdef STACK_EXPORTS
#define STACK_API __declspec(dllexport)
#else
#define STACK_API __declspec(dllimport)
#endif

namespace PPLNS
{
  class STACK_API Stack : public BaseClass
  {
  public:
    Stack(PPL* ppl);
    void AddToKeywordDictionary();
    bool FuncCreate(const vector<string>& parameters, string& result,
Composite* node);
    bool FuncDelete(const vector<string>& parameters, string& result,
Composite* node);
    bool FuncCount(const vector<string>& parameters, string& result,
Composite* node);
    bool FuncWrite(const vector<string>& parameters, string& result,
Composite* node);
    bool FuncPush(const vector<string>& parameters, string& result,
Composite* node);
    bool FuncPop(const vector<string>& parameters, string& result,
Composite* node);
    bool FuncPeek(const vector<string>& parameters, string& result,
Composite* node);
    bool FuncClear(const vector<string>& parameters, string& result,
Composite* node);
    bool FuncContains(const vector<string>& parameters, string&
result, Composite* node);
    bool FuncAddArray(const vector<string>& parameters, string&
result, Composite* node);
    bool FuncToArray(const vector<string>& parameters, string&
result, Composite* node);
  };
  extern "C" void STACK_API Stack_CreateInstance(PPL* ppl);
}
#endif
```

Add commentary in lines:

```
help_dict->insert({ "…", "\t..." });
```

Add code to each of methods in Stack.cpp –

```
      FuncCreate,
      FuncDelete,
      …….
```

Add to project file from CodeGen\x64\Debug\pch.h
Add to Project References the project **PPL.**
Add created DLL-file to directory ForestPPL.

Error detection

Error detection in script files is carried out in three stages -
1. at the stage of reading files - checking the pairing of parentheses and square brackets, as well as the pairing of quotes;
2. at the pre-translation stage - checking the syntax of operators and commands, indicating name of PPL-module, the file name and line number;
3. at runtime stage.

Right code:

```
var x = 1;
write (x);
```

```
var x = 1;
write x);
Error: [TCheckNumberOfParentheses] not paired number of parentheses
       [write x)]
```

```
var x = 1;
awrite (x);
Error:  [TCreateCodeTree] file: [examples\err5.scr] line: [2] wrong
       cmd [awrite]
```

```
var x  1;
write (x);
Error:  [TFuncVar] file: [examples\err5.scr] line: [1] wrong format
       cmd 'var' [var x  1;]
```

```
defence from 'glued commands'
>var x = 0 display
Error:  [TFuncVar] syntax error: unexpected tokens in expression
(missing semicolon?) [var x = 0 display;]
```

Error detection in interactive mode:

```
>var x 1;
Error:  [TFuncVar] wrong format cmd 'var' [var x 1;]
```

Before executing the script, a syntax check is performed if **checksyntax yes**.

Inside checksyntax: the syntax table (SyntaxChecker)

The command **checksyntax yes** relies on the class `SyntaxChecker`. It knows nothing about individual commands: all knowledge is in a data table, written as JSON in the file **DefaultSyntaxTable.h**. A new command, or a new calling form of an existing one, is added by editing that table, not the checker.

Where the table lives

The header contains one constant, `kDefaultSyntaxTableJson`, a C++ raw string literal that holds the whole JSON text:

```
inline constexpr std::string_view kDefaultSyntaxTableJson =
R"SYNTAX_JSON({
 "write": {
  "group": "Input and Output",
  "kind": "core",
  "any": [ "write(value) | write(c# format)( value )(value)…" ]
 },
 ...
})SYNTAX_JSON";
```

The literal is compiled into the executable. On the first use `SyntaxChecker::instance()` parses it with `load_table()` (the small parser in `json_mini.hpp`) and translates every pattern into a `std::regex`, once for the whole process. Reading an external `syntax_table.json` instead is prepared in `instance()` but is commented out in the current build, so **after editing DefaultSyntaxTable.h the interpreter must be rebuilt**.

Structure of a record

The JSON object maps a command name to a record. Real example for a library function:

```
 "Storage.Sum": {
  "group": "Storage",
  "kind": "lib",
  "call": [ "storage.Sum(storage_name)[(idx1)...](target_var)" ]
 },
```

Field	Meaning
`"Storage.Sum"` (key)	Command name. Special forms: "display (d)" = names display and d; "start, stop" and "A and B" = several commands with one record. Library entries use Library.Function. All names are matched case-insensitively and become the alias index.
`group`	Section of the documentation the command belongs to (used for reports only).
`kind`	"core" for built-in commands, "lib" for library functions.
`ppl`	Patterns valid only in interactive (PPL, prefix) mode.
`scr`	Patterns valid only in script (SCR, infix) mode.
`any`	Patterns valid in both modes.
`call`	Patterns for the function-call form (storage.Sum(...), String.Split(...), ...).

Pattern notation

A pattern is not a regular expression. It is a readable description of the call (the same text that documents the command), which the checker turns into a regular expression:

Pattern	Example	Meaning
`literal word / symbol`	`storage.Sum`	Must appear exactly as written (case-insensitive for command names).
`(x)`	`(target_var)`	Literal parentheses with a placeholder inside. Spaces before "(" are tolerated.
`placeholder word`	`storage_name, value, idx1`	Any single value: a number, a name, a quoted string or an expression. The word itself is only documentation.
`[ ... ]`	`[(delimiter)]`	Optional part.
`a | b`	`yes | no`	Alternatives; inside "(...)" the bar applies only to that pair of parentheses.
`... or …`	`call function_name(…)`	Free continuation (any further arguments).
`[(idx1)...]`	`storage.Sum(s)[(idx1)...](r)`	Zero or more extra parenthesised groups (the N-D indices of StorageLib).
`name:= ...`	`length:= value | ppl expression`	A documentation line that defines a placeholder. It is skipped when patterns are compiled.

If a pattern does not start with the command name, the name is put in front of it automatically, so records can be written briefly ("`var (name)| (name)(initial value)`").

How one statement is checked

`IsValidCommandSyntax(statement, mode)` returns one of three verdicts: **Valid**, **Invalid** or **Unknown** (the checker has no opinion; this is not an error). Steps:

Step	What happens
1. Normalise	Trim the text and remove a leading ">" (the console prompt). An empty statement gives Unknown.
2. Identify the command	identify_command() looks the first word up in the alias index, longest alias first, and accepts it only if the next character is a

Step	What happens
	boundary (space, "(", ";" ...). No match: a bare assignment ("x = 5;") is Valid, anything else is Unknown ("unrecognized command").
3. Look for a missing ";"	find_concatenation() scans the rest of the statement for a second command keyword (it knows that "call", "if", "while", "for" and "else" legitimately take another command as an argument). If one is found the result is Invalid: "looks like a missing ';'".
4. Choose the patterns	The patterns of the record are compiled once into std::regex objects. Modes are tried in order: the requested mode, then "any", then "call"; without a requested mode: ppl, scr, any, call. A record with no machine-checkable pattern gives Unknown.
5. Match	std::regex_match() of the whole statement against each pattern. First match gives Valid (the mode that matched is reported). If none matches the result is Invalid and the result carries expected_patterns, the list that is printed in the error message.

For a whole script `check_script()` first calls `split_statements_by_semicolon()`, which cuts the text into statements at ";" while ignoring semicolons inside strings, comments and `{ ... }` blocks, and then checks each statement as described above.

Example: why the table has to follow the library

Before StorageLib got indices, the record was `"storage.Sum(storage_name)(target_var)"`. The call `storage.Sum(s)(1)(2)(res);` has two parentheses too many for this pattern, so step 5 returned Invalid and checksyntax stopped the script, although StorageLib itself accepts the call. The fix is in the table only:

```
before:  "call": [ "storage.Sum(storage_name)(target_var)" ]
after:   "call": [ "storage.Sum(storage_name)[(idx1)...](target_var)"
]
```

The same notation `[(idx1)...]` is used in all StorageLib records. A new function that was added to a library but not to the table is reported as Unknown or Invalid; as a temporary measure run `checksyntax no`.

Adding a command

1. Add a record to the JSON in DefaultSyntaxTable.h. For a library method the key is Library.Method, kind is "lib", and the pattern goes into "call":

```
 "Storage.PushRow": {"group": "Storage","kind": "lib","call":
["storage.PushRow(storage_name)[(idx1)...]"]},
```

2. Rebuild. 3. Check the table without starting the interpreter, for example in a small test program:

```
auto& chk = forestppl::SyntaxChecker::instance();
auto r = chk.IsValidCommandSyntax("storage.PushRow(s)(0)");
// r.valid == forestppl::Validity::Valid
auto bad = chk.IsValidCommandSyntax("storage.Sum(s)");
// bad.valid == Invalid, bad.expected_patterns lists the patterns of
storage.Sum
```

4. Put at least one valid and one invalid call of the new form into a test script. When a correct statement is rejected, the error message prints the expected patterns of the record: compare them with the statement character by character (missing optional part, wrong number of parentheses, a name used as value).

Examples of code

```
//Sieve of Eratosthenes
//rc Examples\scr\sieve3.scr
var m = 1000;
showcode no;
var len;
len = m + 1;
array primes[len];

for (k, 0, len)
{
  primes[k] = k;
}

primes[1] = 0;

start;
  for (n, 2, len)
  {
    if(n * n >= len)
       break;

    if (primes[n] != 0)
    {
      var start_j = n * n;

      for(j, start_j, len, n)
      {
        primes[j] = 0;
      }
    }
  }
stop("1. duration = ");
```

```
start;
// not write items with zero-value
writearray primes row false;
stop("2. duration = ");

start;
array dst;
//copy to array dst items with non zero value only
//array.filtercopy(primes, dst);
array.filtercopy primes, dst;

writearray dst row;
stop("3. duration = ");
```

```
>rc examples\scr\sieve3.scr
1. duration = 1.2613 msec
-----Array [primes]-----
2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61,
67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137,
139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211,
223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283,
293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379,
383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461,
463, 467, 479, 487, 491, 499, 503, 509, 521, 523, 541, 547, 557, 563,
569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 643,
647, 653, 659, 661, 673, 677, 683, 691, 701, 709, 719, 727, 733, 739,
743, 751, 757, 761, 769, 773, 787, 797, 809, 811, 821, 823, 827, 829,
839, 853, 857, 859, 863, 877, 881, 883, 887, 907, 911, 919, 929, 937,
941, 947, 953, 967, 971, 977, 983, 991, 997
2. duration = 0.579 msec
-----Array [dst]-----
2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61,
67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137,
139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211,
223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283,
293, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379,
383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461,
463, 467, 479, 487, 491, 499, 503, 509, 521, 523, 541, 547, 557, 563,
569, 571, 577, 587, 593, 599, 601, 607, 613, 617, 619, 631, 641, 643,
647, 653, 659, 661, 673, 677, 683, 691, 701, 709, 719, 727, 733, 739,
743, 751, 757, 761, 769, 773, 787, 797, 809, 811, 821, 823, 827, 829,
839, 853, 857, 859, 863, 877, 881, 883, 887, 907, 911, 919, 929, 937,
941, 947, 953, 967, 971, 977, 983, 991, 997
3. duration = 0.7624 msec
```

Another faster solution when processing is in progress by C++ code:
>rc examples\scr\Erato.scr

```
import Erato;
start;
call Erato.Solve3(1000);
stop("duration = ");
```

Result:

```
>rc examples\scr\erato.scr
code: scr
Imported [Erato]
2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,83,89,
97,101,103,107,109,113,127,131,137,139,149,151,157,163,167,173,179,
181,191,193,197,199,211,223,227,229,233,239,241,251,257,263,269,271,
277,281,283,293,307,311,313,317,331,337,347,349,353,359,367,373,379,
383,389,397,401,409,419,421,431,433,439,443,449,457,461,463,467,479,
487,491,499,503,509,521,523,541,547,557,563,569,571,577,587,593,599,
601,607,613,617,619,631,641,643,647,653,659,661,673,677,683,691,701,
709,719,727,733,739,743,751,757,761,769,773,787,797,809,811,821,823,
827,829,839,853,857,859,863,877,881,883,887,907,911,919,929,937,941,
947,953,967,971,977,983,991,997
duration =  0.1103 msec
```

Here C++ code:

```
void Erato::Print(vector<int> v)
{
  std::stringstream ss;
  for (size_t i = 0; i < v.size(); ++i)
  {
    ss << v[i];
    if (i < v.size() - 1)
      ss << ",";
  }
  std::cout << ss.str() << std::endl;
}
// the fastest algorithm
// Processing data without Vector & print results with erasing 0-
elements after processing
bool Erato::FuncSolve3(vector<string> parameters, string& result,
Composite* node)
{
  const std::string function_name = "Erato.Solve3";
  if (parameters.size() != 1)
  {
    cerr << "Error: [" << function_name << "] wrong format" << endl;
    return false;
  }
```

```
  try
  {
    int vector_length = stoi(parameters[0]);
    vector<int> v(vector_length);
    for (int i = 0; i < vector_length; i++) v[i] = i;
    v[1] = 0;
    for (int n = 2; n * n < vector_length; n++)
    {
      if (v[n] != 0)
      {
        // the fastest algorithm
        for (int j = n * n; j < vector_length; j += n)
        {
          v[j] = 0;
        }
      }
    }
    v.erase(remove(v.begin(), v.end(), 0), v.end());
    Print(v);
  }
  catch (const exception& ex)
  {
    cerr << "Error: [" << function_name << "] ... " << ex.what()
            <<endl;
    return false;
  }

  return true;
}
```

Open console window:
cmd
and run:
Examples.bat
Callfunc.bat
Delegates.bat
Struct.bat
Dataframe.bat
ArrayFunc.bat
MatrixVector.bat
or
RunAll.bat => Log.txt
NumPPL.bat => Log_NumPPL.txt
RunRequestsAll.bat => LogRequests.txt
StorageLib.bat > LogStorageLib.txt
with numerous examples of code.

References

1. Crafting Interpreters by Robert Nystrom
2. Compilers: Principles, Techniques, and Tools (The Dragon Book) by Aho, Lam, Sethi, and Ullman
3. Structure and Interpretation of Computer Programs (SICP) by Harold Abelson and Gerald Jay Sussman
4. Programming Language Pragmatics by Michael L. Scott